arXiv:2205.14135v2 [cs.LG] 23 Jun 2022

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao[†], Daniel Y. Fu[†], Stefano Ermon[†], Atri Rudra[‡], and Christopher Ré[†]

[†]Department of Computer Science, Stanford University
[‡]Department of Computer Science and Engineering, University at Buffalo, SUNY
{trid,danfu}@cs.stanford.edu, ermon@stanford.edu, atri@buffalo.edu, chrismre@cs.stanford.edu

June 24, 2022

Abstract

Transformers are slow and memory-hungry on long sequences, since the time and memory complexity of self-attention are quadratic in sequence length. Approximate attention methods have attempted to address this problem by trading off model quality to reduce the compute complexity, but often do not achieve wall-clock speedup. We argue that a missing principle is making attention algorithms *IO-aware*—accounting for reads and writes between levels of GPU memory. We propose FlashAttention, an IO-aware exact attention algorithm that uses tiling to reduce the number of memory reads/writes between GPU high bandwidth memory (HBM) and GPU on-chip SRAM. We analyze the IO complexity of FlashAttention, showing that it requires fewer HBM accesses than standard attention, and is optimal for a range of SRAM sizes. We also extend FlashAttention to block-sparse attention, yielding an approximate attention algorithm that is faster than any existing approximate attention method. FlashAttention trains Transformers faster than existing baselines: 15% end-to-end wall-clock speedup on BERT-large (seq. length 512) compared to the MLPerf 1.1 training speed record, 3× speedup on GPT-2 (seq. length 1K), and 2.4× speedup on long-range arena (seq. length 1K-4K). FlashAttention and block-sparse FlashAttention enable longer context in Transformers, yielding higher quality models (0.7 better perplexity on GPT-2 and 6.4 points of lift on long-document classification) and entirely new capabilities: the first Transformers to achieve better-than-chance performance on the Path-X challenge (seq. length 16K, 61.4% accuracy) and Path-256 (seq. length 64K, 63.1% accuracy).

1 Introduction

Transformer models [82] have emerged as the most widely used architecture in applications such as natural language processing and image classification. Transformers have grown larger [5] and deeper [83], but equipping them with longer context remains difficult [80], since the self-attention module at their heart has time and memory complexity quadratic in sequence length. An important question is whether making attention faster and more memory-efficient can help Transformer models address their runtime and memory challenges for long sequences.

Many approximate attention methods have aimed to reduce the compute and memory requirements of attention. These methods range from sparse-approximation [51, 74] to low-rank approximation [12, 50, 84], and their combinations [3, 9, 92]. Although these methods reduce the compute requirements to linear or near-linear in sequence length, many of them do not display wall-clock speedup against standard attention and have not gained wide adoption. One main reason is that they focus on FLOP reduction (which may not correlate with wall-clock speed) and tend to ignore overheads from memory access (IO).

In this paper, we argue that a missing principle is making attention algorithms *IO-aware* [1]—that is, carefully accounting for reads and writes to different levels of fast and slow memory (e.g., between fast GPU on-chip SRAM and relatively slow GPU high bandwidth memory, or HBM [45], Figure 1 left). On modern

arXiv:2205.14135v2 [cs.LG] 23 Jun 2022

FlashAttention算法：具备I/O感知能力的快速且内存效率高的精确注意力机制

Tri Dao[†]、Daniel Y. Fu[†]、Stefano Ermon[†]、Atri Rudra[‡]以及Christopher Ré [†]

[†]斯坦福大学计算机科学系
[‡]布法罗大学计算机科学与工程系，邮箱地址为{trid,danfu}@cs.stanford.edu、ermon@stanford.edu、atri@buffalo.edu、chrismre@cs.stanford.edu

2022年6月24日

摘要

Transformers are slow and memory-hungry on long sequences, since the time and memory complexity of self-attention are quadratic in sequence length. Approximate attention methods have attempted to address this problem by trading off model quality to reduce the compute complexity, but often do not achieve wall-clock speedup. We argue that a missing principle is making attention algorithms *IO-aware*—accounting for reads and writes between levels of GPU memory. We propose FlashAttention, an IO-aware exact attention algorithm that uses tiling to reduce the number of memory reads/writes between GPU high bandwidth memory (HBM) and GPU on-chip SRAM. We analyze the IO complexity of FlashAttention, showing that it requires fewer HBM accesses than standard attention, and is optimal for a range of SRAM sizes. We also extend FlashAttention to block-sparse attention, yielding an approximate attention algorithm that is faster than any existing approximate attention method. FlashAttention trains Transformers faster than existing baselines: 15% end-to-end wall-clock speedup on BERT-large (seq. length 512) compared to the MLPerf 1.1 training speed record, 3× speedup on GPT-2 (seq. length 1K), and 2.4× speedup on long-range arena (seq. length 1K-4K). FlashAttention and block-sparse FlashAttention enable longer context in Transformers, yielding higher quality models (0.7 better perplexity on GPT-2 and 6.4 points of lift on long-document classification) and entirely new capabilities: the first Transformers to achieve better-than-chance performance on the Path-X challenge (seq. length 16K, 61.4% accuracy) and Path-256 (seq. length 64K, 63.1% accuracy).

1 引言

[82] Transformer模型已成为自然语言处理和图像分类等应用中最广泛使用的架构。这些模型的规模 [5] 越来越大，深度 [83]，也越来越深，但要为其支持更长的上下文仍然十分困难 [80]，，因为其核心的自注意力机制的运行时间和内存复杂度会随着序列长度的增加而呈二次方增长。一个重要的问题是：能否通过提升注意力的运算速度并降低其内存占用，帮助Transformer模型解决处理长序列时所面临的运行时间和内存限制问题。

许多近似注意力机制都旨在降低注意力机制的运算量和内存需求。这类方法包括稀疏近似 [51, 74] 、低秩近似 [12, 50, 84],以及两者的组合 [3, 9, 92]。尽管这些方法能将运算量降至与序列长度成线性或接近线性关系，但其中很多方法在实际运行速度上并未展现出相较于标准注意力机制的显著提升，也因此未能得到广泛应用。其中一个主要原因在于，它们侧重于减少浮点运算次数（而这可能与实际运行速度并无直接关联），同时还往往忽视了内存访问所带来的开销。

在本文中，我们认为目前缺失的一个关键原则正在阻碍注意力算法实现*IO*感知 [1]——也就是说，未能妥善处理对不同层级快速与慢速内存的读写操作（例如，在速度较快的GPU片上静态随机存取存储器与相对慢速的GPU高带宽内存或HBM之间 [45], 见图1左侧）。在现代系统中

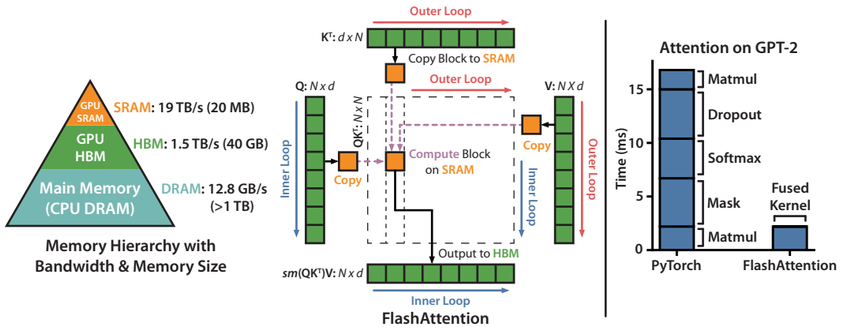


Figure 1: **Left:** FlashAttention uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FlashAttention loops through blocks of the **K** and **V** matrices and loads them to fast on-chip SRAM. In each block, FlashAttention loops over blocks of **Q** matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FlashAttention does not read and write the large $N \times N$ attention matrix to HBM, resulting in an 7.6× speedup on the attention computation.

GPUs, compute speed has out-paced memory speed [61, 62, 63], and most operations in Transformers are bottlenecked by memory accesses [43]. IO-aware algorithms have been critical for similar memory-bound operations, when reading and writing data can account for a large portion of the runtime—such as database joins [71], image processing [70], numerical linear algebra [4], and more [40, 85]. However, common Python interfaces to deep learning such as PyTorch and Tensorflow do not allow fine-grained control of memory access.

We propose FlashAttention, a new attention algorithm that computes exact attention with far fewer memory accesses. Our main goal is to avoid reading and writing the attention matrix to and from HBM. This requires (i) computing the softmax reduction without access to the whole input (ii) not storing the large intermediate attention matrix for the backward pass. We apply two well-established techniques to address these challenges. (i) We restructure the attention computation to split the input into blocks and make several passes over input blocks, thus incrementally performing the softmax reduction (also known as **tiling**). (ii) We store the softmax normalization factor from the forward pass to quickly **recompute** attention on-chip in the backward pass, which is faster than the standard approach of reading the intermediate attention matrix from HBM. We implement FlashAttention in CUDA to achieve fine-grained control over memory access and fuse all the attention operations into one GPU kernel. Even with the increased FLOPs due to recomputation, our algorithm both **runs faster** (up to 7.6x on GPT-2 [67], Figure 1 right) and **uses less memory**—linear in sequence length—than standard attention, thanks to the massively reduced amount of HBM access.

We analyze the IO complexity [1] of FlashAttention, proving that it requires $O(N^2 d^2 M^{-1})$ HBM accesses where d is the head dimension and M is the size of SRAM, as compared to $\Omega(Nd + N^2)$ of standard attention. For typical values of d and M, FlashAttention requires many times fewer HBM accesses compared to standard attention (up to 9× fewer, as shown in Fig. 2). Moreover, we provide a lower bound, showing that no exact attention algorithm can asymptotically improve on the number of HBM accesses over all SRAM sizes.

We also show that FlashAttention can serve as a useful primitive for realizing the potential of approximate attention algorithms by overcoming their issues with memory access overhead. As a proof of concept, we implement block-sparse FlashAttention, a sparse attention algorithm that is 2-4× faster than even FlashAttention, scaling up to sequence length of 64k. We prove that block-sparse FlashAttention has better IO complexity than FlashAttention by a factor proportional to the sparsity ratio. We discuss further extensions to other operations (attention on multi-GPU, kernel regression, block-sparse matrix

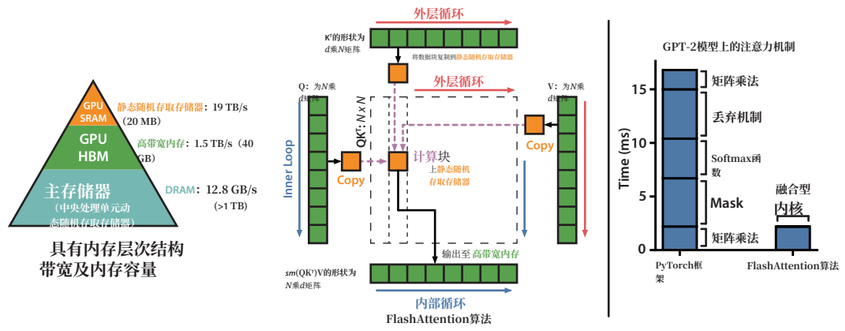


Figure 1: **Left:** FlashAttention uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FlashAttention loops through blocks of the **K** and **V** matrices and loads them to fast on-chip SRAM. In each block, FlashAttention loops over blocks of **Q** matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FlashAttention does not read and write the large $N \times N$ attention matrix to HBM, resulting in an 7.6× speedup on the attention computation.

GPU的运算速度已远远超过内存速度 [61, 62, 63], ，而Transformer中的大多数操作都因频繁的内存访问而成为瓶颈 [43]。对于那些存在严重内存受限问题的类似操作而言，具备IO感知能力的算法显得尤为重要，因为在这些场景中，数据的读写操作往往占据了运行时间的大部分——例如数据库连接 [71], 、图像处理 [70], 、数值线性代数 [4], 以及更多其他应用 [40, 85]。然而，像PyTorch和Tensorflow这类常见的深度学习Python接口，并不支持对内存访问进行精细控制。

We propose FlashAttention, a new attention algorithm that computes exact attention with far fewer memory accesses. Our main goal is to avoid reading and writing the attention matrix to and from HBM. This requires (i) computing the softmax reduction without access to the whole input (ii) not storing the large intermediate attention matrix for the backward pass. We apply two well-established techniques to address these challenges. (i) We restructure the attention computation to split the input into blocks and make several passes over input blocks, thus incrementally performing the softmax reduction (also known as **tiling**). (ii) We store the softmax normalization factor from the forward pass to quickly **recompute** attention on-chip in the backward pass, which is faster than the standard approach of reading the intermediate attention matrix from HBM. We implement FlashAttention in CUDA to achieve fine-grained control over memory access and fuse all the attention operations into one GPU kernel. Even with the increased FLOPs due to recomputation, our algorithm both **runs faster** (up to 7.6x on GPT-2 [67], Figure 1 right) and **uses less memory**—linear in sequence length—than standard attention, thanks to the massively reduced amount of HBM access.

我们分析了FlashAttention算法的输入输出复杂度 [1] ，证明与标准注意力机制的 $\Omega(Nd + N^2)$相比，它需要 $O(N^2 d^2 M^{-1})$次HBM访问；其中 d 为头维度， M 为静态随机存取存储器的容量。在 d 和 M取典型值时，FlashAttention所需的HBM访问次数远少于标准注意力机制（如图2所示，最多可少 9× 次）。此外，我们还给出了一个下界，表明在任何静态随机存取存储器容量下，都没有任何精确的注意力机制算法能在HBM访问次数上实现渐进式的优化。

我们还证明了FlashAttention算法能够作为一种实用的基元，通过解决传统近似注意力算法在内存访问方面的问题，从而充分释放这类算法的潜力。作为概念验证，我们实现了分块稀疏FlashAttention算法——这是一种稀疏注意力算法，其运算速度甚至比FlashAttention快2-4× 倍，且能够处理长度达64k的序列。研究表明，分块稀疏FlashAttention的输入输出复杂度优于FlashAttention，优化幅度与稀疏度比例成正比。此外，我们还探讨了将该算法进一步扩展应用于其他操作的可能性，包括多GPU环境下的注意力计算、内核回归以及分块稀疏矩阵相关任务。4× 我们已将FlashAttention开源，以便更多人能够基于这一基元进行开发。

multiply) in Section 5. We open-source FlashAttention to make it easier to build on this primitive.[1]

We empirically validate that FlashAttention speeds up model training and improves model quality by modeling longer context. We also benchmark the runtime and memory footprint of FlashAttention and block-sparse FlashAttention compared to prior attention implementations.

- **Faster Model Training.** FlashAttention trains Transformer models faster in wall-clock time. We train BERT-large (seq. length 512) 15% faster than the training speed record in MLPerf 1.1 [58], GPT2 (seq. length 1K) 3× faster than baseline implementations from HuggingFace [87] and Megatron-LM [77], and long-range arena (seq. length 1K-4K) 2.4× faster than baselines.
- **Higher Quality Models.** FlashAttention scales Transformers to longer sequences, which improves their quality and enables new capabilities. We observe a 0.7 improvement in perplexity on GPT-2 and 6.4 points of lift from modeling longer sequences on long-document classification [13]. FlashAttention enables the first Transformer that can achieve better-than-chance performance on the Path-X [80] challenge, solely from using a longer sequence length (16K). Block-sparse FlashAttention enables a Transformer to scale to even longer sequences (64K), resulting in the first model that can achieve better-than-chance performance on Path-256.
- **Benchmarking Attention.** FlashAttention is up to 3× faster than the standard attention implementation across common sequence lengths from 128 to 2K and scales up to 64K. Up to sequence length of 512, FlashAttention is both faster and more memory-efficient than any existing attention method, whereas for sequence length beyond 1K, some approximate attention methods (e.g., Linformer) start to become faster. On the other hand, block-sparse FlashAttention is faster than all existing approximate attention methods that we know of.

2 Background

We provide some background on the performance characteristics of common deep learning operations on modern hardware (GPUs). We also describe the standard implementation of attention.

2.1 Hardware Performance

We focus here on GPUs. Performance on other hardware accelerators are similar [46, 48].

GPU Memory Hierarchy. The GPU memory hierarchy (Fig. 1 left) comprises multiple forms of memory of different sizes and speeds, with smaller memory being faster. As an example, the A100 GPU has 40-80GB of high bandwidth memory (HBM) with bandwidth 1.5-2.0TB/s and 192KB of on-chip SRAM per each of 108 streaming multiprocessors with bandwidth estimated around 19TB/s [44, 45]. The on-chip SRAM is an order of magnitude faster than HBM but many orders of magnitude smaller in size. As compute has gotten faster relative to memory speed [61, 62, 63], operations are increasingly bottlenecked by memory (HBM) accesses. Thus exploiting fast SRAM becomes more important.

Execution Model. GPUs have a massive number of threads to execute an operation (called a kernel). Each kernel loads inputs from HBM to registers and SRAM, computes, then writes outputs to HBM.

Performance characteristics. Depending on the balance of computation and memory accesses, operations can be classified as either compute-bound or memory-bound. This is commonly measured by the *arithmetic intensity* [85], which is the number of arithmetic operations per byte of memory access.

1. Compute-bound: the time taken by the operation is determined by how many arithmetic operations there are, while time accessing HBM is much smaller. Typical examples are matrix multiply with large inner dimension, and convolution with large number of channels.
2. Memory-bound: the time taken by the operation is determined by the number of memory accesses, while time spent in computation is much smaller. Examples include most other operations: elementwise (e.g., activation, dropout), and reduction (e.g., sum, softmax, batch norm, layer norm).

Kernel fusion. The most common approach to accelerate memory-bound operations is kernel fusion: if there are multiple operations applied to the same input, the input can be loaded once from HBM, instead of multiple times for each operation. Compilers can automatically fuse many elementwise operations [53, 65, 75].

[1] FlashAttention code is available at `https://github.com/HazyResearch/flash-attention`

在第<code>multiply</code>节中 我们已将FlashAttention开源，以便更多人能够基于这一基元进行开发。

通过实验证明，FlashAttention算法能够通过建模更长的上下文来加速模型训练并提升模型质量。同时，我们还对比了FlashAttention算法与分块稀疏FlashAttention算法以及以往的注意力机制在运行时间和内存占用方面的表现。

- **更快速的模型训练**。FlashAttention算法能让Transformer模型的训练在实际时间上更快完成。我们使用该算法训练BERT-large模型（序列长度为512）时，其训练速度比MLPerf 1测试中的最快纪录快15%；1 [58],GPT2模型（序列长度为1K）的训练速度则 3× 比HuggingFace及Megatron-LM提供的基准实现快， [87] 同时也在长距离任务集（序列长度为1K-4K）的测试中 [77],比相应基准实现快2倍4× 。
- **更高品质的模型**。FlashAttention算法能够让Transformer模型处理更长的序列，从而提升模型质量并赋予其新的功能。在GPT-2模型上，该算法使困惑度降低了0.7；而在长文档分类 [13]任务中，通过处理更长的序列，模型的性能提升了6.4分。FlashAttention算法使得首个Transformer模型仅通过使用更长的序列长度（16K），便能在Path-X [80] 挑战赛中取得优于随机水平的性能。而分块稀疏FlashAttention算法则进一步让Transformer模型能够处理更长的序列（64K），进而诞生了首个在路径256挑战赛中也能实现优于随机水平性能的模型。
- **注意力机制性能测试**。在128到2K的常见序列长度范围内，且序列长度可扩展至64K时，FlashAttention算法的运行速度比标准注意力机制实现快达 3× 倍。在序列长度为512的情况下，FlashAttention不仅速度更快，而且内存利用率也高于所有现有的注意力机制；而当序列长度超过1K时，某些近似注意力机制（如Linformer）的速度会开始变得更快。另一方面，分块稀疏FlashAttention算法的速度则优于我们所知的所有现有近似注意力机制。

2 背景

我们首先介绍现代硬件（GPU）上常见深度学习运算的性能特点相关背景知识，随后还会阐述注意力机制的标准实现方式。

2.1 硬件性能

We 此处重点讨论GPU。其他硬件加速器的性能与之类似 [46, 48]。

GPU Memory Hierarchy. The GPU memory hierarchy (Fig. 1 left) comprises multiple forms of memory of different sizes and speeds, with smaller memory being faster. As an example, the A100 GPU has 40-80GB of high bandwidth memory (HBM) with bandwidth 1.5-2.0TB/s and 192KB of on-chip SRAM per each of 108 streaming multiprocessors with bandwidth estimated around 19TB/s [44, 45]. The on-chip SRAM is an order of magnitude faster than HBM but many orders of magnitude smaller in size. As compute has gotten faster relative to memory speed [61, 62, 63], operations are increasingly bottlenecked by memory (HBM) accesses. Thus exploiting fast SRAM becomes more important.

执行模型。GPU拥有海量线程，用于执行某一操作（即内核）。每个内核都会将输入数据从高带宽内存加载到寄存器及静态随机存取存储器中，进行计算后，再将输出结果写回高带宽内存。

性能特性。根据计算与内存访问之间的平衡程度，各种运算可以被划分为计算受限型或内存受限型。这一特性通常通过算术强度 [85], 来衡量，它表示每次内存访问所对应的算术运算次数。

1. 计算受限型：此类操作所需的时间取决于其中的算术运算次数，而访问高带宽内存所需的时间则要短得多。典型的例子包括具有较大内部维度的矩阵乘法，以及通道数众多的卷积操作。

2. 内存受限型：该操作所需的耗时主要由内存访问次数决定，而实际计算所花费的时间则要短得多。大多数其他操作都属于此类，例如逐元素运算（如激活函数、丢弃机制）以及聚合运算（如求和、Softmax函数、批量归一化、层归一化）。

内核融合。加速内存受限型运算的最常见方法就是内核融合：当有多个运算作用于同一输入数据时，只需从高带宽内存中读取一次该数据，而无需为每个运算分别读取。编译器能够自动对许多逐元素运算进行内核融合 [53, 65, 75]。

[1] FlashAttention算法的代码可在https://github.com/HazyResearch/flash-attention获取。

However, in the context of model training, the intermediate values still need to be written to HBM to save for the backward pass, reducing the effectiveness of naive kernel fusion.

2.2 Standard Attention Implementation

Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ where N is the sequence length and d is the head dimension, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where softmax is applied row-wise.

Standard attention implementations materialize the matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM, which takes $O(N^2)$ memory. Often $N \gg d$ (e.g., for GPT2, $N = 1024$ and $d = 64$). We describe the standard attention implementation in Algorithm 0. As some or most of the operations are memory-bound (e.g., softmax), the large number of memory accesses translates to slow wall-clock time.

This problem is exacerbated by other elementwise operations applied to the attention matrix, such as masking applied to $\mathbf{S}$ or dropout applied to $\mathbf{P}$. As a result, there have been many attempts to fuse several elementwise operations, such as fusing masking with softmax [77].

In Section 3.2, we will show that the standard attention implementation performs HBM accesses quadratic in the sequence length N. We also compare the number of FLOPs and number of HBM accesses of standard attention and of our method (FlashAttention).

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

3 FlashAttention: Algorithm, Analysis, and Extensions

We show how to compute exact attention with fewer HBM reads/writes and without storing large intermediate matrices for the backward pass. This yields an attention algorithm that is both memory efficient and faster in wall-clock time. We analyze its IO complexity, showing that our method requires much fewer HBM accesses compared to standard attention. We further show that FlashAttention can serve as a useful primitive by extending it to handle block-sparse attention.

We focus here on the forward pass for ease of exposition; Appendix B contains details for the backward.

3.1 An Efficient Attention Algorithm With Tiling and Recomputation

Given the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, we aim to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$ and write it to HBM. Our goal is to reduce the amount of HBM accesses (to sub-quadratic in N).

We apply two established techniques (tiling, recomputation) to overcome the technical challenge of computing exact attention in sub-quadratic HBM accesses. We describe this in Algorithm 1. The main idea is that we split the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into blocks, load them from slow HBM to fast SRAM, then compute the attention output with respect to those blocks. By scaling the output of each block by the right normalization factor before adding them up, we get the correct result at the end.

Tiling. We compute attention by blocks. Softmax couples columns of $\mathbf{K}$, so we decompose the large softmax with scaling [51, 60, 66]. For numerical stability, the softmax of vector $x \in \mathbb{R}^B$ is computed as:

$$m(x) := \max_i \; x_i, \quad f(x) := \begin{bmatrix} e^{x_1 - m(x)} & \dots & e^{x_B - m(x)} \end{bmatrix}, \quad \ell(x) := \sum_i f(x)_i, \quad \mathrm{softmax}(x) := \frac{f(x)}{\ell(x)}.$$

然而在模型训练的场景下，这些中间计算结果仍需被写入高带宽内存以便在反向传播时使用，这就降低了简单核融合操作的实际效果。

2.2 StandardAttention的实现方式

给定输入序列 $\mathbf{Q}$、$\mathbf{K}$以及$\mathbf{V} \in \mathbb{R}^{N \times d}$，其中 N表示序列长度， d表示头维度，我们需要计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

其中Softmax函数是按行应用的。

标准注意力机制的实现方式是将$\mathbf{S}$ 和$\mathbf{P}$ 矩阵加载到高带宽内存中，这一操作需要 $O(N^2)$的内存容量。通常情况下 $N \gg d$ （例如在GPT2模型中， $N = 1024$ 和 $d = 64$）。我们在算法0中对标准注意力机制的实现进行了描述。由于部分或大部分运算都属于内存受限型操作（如Softmax函数），大量的内存访问会导致实际运行时间显著变长。

当对注意力矩阵施加其他逐元素运算时，这一问题会愈发严重，例如对<span style id='1'>S进行的掩码处理，或是对<span style id='3'>P应用的丢弃机制。正因如此，人们提出了许多将多种逐元素运算融合在一起的方案，比如将掩码处理与Softmax函数 [77]相结合。

在3.2节中，我们将证明标准注意力机制的实现会随着序列长度 N的增加而产生与序列长度二次方成正比的HBM访问次数。同时，我们还会对比标准注意力机制以及我们的方法（FlashAttention算法）的浮点运算次数与HBM访问次数的差异。

算法0标准注意力机制的实现方式

需要： 位于高带宽内存中的 $\mathbf{Q}$、$\mathbf{K}$以及$\mathbf{V} \in \mathbb{R}^{N \times d}$这些矩阵。

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

3 FlashAttention：算法、分析及扩展

我们展示了如何通过减少HBM的读写操作，并且在后向传播时无需存储庞大的中间矩阵，从而实现精确的注意力机制计算。这样一来，所得到的注意力算法不仅具备较高的内存效率，其实际运行速度也更快。我们分析了该算法的输入输出复杂度，结果表明与标准注意力机制相比，我们的方法所需的HBM访问次数要少得多。此外，我们还证明了通过扩展FlashAttention算法以支持分块稀疏注意力机制，它能够作为一种实用的底层算子被广泛应用。

我们在此重点探讨 为便于阐述，此处介绍前向传播过程；附录B则包含了关于反向传播的详细内容。 .

3.1 A基于平铺技术与重计算的另一种高效注意力算法 n

给定存储在HBM中的输入$\mathbf{Q}$、$\mathbf{K}$以及$\mathbf{V} \in \mathbb{R}^{N \times d}$，我们的目标是计算注意力输出$\mathbf{O} \in \mathbb{R}^{N \times d}$并将其写入HBM。我们的目标是将HBM访问次数降至 N的次二次方级别以内。

我们采用了两种成熟的技术（平铺技术与重新计算）来克服在次二次方级的HBM访问次数下计算精确注意力机制所带来的技术难题。算法1中对这一方法进行了详细阐述。其核心思路是将输入的$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$ 分割成多个块，从速度较慢的HBM中将这些块加载到速度更快的静态随机存取存储器中，随后针对这些块计算注意力输出。通过在将各块的输出相加之前用适当的归一化因子对它们进行缩放，最终即可得到正确的结果。

平铺技术。我们通过分块的方式来计算注意力机制。Softmax函数用于处理$\mathbf{K}$矩阵的各列，因此我们通过缩放因子[51, 60, 66]来分解这个庞大的Softmax函数。为保证数值稳定性，向量 $x \in \mathbb{R}^B$ 的Softmax函数会按如下方式计算:

$$m(x) := \max_i \; x_i, \quad f(x) := \begin{bmatrix} e^{x_1 - m(x)} & \dots & e^{x_B - m(x)} \end{bmatrix}, \quad \ell(x) := \sum_i f(x)_i, \quad \mathrm{softmax}(x) := \frac{f(x)}{\ell(x)}.$$

For vectors $x^{(1)}, x^{(2)} \in \mathbb{R}^B$, we can decompose the softmax of the concatenated $x = \left[x^{(1)}\ x^{(2)}\right] \in \mathbb{R}^{2B}$ as:

$$m(x) = m(\left[x^{(1)}\ x^{(2)}\right]) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \left[e^{m(x^{(1)})-m(x)} f(x^{(1)}) \quad e^{m(x^{(2)})-m(x)} f(x^{(2)})\right],$$

$$\ell(x) = \ell(\left[x^{(1)}\ x^{(2)}\right]) = e^{m(x^{(1)})-m(x)}\ell(x^{(1)}) + e^{m(x^{(2)})-m(x)}\ell(x^{(2)}), \quad \text{softmax}(x) = \frac{f(x)}{\ell(x)}.$$

Therefore if we keep track of some extra statistics $(m(x), \ell(x))$, we can compute softmax one block at a time.[2] We thus split the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into blocks (Algorithm 1 line 3), compute the softmax values along with extra statistics (Algorithm 1 line 10), and combine the results (Algorithm 1 line 12).

Recomputation. One of our goals is to not store $O(N^2)$ intermediate values for the backward pass. The backward pass typically requires the matrices $\mathbf{S}, \mathbf{P} \in \mathbb{R}^{N \times N}$ to compute the gradients with respect to $\mathbf{Q}, \mathbf{K}, \mathbf{V}$. However, by storing the output $\mathbf{O}$ and the softmax normalization statistics (m, ℓ), we can recompute the attention matrix $\mathbf{S}$ and $\mathbf{P}$ easily in the backward pass from blocks of $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ in SRAM. This can be seen as a form of selective gradient checkpointing [10, 34]. While gradient checkpointing has been suggested to reduce the maximum amount of memory required [66], all implementations (that we know off) have to trade speed for memory. In contrast, even with more FLOPs, our recomputation speeds up the backward pass due to reduced HBM accesses (Fig. 2). The full backward pass description is in Appendix B.

Implementation details: Kernel fusion. Tiling enables us to implement our algorithm in one CUDA kernel, loading input from HBM, performing all the computation steps (matrix multiply, softmax, optionally masking and dropout, matrix multiply), then write the result back to HBM (masking and dropout in Appendix B). This avoids repeatedly reading and writing of inputs and outputs from and to HBM.

Algorithm 1 FlashAttention

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

1: Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil, B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$.
2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
3: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.
5: **for** $1 \le j \le T_c$ **do**
6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
7: **for** $1 \le i \le T_r$ **do**
8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
14: **end for**
15: **end for**
16: Return $\mathbf{O}$.

We show FlashAttention's correctness, runtime, and memory requirement (proof in Appendix C).

Theorem 1. *Algorithm 1 returns* $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$ *with* $O(N^2 d)$ *FLOPs and requires* $O(N)$ *additional memory beyond inputs and output.*

3.2 Analysis: IO Complexity of FlashAttention

We analyze the IO complexity of FlashAttention, showing significant reduction in HBM accesses compared to standard attention. We also provide a lower bound, proving that no exact attention algorithm can

[2] This style of aggregation is called *algebraic aggregation* [33].

For vectors $x^{(1)}, x^{(2)} \in \mathbb{R}^B$, we can decompose the softmax of the concatenated $x = \left[x^{(1)}\ x^{(2)}\right] \in \mathbb{R}^{2B}$ as:

$$m(x) = m(\left[x^{(1)}\ x^{(2)}\right]) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \left[e^{m(x^{(1)})-m(x)} f(x^{(1)}) \quad e^{m(x^{(2)})-m(x)} f(x^{(2)})\right],$$

$$\ell(x) = \ell(\left[x^{(1)}\ x^{(2)}\right]) = e^{m(x^{(1)})-m(x)}\ell(x^{(1)}) + e^{m(x^{(2)})-m(x)}\ell(x^{(2)}), \quad \text{softmax}(x) = \frac{f(x)}{\ell(x)}.$$

因此，如果我们能够记录一些额外的统计信息（$m(x)$,$l(x)$），就可以逐个块地计算Softmax函数。<v6>于是我们将输入$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$ 分割成多个块（算法1的第3行），同时计算这些块的Softmax值以及相关的额外统计信息（算法1的第10行），最后再将所有结果整合起来（算法1的第12行）。

Recomputation. One of our goals is to not store $O(N^2)$ intermediate values for the backward pass. The backward pass typically requires the matrices $\mathbf{S}, \mathbf{P} \in \mathbb{R}^{N \times N}$ to compute the gradients with respect to $\mathbf{Q}, \mathbf{K}, \mathbf{V}$. However, by storing the output $\mathbf{O}$ and the softmax normalization statistics (m, ℓ), we can recompute the attention matrix $\mathbf{S}$ and $\mathbf{P}$ easily in the backward pass from blocks of $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ in SRAM. This can be seen as a form of selective gradient checkpointing [10, 34]. While gradient checkpointing has been suggested to reduce the maximum amount of memory required [66], all implementations (that we know off) have to trade speed for memory. In contrast, even with more FLOPs, our recomputation speeds up the backward pass due to reduced HBM accesses (Fig. 2). The full backward pass description is in Appendix B.

实现细节：内核融合。通过平铺技术，我们能够将该算法实现为一个CUDA内核，从高带宽内存中加载输入数据，执行所有的计算步骤（矩阵乘法、Softmax函数，可选的掩码处理与丢弃机制，以及再次的矩阵乘法），最后再将结果写回高带宽内存中（附录B中介绍了掩码处理与丢弃机制的具体实现方式）。这样一来就可以避免频繁地从高带宽内存中读取和写入输入输出数据。

算法1FLASH算法

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

1: Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil, B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$.
2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
3: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.
5: **for** $1 \le j \le T_c$ **do**
6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
7: **for** $1 \le i \le T_r$ **do**
8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
14: **end for**
15: **end for**
16: Return $\mathbf{O}$.

我们展示了FlashAttention的正确性、运行时间以及内存需求（证明见附录C）。

定理1。算法*1*能够以 $O(N^2 d)$ 次浮点运算次数计算出$\mathbf{O} = \text{Softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$ 的结果，同时除了输入和输出数据外还需 $O(N)$ 额外的内存。

3.2 分析：FlashAttention的输入输出复杂度

我们分析了FlashAttention算法的输入输出复杂度，发现与标准注意力机制相比，其HBM访问次数显著减少。此外，我们还给出了一个下界证明，表明没有任何标准的注意力机制算法能够在所有SRAM容量条件下都实现HBM访问次数的渐进式优化。相关证明内容详见附录C。

[2] 这种聚合方式被称为代数聚合 [33]。

Attention	Standard	FlashAttention
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

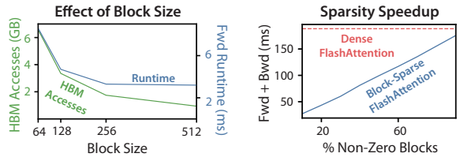


Figure 2: **Left**: Forward + backward runtime of standard attention and FlashAttention for GPT-2 medium (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. HBM access is the primary factor affecting runtime. **Middle**: Forward runtime of FlashAttention (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. Fewer HBM accesses result in faster runtime, up to a point. **Right**: The runtime (for seq. length 4K) of block-sparse FlashAttention is faster than FlashAttention by a factor proportional to the sparsity.

asymptotically improve on HBM accesses over all SRAM sizes. Proofs are in Appendix C.

Theorem 2. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. Standard attention (Algorithm 0) requires $\Theta(Nd + N^2)$ HBM accesses, while* FlashAttention *(Algorithm 1) requires $\Theta(N^2d^2M^{-1})$ HBM accesses.*

For typical values of d (64-128) and M (around 100KB), d^2 is many times smaller than M, and thus FlashAttention requires many times fewer HBM accesses than standard implementation. This leads to both faster execution and lower memory footprint, which we validate in Section 4.3.

The main idea of the proof is that given the SRAM size of M, we can load blocks of $\mathbf{K}, \mathbf{V}$ of size $\Theta(M)$ each (Algorithm 1 line 6). For each block of $\mathbf{K}$ and $\mathbf{V}$, we iterate over all blocks of $\mathbf{Q}$ (Algorithm 1 line 8) to compute the intermediate values, resulting in $\Theta(NdM^{-1})$ passes over $\mathbf{Q}$. Each pass loads $\Theta(Nd)$ elements, which amounts to $\Theta(N^2d^2M^{-1})$ HBM accesses. We similarly prove that the backward pass of standard attention requires $\Theta(Nd + N^2)$ HBM accesses while the backward pass of FlashAttention requires $\Theta(N^2d^2M^{-1})$ HBM accesses (Appendix B).

We prove a lower-bound: one cannot asymptotically improve on the number of HBM accesses for all values of M (the SRAM size) when computing exact attention.

Proposition 3. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. There does not exist an algorithm to compute exact attention with $o(N^2d^2M^{-1})$ HBM accesses for all M in the range $[d, Nd]$.*

The proof relies on the fact that for $M = \Theta(Nd)$ any algorithm must perform $\Omega(N^2d^2M^{-1}) = \Omega(Nd)$ HBM accesses. This type of lower bound over a subrange of M is common in the streaming algorithms literature [88]. We leave proving parameterized complexity [27] lower bounds in terms of M as exciting future work.

We validate that the number of HBM accesses is the main determining factor of attention run-time. In Fig. 2 (left), we see that even though FlashAttention has higher FLOP count compared to standard attention (due to recomputation in the backward pass), it has much fewer HBM accesses, resulting in much faster runtime. In Fig. 2 (middle), we vary the block size B_c of FlashAttention, which results in different amounts of HBM accesses, and measure the runtime of the forward pass. As block size increases, the number of HBM accesses decreases (as we make fewer passes over the input), and runtime decreases. For large enough block size (beyond 256), the runtime is then bottlenecked by other factors (e.g., arithmetic operations). Moreover, larger block size will not fit into the small SRAM size.

3.3 Extension: Block-Sparse FlashAttention

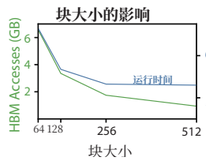
We extend FlashAttention to approximate attention: we propose block-sparse FlashAttention, whose IO complexity is smaller than FlashAttention by a factor proportional to the sparsity.

Given inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ and a mask matrix $\tilde{\mathbf{M}} \in \{0,1\}^{N \times N}$, we want to compute:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where $(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}})_{kl} = \mathbf{S}_{kl}$ if $\tilde{\mathbf{M}}_{kl} = 1$ and $-\infty$ if $\mathbf{M}_{kl} = 0$. We require $\tilde{\mathbf{M}}$ to have block form: for some block sizes B_r, B_c, for all k, l, $\tilde{\mathbf{M}}_{k,l} = \mathbf{M}_{ij}$ with $i = \lfloor k/B_r \rfloor, j = \lfloor l/B_c \rfloor$ for some $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$.

注意力机制	传统方法	FlashAttention算法
GFLOPs运算量	66.6	75.2
高带宽内存读写量（GB）	40.3	4.4
运行时间（毫秒）	41.7	7.3



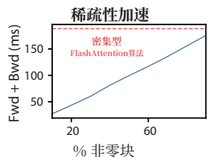


Figure 2: **Left**: Forward + backward runtime of standard attention and FlashAttention for GPT-2 medium (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. HBM access is the primary factor affecting runtime. **Middle**: Forward runtime of FlashAttention (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. Fewer HBM accesses result in faster runtime, up to a point. **Right**: The runtime (for seq. length 4K) of block-sparse FlashAttention is faster than FlashAttention by a factor proportional to the sparsity.

asy相关证明内容详见附录C。

定理2。设 N 为序列长度， d 为头维度， M 为具有 $d \le M \le Nd$的*SRAM*容量。标准注意力机制（算法*0*）需要 $\Theta(Nd + N^2)$ 次*HBM*访问，而FlashAttention算法（算法*1*）则需要 $\Theta(N^2d^2M^{-1})$ 次*HBM*访问。

在 d 为64-128、 M 约为100KB的典型数值情况下， d^2 的值远小于 M，因此FlashAttention算法所需的HBM访问次数也比传统实现少很多。这不仅使得计算速度更快，还能降低内存占用，我们将在第4.3节中对这一点进行验证。

The main idea of the proof is that given the SRAM size of M, we can load blocks of $\mathbf{K}, \mathbf{V}$ of size $\Theta(M)$ each (Algorithm 1 line 6). For each block of $\mathbf{K}$ and $\mathbf{V}$, we iterate over all blocks of $\mathbf{Q}$ (Algorithm 1 line 8) to compute the intermediate values, resulting in $\Theta(NdM^{-1})$ passes over $\mathbf{Q}$. Each pass loads $\Theta(Nd)$ elements, which amounts to $\Theta(N^2d^2M^{-1})$ HBM accesses. We similarly prove that the backward pass of standard attention requires $\Theta(Nd + N^2)$ HBM accesses while the backward pass of FlashAttention requires $\Theta(N^2d^2M^{-1})$ HBM accesses (Appendix B).

我们证明了这样一个下界：在计算精确注意力机制时，无论 M （即SRAM容量）取何数值，都无法在HBM访问次数方面实现渐进式的优化。

命题3。设 N 为序列长度， d 为头维度， M 为具有$d \le M \le Nd$的*SRAM*容量。目前还不存在一种算法，能够在 $[d, Nd]$范围内的所有 M情况下，仅通过 o次$(N^2d^2M^{-1})$ *HBM*访问次数即可计算出精确的注意力机制结果。

该证明的依据在于：对于 $M = \Theta(Nd)$，任何算法都必然需要执行 $\Omega(N^2d^2M^{-1}) = \Omega(Nd)$次HBM访问。在 M 的某个子范围内设定此类下界的情况，在流式算法相关文献中十分常见 [88]。我们将通过 M 来证明参数化复杂度 [27]的下界这一课题留作未来值得探索的重要工作。

我们验证了HBM访问次数是决定注意力运算时间的主要因素。在图2（左侧）中可以发现，尽管由于后向传播过程中的重新计算，FlashAttention的浮点运算次数高于标准注意力机制，但其HBM访问次数却要少得多，因此运行速度更快。在图2（中间）中，我们改变了FlashAttention的块大小 B_c ，这会导致不同的HBM访问量，同时我们也测量了前向传播的运行时间。随着块大小的增加，HBM访问次数会减少（因为对输入数据的传递次数变少了），进而运行时间也会降低。当块大小大到一定程度（超过256）时，运行时间就会受到其他因素的制约（例如算术运算）。此外，过大的块大小也无法容纳在有限的SRAM容量之中。

3.3 扩展：分块稀疏FlashAttention算法

我们将对FlashAttention算法进行扩展以实现注意力机制的近似计算，由此提出了分块稀疏FlashAttention算法；由于该算法具有与稀疏度成正比的更低输入输出复杂度，其性能优于原始FlashAttention算法。

给定输入 $\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V} \in \mathbb{R}^{N \times d}$ 以及一个掩码矩阵 $\tilde{\mathbf{M}} \in \{0、1\}^{N \times N}$，我们需要计算的是：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where $(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}})_{kl} = \mathbf{S}_{kl}$ if $\tilde{\mathbf{M}}_{kl} = 1$ and $-\infty$ if $\mathbf{M}_{kl} = 0$. We require $\tilde{\mathbf{M}}$ to have block form: for some block sizes B_r, B_c, for all k, l, $\tilde{\mathbf{M}}_{k,l} = \mathbf{M}_{ij}$ with $i = \lfloor k/B_r \rfloor, j = \lfloor l/B_c \rfloor$ for some $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$.

Given a predefined block sparsity mask $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$ we can easily adapt Algorithm 1 to only compute the nonzero blocks of the attention matrix. The algorithm is identical to Algorithm 1, except we skip zero blocks. We reproduce the algorithm description in Algorithm 5 in Appendix B.

We also analyze the IO complexity of block-sparse FlashAttention.

Proposition 4. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. Block-sparse* FlashAttention *(Algorithm 5) requires $\Theta(Nd + N^2d^2M^{-1}s)$ HBM accesses where s is the fraction of nonzero blocks in the block-sparsity mask.*

We see that applying block-sparsity yields a direct improvement by the sparsity to the larger term in the IO complexity. For large sequence lengths N, s is often set to $N^{-1/2}$ [11] or $N^{-1}\log N$ [3, 17, 92], resulting in $\Theta(N\sqrt{N})$ or $\Theta(N\log N)$ IO complexity. For downstream experiments, we use the fixed butterfly sparsity pattern [17], which has been shown to be able to approximate arbitrary sparsity [16].

In Fig. 2 (right), we validate that as the sparsity increases, the runtime of block-sparse FlashAttention improves proportionally. On the LRA benchmark, block-sparse FlashAttention achieves 2.8× speedup, while performing on par with standard attention (Section 4).

4 Experiments

We evaluate the impact of using FlashAttention to train Transformer models. We validate two claims about training time and model accuracy, and report attention runtime and memory benchmarks.

- **Training Speed.** FlashAttention outperforms the MLPerf 1.1 [58] speed record for BERT by 15%, and speeds up GPT-2 up to 3× over HuggingFace [87] and 1.8× over Megatron [77] over standard Transformers. FlashAttention speeds up the long-range arena (LRA) benchmark 2.4×.
- **Quality.** FlashAttention scales Transformers to longer sequences, yielding higher quality. FlashAttention trains GPT-2 with context length 4K faster than Megatron trains GPT-2 with context length 1K, while achieving 0.7 better perplexity. Modeling longer sequences yields 6.4 points of lift on two long-document classification tasks. Finally, FlashAttention yields the **first Transformer** that can achieve better-than-random performance on the challenging Path-X task (sequence length 16K), and block-sparse FlashAttention yields the **first sequence model** that we know of that can achieve better-than-random performance on Path-256 (sequence length 64K).
- **Benchmarking Attention.** We measure the runtime and memory performance of FlashAttention and block-sparse FlashAttention based on sequence length. We confirm that the memory footprint of FlashAttention scales linearly with seq. length and is up to 3× faster than standard attention for common seq. lengths (up to 2K). We confirm that runtime of block-sparse FlashAttention scales linearly in seq. length and is faster than all existing approximate attention baselines.

Additional experiment details are in Appendix E.

4.1 Faster Models with FlashAttention

BERT. FlashAttention yields the fastest single-node BERT training speed that we know of. We train a BERT-large [22] model with FlashAttention on Wikipedia. Table 1 compares our training time to the implementation from Nvidia that set the training speed record for MLPerf 1.1 [58]. Our implementation is 15% faster.

Table 1: Training time of BERT-large, starting from the same initialization provided by the MLPerf benchmark, to reach the target accuracy of 72.0% on masked language modeling. Averaged over 10 runs on 8×A100 GPUs.

BERT Implementation	Training time (minutes)
Nvidia MLPerf 1.1 [58]	20.0 ± 1.5
FlashAttention (ours)	**17.4** ± 1.4

GPT-2. FlashAttention yields faster training times for GPT-2 [67] on the large OpenWebtext dataset [32] than the widely used HuggingFace [87] and Megatron-LM [77] implementations. Table 2 shows up to 3× end-to-end speedup compared to Huggingface and 1.7× speedup compared to Megatron-LM. FlashAttention

鉴于存在一个预定义的块稀疏度掩码 $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$，我们能够轻松修改算法1，使其仅计算注意力矩阵中的非零块。该算法与算法1基本相同，不同之处在于它会跳过那些值为零的块。我们在附录B的算法5中重新给出了该算法的描述。

我们还分析了分块稀疏FlashAttention算法的输入输出复杂度。

命题4。设 N 为序列长度， d 为头维度， M 为带有$d \le M \le Nd$的*SRAM*容量。块稀疏FlashAttention（算法*5*）需要 $\Theta(Nd + N^2d^2M^{-1}s)$次*HBM*访问，其中 s 表示块稀疏掩码中非零块的比例。

我们可以看到，通过引入块稀疏性，能够直接利用这种稀疏特性来降低输入输出复杂度中占比最大的那部分数值。对于较长的序列长度而言， N, s通常被设定为 $N^{-1/2}$ [11]或 $N^{-1}\log N$ [3, 17, 92]，这样一来输入输出复杂度就会变为 $\Theta(N\sqrt{N})$或 $\Theta(N\log N)$。在后续实验中，我们采用了固定的蝴蝶形稀疏模式 [17]，，因为已有研究表明该模式能够近似实现任意形式的稀疏性 [16]。

在图2（右侧）中，我们验证了随着稀疏度的提升，分块稀疏FlashAttention算法的运行时间会呈比例改善。在长距离注意力基准测试上，该算法可实现2.8× 倍的加速效果，同时其性能与标准注意力机制相当（见第4节）。

4项实验

我们评估了使用FlashAttention训练Transformer模型的影响。我们对训练时间与模型准确率这两个方面的说法进行了验证，同时给出了注意力机制的运行时间及内存使用基准数据。

- **训练速度**。FlashAttention算法在BERT的MLPerf 1.1 [58] 速度测试中比现有最佳成绩快15%，同时能让GPT-2的运行速度比HuggingFace [87] 提供的版本快 3× ，相比标准Transformer模型则快1倍。8× 在与Megatron架构的对比中更是具备显著优势 [77] 。此外，该算法还能将长距离任务集（LRA）基准测试的运行速度提升2.4×。
- **Quality.** FlashAttention scales Transformers to longer sequences, yielding higher quality. FlashAttention trains GPT-2 with context length 4K faster than Megatron trains GPT-2 with context length 1K, while achieving 0.7 better perplexity. Modeling longer sequences yields 6.4 points of lift on two long-document classification tasks. Finally, FlashAttention yields the **first Transformer** that can achieve better-than-random performance on the challenging Path-X task (sequence length 16K), and block-sparse FlashAttention yields the **first sequence model** that we know of that can achieve better-than-random performance on Path-256 (sequence length 64K).
- **注意力机制基准测试**。我们根据序列长度来衡量FlashAttention及分块稀疏FlashAttention算法的运行时间与内存使用表现。实验结果表明，对于常见的序列长度（最高2K），FlashAttention的内存占用与序列长度呈线性增长关系，其速度比标准注意力机制快 3× 倍；同时，分块稀疏FlashAttention的运行时间也与序列长度成线性关系，且其速度优于所有现有的近似注意力算法基准。

更多实验细节详见附录E。

4.1 基于FlashAttention的更快模型

BERT。FlashAttention实现了我们所知最快的单节点BERT训练速度。我们使用FlashAttention在维基百科数据集上训练了一个BERT-large [22]模型。表1将我们的训练时间与曾创下MLPerf 1.1 [58]训练速度纪录的英伟达实现进行了对比，结果显示我们的实现速度快了15%。

表1：在采用MLPerf基准测试所规定的相同初始化条件的前提下，BERT-large模型通过掩码语言建模达到72.0%的目标准确率所需的训练时间。数据为在 8×台A100显卡上进行10次实验后的平均值。

BERT实现方案	训练时间（分钟）
英伟达MLPerf 1.1 [58]	20.0 ± 1.5
FlashAttention（我们开发的版本）	**17.4** ± 1.4

GPT-2。 在大型OpenWebText数据集 [32]上，FlashAttention算法用于训练GPT-2 [67]模型时，其训练时间比广泛使用的HuggingFace [87] 及Megatron-LM [77] 实现方式更快。表2显示，与HuggingFace相比，其速度可提升多达 3× 倍；与Megatron-LM相比，则可提升1.7× 倍。FlashAttention算法

achieves the same perplexity as the other two implementations, as we do not change the model definition. Appendix E includes plots of the validation perplexity throughout training, confirming that FlashAttention is as numerically stable as the baselines and produces the same training / validation curves.

Table 2: GPT-2 small and medium using FlashAttention achieve up to 3× speed up compared to Huggingface implementation and up to 1.7× compared to Megatron-LM. Training time reported on 8×A100s GPUs.

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FlashAttention	18.2	**2.7 days (3.5×)**
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FlashAttention	14.3	**6.9 days (3.0×)**

Long-range Arena. We compare vanilla Transformer (with either standard implementation or FlashAttention) on the long-range arena (LRA [80]) benchmark. We measure accuracy, throughput, and training time of all models. Each task has a different sequence length varying between 1024 and 4096. We follow the implementation and experimental setting in Tay et al. [80]and Xiong et al. [90].[3] Table 3 shows that FlashAttention achieves up 2.4× speed-up compared to standard attention. Block-sparse FlashAttention is faster than all of the approximate attention methods that we have tested.

Table 3: The performance of standard attention, FlashAttention, block-sparse FlashAttention, and approximate attention baselines on the Long-Range-Arena benchmarks.

Models	ListOps	Text	Retrieval	Image	Pathfinder	Avg	Speedup
Transformer	36.0	63.6	81.6	42.3	72.7	59.3	-
FlashAttention	37.6	63.9	81.4	43.5	72.7	59.8	2.4×
Block-sparse FlashAttention	37.0	63.0	81.3	43.6	73.3	59.6	**2.8**×
Linformer [84]	35.6	55.9	77.7	37.8	67.6	54.9	2.5×
Linear Attention [50]	38.8	63.2	80.7	42.6	72.5	59.6	2.3×
Performer [12]	36.8	63.6	82.2	42.1	69.9	58.9	1.8×
Local Attention [80]	36.1	60.2	76.7	40.6	66.6	56.0	1.7×
Reformer [51]	36.5	63.8	78.5	39.6	69.4	57.6	1.3×
Smyrf [19]	36.1	64.1	79.0	39.6	70.5	57.9	1.7×

4.2 Better Models with Longer Sequences

Language Modeling with Long Context. The runtime and memory-efficiency of FlashAttention allow us to increase the context length of GPT-2 by 4× while still running faster than the optimized implementation from Megatron-LM. Table 4 shows that that GPT-2 with FlashAttention and context length 4K is still 30% faster than GPT-2 from Megatron with context length 1K, while achieving 0.7 better perplexity.

Table 4: GPT-2 small with FlashAttention, with 4× larger context length compared to Megatron-LM, is still 30% faster while achieving 0.7 better perplexity. Training time on 8×A100 GPUs is reported.

Model implementations	Context length	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Megatron-LM	1k	18.2	4.7 days (1.0×)
GPT-2 small - FlashAttention	1k	18.2	**2.7 days (1.7×)**
GPT-2 small - FlashAttention	2k	17.6	3.0 days (1.6×)
GPT-2 small - FlashAttention	4k	**17.5**	3.6 days (1.3×)

Long Document Classification. Training Transformers with longer sequences with FlashAttention improves performance on the MIMIC-III [47] and ECtHR [6, 7] datasets. MIMIC-III contains intensive care unit patient discharge summaries, each annotated with multiple labels. ECtHR contains legal cases from the

[3]LRA accuracy results are known to be highly dependent on the tuning procedure [90]. Our reproduced baselines perform better than as reported in the original comparison [80].

由于我们并未更改模型定义，该实现所达到的困惑度与另外两种实现完全相同。附录E中展示了训练过程中的验证困惑度曲线，这些数据证实FlashAttention在数值稳定性方面与基准模型相当，且生成的训练/验证曲线也一致。表2显示：使用FlashAttention的GPT-2小型模型和中型模型的速度相比HuggingFace的实现可提升至 3× 倍，相比Megatron-LM则可提升至1.7× 倍。此处列出的训练时间均是在 8×颗A100显卡上测试所得。

模型实现方式	OpenWebText（ppl）	训练时间（加速比）
GPT-2小型模型 – HuggingFace [87]	18.2	9.5天（1.0×）
GPT-2小型模型 – Megatron-LM [77]	18.2	4.7天（2.0×）
GPT-2小型模型 – FlashAttention算法	18.2	**2.7天（3.5×）**
GPT-2中型模型 – HuggingFace [87]	14.2	21.0天（1.0×）
GPT-2中型模型 – Megatron-LM [77]	14.3	11.5天（1.8×）
GPT-2中型模型 - FlashAttention算法	14.3	**6.9天（3.0×）**

长距离任务集。我们在长距离任务集（LRA [80]）基准测试中，对比了采用标准实现或FlashAttention算法的普通Transformer模型。我们测量了所有模型的准确率、处理速度以及训练时间。每项任务的序列长度各不相同，介于1024到4096之间。我们的实现方式和实验设置参考了Tay等人 [80]以及Xiong等人 [90]、[3] 的研究成果。表3显示，与标准注意力机制相比，FlashAttention算法的运行速度可提升高达2.4× 倍；而分块稀疏FlashAttention算法的速度则优于我们测试过的所有近似注意力机制。

表3：在长距离竞技场基准测试上，标准注意力机制、FlashAttention算法、分块稀疏FlashAttention算法以及近似注意力机制基线模型的性能表现。

模型	ListOps	Text	检索模块	图像	路径查找器	Avg	加速效果
Transformer模型	36.0	63.6	81.6	42.3	72.7	59.3	-
FlashAttention算法	37.6	63.9	81.4	43.5	72.7	59.8	2.4×
分块稀疏FlashAttention算法	37.0	63.0	81.3	43.6	73.3	59.6	**2.8**×
Linformer模型 [84]	35.6	55.9	77.7	37.8	67.6	54.9	2.5×
线性注意力机制 [50]	38.8	63.2	80.7	42.6	72.5	59.6	2.3×
Performer模型 [12]	36.8	63.6	82.2	42.1	69.9	58.9	1.8×
局部注意力机制 [80]	36.1	60.2	76.7	40.6	66.6	56.0	1.7×
Reformer [51]	36.5	63.8	78.5	39.6	69.4	57.6	1.3×
Smyrf [19]	36.1	64.1	79.0	39.6	70.5	57.9	1.7×

4.2 适用于更长序列的BetterModels模型

长上下文的语言建模。FlashAttention算法的运行时间较短且内存占用较低，这使我们能够在保持速度优于Megatron-LM优化实现的同时，将GPT-2模型的上下文长度提升 4× 。表格4显示，采用FlashAttention算法且上下文长度为4K的GPT-2，其运行速度仍比上下文长度为1K的Megatron版GPT-2快30%，同时困惑度还降低了0.7。

表格4：采用FlashAttention算法的GPT-2小型模型虽然其 4× 上下文长度大于Megatron-LM，但其速度仍快30%，同时困惑度降低了0.7。此处列出了在 8×A100显卡上的训练时间数据。

模型实现	上下文长度	OpenWebText（数据量）	训练时间（加速比）
GPT-2小型模型 – Megatron-LM	1k	18.2	4.7天（1.0×）
GPT-2小型模型 – FlashAttention算法	1k	18.2	**2.7天（1.7×）**
GPT-2小型模型 – FlashAttention算法	2k	17.6	3.0天（1.6×）
GPT-2小型模型 – FlashAttention算法	4k	**17.5**	3.6天（1.3×）

长文档分类。使用FlashAttention对包含更长序列的Transformer模型进行训练，能够在MIMIC-III [47] 和欧洲人权法院 [6, 7] 数据集上提升性能。MIMIC-III数据集包含重症监护病房患者的出院摘要，每份摘要都带有多个标注标签；而欧洲人权法院数据集则包含相关的法律案例。

[3]，众所周知，长距离注意力的准确率结果在很大程度上取决于参数调优流程 [90]。我们重新实现的基准模型表现优于原始对比研究中所报告的结果 [80]。

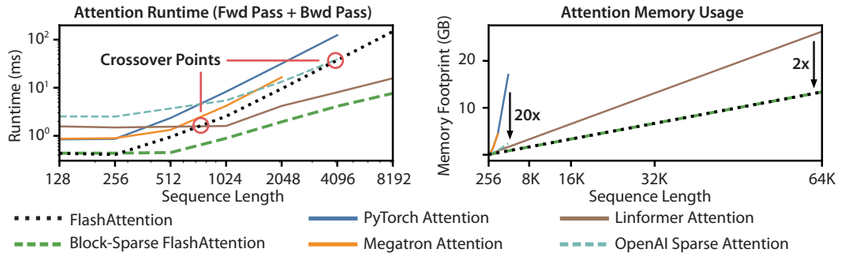


Figure 3: **Left:** runtime of forward pass + backward pass. **Right:** attention memory usage.

European Court of Human Rights, each of which is mapped to articles of the Convention of Human Rights that were allegedly violaged. Both of these datasets contain very long text documents; the average number of tokens in MIMIC is 2,395 tokens, and the longest document contains 14,562 tokens, while the average and longest numbers in ECtHR are 2,197 and 49,392, respectively. We evaluate lift from increasing the sequence length of a pretrained RoBERTa model [56] (we repeat the positional embeddings, as in Beltagy et al. [3]).

Table 5 shows that sequence length 16K outperforms length 512 by 4.3 points on MIMIC, and that length 8K outperforms length 512 by 8.5 points on ECtHR. The discrepancies may be due to subtle distribution shifts: MIMIC-III contains specialized medical text and thus may be more susceptible to a distribution shift in the document length, whereas ECtHR contains general language.

Table 5: Long Document performance (micro F_1) at different sequence lengths using FlashAttention.

	512	1024	2048	4096	8192	16384
MIMIC-III [47]	52.8	50.7	51.7	54.6	56.4	**57.1**
ECtHR [6]	72.2	74.3	77.1	78.6	**80.7**	79.2

Table 6: We report the first Transformer model that can achieve non-random performance on Path-X and Path-256.

Model	Path-X	Path-256
Transformer	✗	✗
Linformer [84]	✗	✗
Linear Attention [50]	✗	✗
Performer [12]	✗	✗
Local Attention [80]	✗	✗
Reformer [51]	✗	✗
SMYRF [19]	✗	✗
FlashAttention	**61.4**	✗
Block-sparse FlashAttention	56.0	**63.1**

Path-X and Path-256. The Path-X and Path-256 benchmarks are challenging tasks from the long-range arena benchmark designed to test long context. The task is to classify whether two points in a black and white 128×128 (or 256×256) image have a path connecting them, and the images are fed to the transformer one pixel at a time. In prior work, all transformer models have either run out of memory, or only achieved random performance [80]. There has been a search for alternative architectures that can model such long context [37]. We present here the first result of Transformer models being able to solve Path-X and Path-256 (Table 6). We pretrain a transformer on Path-64, and then transfer to Path-X by spatially interpolating the positional embeddings. FlashAttention achieves 61.4 accuracy on Path-X. Additionally, block-sparse FlashAttention enables the Transformers to scale to sequence length 64K, achieving 63.1 accuracy[4] on Path-256.

4.3 Benchmarking Attention

We vary sequence length and measure runtime and memory usage of FlashAttention and block-sparse FlashAttention against various attention baselines on one A100 GPU with 40 GB HBM, with dropout and a padding mask. We compare against reference implementations for exact attention, approximate attention, and sparse attention. We report a subset of baselines in the main body; Appendix E contains more baselines and full details.

[4]Path-256 requires longer sequences but has relatively shorter paths than Path-X, so it is easier to obtain a higher accuracy.

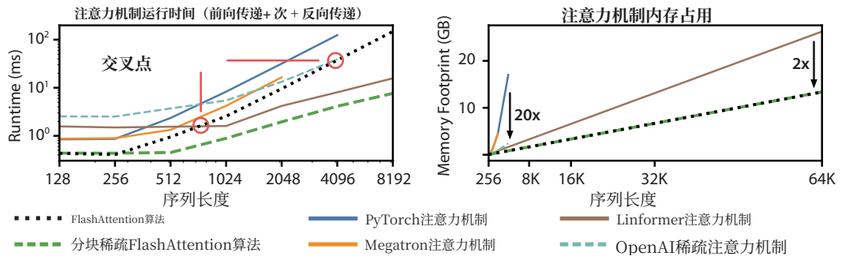


图3：**左侧显示：**前向传播的运行时间 + 以及后向传播的运行时间。**右侧显示：**注意力机制内存占用。

欧洲人权法院的判例，这些判例分别对应着据称被违反的《人权公约》中的相关条款。这两个数据集都包含长度极长的文本文档：MIMIC数据集中文本的平均标记数为2,395个，最长的文档则有14,562个标记；而ECtHR数据集的平均标记数和最长文档的标记数则分别为2,197个和49,392个。我们通过增加预训练好的RoBERTa模型 [56] 的序列长度来评估其性能提升效果（与Beltagy等人 [3]的做法一样，我们会重复使用位置嵌入）。

表格5显示，在MIMIC数据集上，序列长度为16K时的性能比512高4.3分；而在ECtHR数据集上，序列长度为8K时的性能则比512高8.5分。出现这种差异可能是由于数据分布的细微变化所致：MIMIC-III包含专业的医学文本，因此更容易受到文档长度变化的影响，而ECtHR则包含通用语言文本。

表格5：使用FlashAttention算法在不同序列长度下处理长文档（micro F_1）时的性能表现。

	512	1024	2048	4096	8192	16384
MIMIC-III [47]	52.8	50.7	51.7	54.6	56.4	**57.1**
ECtHR [6]	72.2	74.3	77.1	78.6	**80.7**	79.2

表格6：我们在此介绍了首个能够在Path-X和Path-256挑战赛中取得非随机性能水平的Transformer模型。

模型	Path-X	路径256
Transformer模型	✗	✗
Linformer [84]	✗	✗
线性注意力机制 [50]	✗	✗
Performer模型 [12]	✗	✗
局部注意力机制 [80]	✗	✗
Reformer [51]	✗	✗
SMYRF [19]	✗	✗
FlashAttention算法	**61.4**	✗
分块稀疏FlashAttention算法	56.0	**63.1**

Path-X and Path-256. The Path-X and Path-256 benchmarks are challenging tasks from the long-range arena benchmark designed to test long context. The task is to classify whether two points in a black and white 128×128 (or 256×256) image have a path connecting them, and the images are fed to the transformer one pixel at a time. In prior work, all transformer models have either run out of memory, or only achieved random performance [80]. There has been a search for alternative architectures that can model such long context [37]. We present here the first result of Transformer models being able to solve Path-X and Path-256 (Table 6). We pretrain a transformer on Path-64, and then transfer to Path-X by spatially interpolating the positional embeddings. FlashAttention achieves 61.4 accuracy on Path-X. Additionally, block-sparse FlashAttention enables the Transformers to scale to sequence length 64K, achieving 63.1 accuracy[4] on Path-256.

4.3 注意力机制的基准测试

我们在配备40 GB高带宽内存的A100 GPU上，通过改变序列长度，并结合丢弃机制与填充掩码，测试了FlashAttention算法及分块稀疏FlashAttention算法在运行时间与内存占用方面的表现，同时将其与多种不同的注意力机制基线模型进行对比。此外，我们还将其与精确注意力、近似注意力以及稀疏注意力的参考实现进行了比较。正文部分仅介绍了部分基线模型，更多基线模型及详细数据则收录在附录E中。

[4]Path-256需要更长的序列长度，但其路径长度相对Path-X更短，因此更容易获得更高的准确率。

Runtime. Figure 3 (left) reports the runtime in milliseconds of the forward + backward pass of FlashAttention and block-sparse FlashAttention compared to the baselines in exact, approximate, and sparse attention (exact numbers in Appendix E). Runtime grows quadratically with sequence length, but FlashAttention runs significantly faster than **exact attention** baselines, up to 3× faster than the PyTorch implementation. The runtimes of many approximate/sparse attention mechanisms grow linearly with sequence length, but FlashAttention still runs faster than approximate and sparse attention for short sequences due to fewer memory accesses. The **approximate attention** runtimes begin to cross over with FlashAttention at sequences between 512 and 1024. On the other hand, block-sparse FlashAttention is faster than all implementations of exact, sparse, and approximate attention that we know of, across all sequence lengths.

Memory Footprint. Figure 3 (right) shows the memory footprint of FlashAttention and block-sparse FlashAttention compared to various exact, approximate, and sparse attention baselines. FlashAttention and block-sparse FlashAttention have the same memory footprint, which grows linearly with sequence length. FlashAttention is up to 20× more memory efficient than **exact attention** baselines, and is more memory-efficient than the **approximate attention** baselines. All other algorithms except for Linformer run out of memory on an A100 GPU before 64K, and FlashAttention is still 2× more efficient than Linformer.

5 Limitations and Future Directions

We discuss limitations of our approach and future directions. Related work is given in Appendix A.

Compiling to CUDA. Our current approach to building IO-aware implementations of attention requires writing a new CUDA kernel for each new attention implementation. This requires writing the attention algorithm in a considerably lower-level language than PyTorch, and requires significant engineering effort. Implementations may also not be transferrable across GPU architectures. These limitations suggest the need for a method that supports writing attention algorithms in a high-level language (e.g., PyTorch), and compiling to IO-aware implementations in CUDA—similar to efforts such as Halide in image processing [70].

IO-Aware Deep Learning. We believe that the IO-aware approach can extend beyond attention. Attention is the most memory-intensive computation in Transformers, but every layer in a deep network touches GPU HBM. We hope our work inspires IO-aware implementations of additional modules. We discuss these potential extensions in Appendix D.

Multi-GPU IO-Aware Methods. Our IO-aware implementation of attention is optimal within constants for computing attention on a single GPU. However, the attention computation may be parallelizable across multiple GPUs [72]. Using multiple GPUs adds an additional layer to IO analysis—accounting for data transfer between GPUs. We hope our work inspires future work in this direction.

Acknowledgments

Our implementation uses Apex's FMHA code (`https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha`) as a starting point. We thank Young-Jun Ko for the in-depth explanation of his FMHA implementation and for his thoughtful answers to our questions about CUDA. We thank Sabri Eyuboglu, Megan Leszczynski, Laurel Orr, Yuhuai Wu, Beidi Chen, and Xun Huang for their constructive feedback and suggestions on early drafts of the paper. We thank Markus Rabe and Charles Staats for helpful discussion of their attention algorithm.

We gratefully acknowledge the support of NIH under No. U54EB020405 (Mobilize), NSF under Nos. CCF1763315 (Beyond Sparsity), CCF1563078 (Volume to Velocity), and 1937301 (RTML); ARL under No. W911NF-21-2-0251 (Interactive Human-AI Teaming); ONR under No. N000141712266 (Unifying Weak Supervision); ONR N00014-20-1-2480: Understanding and Applying Non-Euclidean Geometry in Machine Learning; N000142012275 (NEPTUNE); NXP, Xilinx, LETI-CEA, Intel, IBM, Microsoft, NEC, Toshiba, TSMC, ARM, Hitachi, BASF, Accenture, Ericsson, Qualcomm, Analog Devices, Google Cloud, Salesforce, Total, the HAI-GCP & HAI-Azure Cloud Credits for Research program, the Stanford Data Science Initiative (SDSI), Department of Defense (DoD) through the National Defense Science and Engineering Graduate Fellowship (NDSEG) Program, and members of the Stanford DAWN project: Facebook, Google, and VMWare. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes

Runtime. Figure 3 (left) reports the runtime in milliseconds of the forward + backward pass of FlashAttention and block-sparse FlashAttention compared to the baselines in exact, approximate, and sparse attention (exact numbers in Appendix E). Runtime grows quadratically with sequence length, but FlashAttention runs significantly faster than **exact attention** baselines, up to 3× faster than the PyTorch implementation. The runtimes of many approximate/sparse attention mechanisms grow linearly with sequence length, but FlashAttention still runs faster than approximate and sparse attention for short sequences due to fewer memory accesses. The **approximate attention** runtimes begin to cross over with FlashAttention at sequences between 512 and 1024. On the other hand, block-sparse FlashAttention is faster than all implementations of exact, sparse, and approximate attention that we know of, across all sequence lengths.

内存占用。图3（右侧）展示了FlashAttention与分块稀疏FlashAttention相较于各类精确、近似及稀疏注意力基线算法的内存占用情况。这两种算法的内存占用相同，且都会随序列长度呈线性增长。FlashAttention在内存效率上比**精确注意力**基线算法高出 20× 倍，同时也优于**近似注意力**基线算法。除Linformer之外的所有其他算法在处理长度小于64K的序列时就会在A100 GPU上出现内存不足的问题，而FlashAttention仍比Linformer更高效 2× 倍。

5项局限性及未来发展方向

我们探讨l　我们方法的仿制品以及未来的发展方向。相关研究内容见附录A。　　.

正在编译为CUDA格式。目前我们构建具有IO感知能力的注意力机制实现的方法是，每当有新的注意力机制需要实现时，就必须为它编写一个新的CUDA内核。这要求用比PyTorch低得多的层级语言来编写注意力算法，且需要投入大量的工程精力。此外，这样的实现往往无法在不同GPU架构之间迁移。这些限制表明，我们需要一种方法，能够支持用高级语言（如PyTorch）编写注意力算法，再将其编译为具备IO感知能力的CUDA实现——类似于图像处理领域中Halide工具所采用的思路 [70]。

面向I/O的深度学习。我们认为，基于注意力机制之外的面向I/O的方法还有更大的发展空间。在Transformer模型中，注意力机制是内存消耗最高的运算方式，但深度网络中的每一层都会使用GPU高带宽内存。我们希望我们的研究能够为其他模块的面向I/O实现提供灵感。我们在附录D中探讨了这些潜在的扩展方向。

多GPU注意力机制与IO感知方法。我们在单个GPU上实现的具有IO感知功能的注意力机制，在计算注意力时已在常数范围内达到最优性能。不过，注意力计算实际上可以在多个GPU之间并行处理 [72]。使用多个GPU会为IO分析增加一个新的考量因素——即需要考虑GPU之间的数据传输问题。我们希望我们的工作能够为这一领域的后续研究提供启发。

致谢

我们的实现以Apex的FMHA代码（https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha）作为基础。在此，我们感谢Young-Jun Ko对其FMHA实现的详细讲解，以及他对我们关于CUDA问题的耐心解答。同时，我们也感谢Sabri Eyuboglu、Megan Leszczynski、Laurel Orr、Yuhuai Wu、Beidi Chen和Xun Huang对论文初稿提出的建设性反馈与建议。此外，我们还要感谢Markus Rabe和Charles Staats就注意力算法问题进行的有益探讨。

We gratefully acknowledge the support of NIH under No. U54EB020405 (Mobilize), NSF under Nos. CCF1763315 (Beyond Sparsity), CCF1563078 (Volume to Velocity), and 1937301 (RTML); ARL under No. W911NF-21-2-0251 (Interactive Human-AI Teaming); ONR under No. N000141712266 (Unifying Weak Supervision); ONR N00014-20-1-2480: Understanding and Applying Non-Euclidean Geometry in Machine Learning; N000142012275 (NEPTUNE); NXP, Xilinx, LETI-CEA, Intel, IBM, Microsoft, NEC, Toshiba, TSMC, ARM, Hitachi, BASF, Accenture, Ericsson, Qualcomm, Analog Devices, Google Cloud, Salesforce, Total, the HAI-GCP & HAI-Azure Cloud Credits for Research program, the Stanford Data Science Initiative (SDSI), Department of Defense (DoD) through the National Defense Science and Engineering Graduate Fellowship (NDSEG) Program, and members of the Stanford DAWN project: Facebook, Google, and VMWare. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes

notwithstanding any copyright notation thereon. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views, policies, or endorsements, either expressed or implied, of NIH, ONR, or the U.S. Government. Atri Rudra's research is supported by NSF grant CCF-1763481.

References

[1] Alok Aggarwal and S Vitter, Jeffrey. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.

[2] Irwan Bello. LambdaNetworks: Modeling long-range interactions without attention. *arXiv preprint arXiv:2102.08602*, 2021.

[3] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[4] L Susan Blackford, Antoine Petitet, Roldan Pozo, Karin Remington, R Clint Whaley, James Demmel, Jack Dongarra, Iain Duff, Sven Hammarling, Greg Henry, et al. An updated set of basic linear algebra subprograms (blas). *ACM Transactions on Mathematical Software*, 28(2):135–151, 2002.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[6] Ilias Chalkidis, Ion Androutsopoulos, and Nikolaos Aletras. Neural legal judgment prediction in English. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4317–4323, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1424. URL `https://www.aclweb.org/anthology/P19-1424`.

[7] Ilias Chalkidis, Manos Fergadiotis, Dimitrios Tsarapatsanis, Nikolaos Aletras, Ion Androutsopoulos, and Prodromos Malakasiotis. Paragraph-level rationale extraction through regularization: A case study on european court of human rights cases. In *Proceedings of the Annual Conference of the North American Chapter of the Association for Computational Linguistics*, Mexico City, Mexico, 2021. Association for Computational Linguistics.

[8] Benjamin Charlier, Jean Feydy, Joan Alexis Glaunès, François-David Collin, and Ghislain Durif. Kernel operations on the gpu, with autodiff, without memory overflows. *Journal of Machine Learning Research*, 22(74):1–6, 2021. URL `http://jmlr.org/papers/v22/20-275.html`.

[9] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[10] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

[11] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[12] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.

[13] Xiang Dai, Ilias Chalkidis, Sune Darkner, and Desmond Elliott. Revisiting transformer-based models for long document classification. *arXiv preprint arXiv:2204.06683*, 2022.

[14] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

无需考虑其上的任何版权声明。本材料中所表达的任何观点、研究发现、结论或建议均属于作者个人观点，并不一定反映美国国立卫生研究院、美国国防高级研究计划局或美国政府的立场、政策或明示或隐含的认可。Atri Rudra的这项研究得到了美国国家科学基金会CCF-1763481资助项目的支持。

参考文献

[1] Alok Aggarwal and S Vitter, Jeffrey. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.

[2] Irwan Bello. LambdaNetworks: Modeling long-range interactions without attention. *arXiv preprint arXiv:2102.08602*, 2021.

[3] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[4] L Susan Blackford, Antoine Petitet, Roldan Pozo, Karin Remington, R Clint Whaley, James Demmel, Jack Dongarra, Iain Duff, Sven Hammarling, Greg Henry, et al. An updated set of basic linear algebra subprograms (blas). *ACM Transactions on Mathematical Software*, 28(2):135–151, 2002.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[6] Ilias Chalkidis, Ion Androutsopoulos, and Nikolaos Aletras. Neural legal judgment prediction in English. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4317–4323, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1424. URL `https://www.aclweb.org/anthology/P19-1424`.

[7] Ilias Chalkidis, Manos Fergadiotis, Dimitrios Tsarapatsanis, Nikolaos Aletras, Ion Androutsopoulos, and Prodromos Malakasiotis. Paragraph-level rationale extraction through regularization: A case study on european court of human rights cases. In *Proceedings of the Annual Conference of the North American Chapter of the Association for Computational Linguistics*, Mexico City, Mexico, 2021. Association for Computational Linguistics.

[8] Benjamin Charlier, Jean Feydy, Joan Alexis Glaunès, François-David Collin, and Ghislain Durif. Kernel operations on the gpu, with autodiff, without memory overflows. *Journal of Machine Learning Research*, 22(74):1–6, 2021. URL `http://jmlr.org/papers/v22/20-275.html`.

[9] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[10] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

[11] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[12] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.

[13] Xiang Dai, Ilias Chalkidis, Sune Darkner, and Desmond Elliott. Revisiting transformer-based models for long document classification. *arXiv preprint arXiv:2204.06683*, 2022.

[14] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

[15] Tri Dao, Albert Gu, Matthew Eichhorn, Atri Rudra, and Christopher Ré. Learning fast algorithms for linear transforms using butterfly factorizations. In *International Conference on Machine Learning (ICML)*, 2019.

[16] Tri Dao, Nimit Sohoni, Albert Gu, Matthew Eichhorn, Amit Blonder, Megan Leszczynski, Atri Rudra, and Christopher Ré. Kaleidoscope: An efficient, learnable representation for all structured linear maps. In *International Conference on Learning Representations (ICLR)*, 2020.

[17] Tri Dao, Beidi Chen, Kaizhao Liang, Jiaming Yang, Zhao Song, Atri Rudra, and Christopher Ré. Pixelated butterfly: Simple and efficient sparse training for neural network models. In *International Conference on Learning Representations (ICLR)*, 2022.

[18] Tri Dao, Beidi Chen, Nimit Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive structured matrices for efficient and accurate training. In *International Conference on Machine Learning (ICML)*, 2022.

[19] Giannis Daras, Nikita Kitaev, Augustus Odena, and Alexandros G Dimakis. Smyrf-efficient attention using asymmetric clustering. *Advances in Neural Information Processing Systems*, 33:6476–6489, 2020.

[20] Christopher De Sa, Albert Gu, Rohan Puttagunta, Christopher Ré, and Atri Rudra. A two-pronged progress in structured dense matrix vector multiplication. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1060–1079. SIAM, 2018.

[21] Peter J Denning. The working set model for program behavior. *Communications of the ACM*, 11(5): 323–333, 1968.

[22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. 2019.

[23] Xin Dong, Shangyu Chen, and Sinno Jialin Pan. Learning to prune deep neural networks via layer-wise optimal brain surgeon. *arXiv preprint arXiv:1705.07565*, 2017.

[24] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020.

[25] Y Eidelman and I Gohberg. On a new class of structured matrices. *Integral Equations and Operator Theory*, 34(3):293–324, 1999.

[26] Jean Feydy, Joan Glaunès, Benjamin Charlier, and Michael Bronstein. Fast geometric learning with symbolic matrices. *Advances in Neural Information Processing Systems*, 33, 2020.

[27] Jörg Flum and Martin Grohe. *Parameterized Complexity Theory*. Springer, 2006.

[28] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*, 2018.

[29] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M Roy, and Michael Carbin. Stabilizing the lottery ticket hypothesis. *arXiv preprint arXiv:1903.01611*, 2019.

[30] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *International Conference on Machine Learning*, pages 3259–3269. PMLR, 2020.

[31] Karan Goel, Albert Gu, Chris Donahue, and Christopher Ré. It's raw! audio generation with state-space models. In *International Conference on Machine Learning (ICML)*, 2022.

[32] Aaron Gokaslan, Vanya Cohen, Pavlick Ellie, and Stefanie Tellex. Openwebtext corpus, 2019.

[15] Tri Dao, Albert Gu, Matthew Eichhorn, Atri Rudra, and Christopher Ré. Learning fast algorithms for linear transforms using butterfly factorizations. In *International Conference on Machine Learning (ICML)*, 2019.

[16] Tri Dao, Nimit Sohoni, Albert Gu, Matthew Eichhorn, Amit Blonder, Megan Leszczynski, Atri Rudra, and Christopher Ré. Kaleidoscope: An efficient, learnable representation for all structured linear maps. In *International Conference on Learning Representations (ICLR)*, 2020.

[17] Tri Dao, Beidi Chen, Kaizhao Liang, Jiaming Yang, Zhao Song, Atri Rudra, and Christopher Ré. Pixelated butterfly: Simple and efficient sparse training for neural network models. In *International Conference on Learning Representations (ICLR)*, 2022.

[18] Tri Dao, Beidi Chen, Nimit Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive structured matrices for efficient and accurate training. In *International Conference on Machine Learning (ICML)*, 2022.

[19] Giannis Daras, Nikita Kitaev, Augustus Odena, and Alexandros G Dimakis. Smyrf-efficient attention using asymmetric clustering. *Advances in Neural Information Processing Systems*, 33:6476–6489, 2020.

[20] Christopher De Sa, Albert Gu, Rohan Puttagunta, Christopher Ré, and Atri Rudra. A two-pronged progress in structured dense matrix vector multiplication. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1060–1079. SIAM, 2018.

[21] Peter J Denning. The working set model for program behavior. *Communications of the ACM*, 11(5): 323–333, 1968.

[22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. 2019.

[23] Xin Dong, Shangyu Chen, and Sinno Jialin Pan. Learning to prune deep neural networks via layer-wise optimal brain surgeon. *arXiv preprint arXiv:1705.07565*, 2017.

[24] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020.

[25] Y Eidelman and I Gohberg. On a new class of structured matrices. *Integral Equations and Operator Theory*, 34(3):293–324, 1999.

[26] Jean Feydy, Joan Glaunès, Benjamin Charlier, and Michael Bronstein. Fast geometric learning with symbolic matrices. *Advances in Neural Information Processing Systems*, 33, 2020.

[27] Jörg Flum and Martin Grohe. *Parameterized Complexity Theory*. Springer, 2006.

[28] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*, 2018.

[29] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M Roy, and Michael Carbin. Stabilizing the lottery ticket hypothesis. *arXiv preprint arXiv:1903.01611*, 2019.

[30] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *International Conference on Machine Learning*, pages 3259–3269. PMLR, 2020.

[31] Karan Goel, Albert Gu, Chris Donahue, and Christopher Ré. It's raw! audio generation with state-space models. In *International Conference on Machine Learning (ICML)*, 2022.

[32] Aaron Gokaslan, Vanya Cohen, Pavlick Ellie, and Stefanie Tellex. Openwebtext corpus, 2019.

[33] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data mining and knowledge discovery*, 1(1):29–53, 1997.

[34] Andreas Griewank and Andrea Walther. *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM, 2008.

[35] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. Hippo: Recurrent memory with optimal polynomial projections. In *Advances in neural information processing systems (NeurIPS)*, 2020.

[36] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in Neural Information Processing Systems*, 34, 2021.

[37] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *The International Conference on Learning Representations (ICLR)*, 2022.

[38] Song Han, Jeff Pool, John Tran, and William J Dally. Learning both weights and connections for efficient neural networks. *arXiv preprint arXiv:1506.02626*, 2015.

[39] Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations*, 2016.

[40] John Hennessy and David Patterson. Memory hierarchy design. *Computer Architecture: A Quantitative Approach*, pages 390–525, 2003.

[41] Sara Hooker. The hardware lottery. *arXiv preprint arXiv:2009.06489*, 2020.

[42] Weizhe Hua, Zihang Dai, Hanxiao Liu, and Quoc V Le. Transformer quality in linear time. *arXiv preprint arXiv:2202.10447*, 2022.

[43] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. *Proceedings of Machine Learning and Systems*, 3: 711–732, 2021.

[44] Zhe Jia and Peter Van Sandt. Dissecting the Ampere GPU architecture via microbenchmarking. GPU Technology Conference, 2021.

[45] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. Dissecting the nvidia Volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826*, 2018.

[46] Zhe Jia, Blake Tillman, Marco Maggioni, and Daniele Paolo Scarpazza. Dissecting the graphcore IPU architecture via microbenchmarking. *arXiv preprint arXiv:1912.03413*, 2019.

[47] Alistair EW Johnson, Tom J Pollard, Lu Shen, Li-wei H Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G Mark. Mimic-iii, a freely accessible critical care database. *Scientific data*, 3(1):1–9, 2016.

[48] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*, pages 1–12, 2017.

[49] Thomas Kailath, Sun-Yuan Kung, and Martin Morf. Displacement ranks of matrices and linear equations. *Journal of Mathematical Analysis and Applications*, 68(2):395–407, 1979.

[50] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.

[33] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data mining and knowledge discovery*, 1(1):29–53, 1997.

[34] Andreas Griewank and Andrea Walther. *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM, 2008.

[35] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. Hippo: Recurrent memory with optimal polynomial projections. In *Advances in neural information processing systems (NeurIPS)*, 2020.

[36] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in Neural Information Processing Systems*, 34, 2021.

[37] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *The International Conference on Learning Representations (ICLR)*, 2022.

[38] Song Han, Jeff Pool, John Tran, and William J Dally. Learning both weights and connections for efficient neural networks. *arXiv preprint arXiv:1506.02626*, 2015.

[39] Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations*, 2016.

[40] John Hennessy and David Patterson. Memory hierarchy design. *Computer Architecture: A Quantitative Approach*, pages 390–525, 2003.

[41] Sara Hooker. The hardware lottery. *arXiv preprint arXiv:2009.06489*, 2020.

[42] Weizhe Hua, Zihang Dai, Hanxiao Liu, and Quoc V Le. Transformer quality in linear time. *arXiv preprint arXiv:2202.10447*, 2022.

[43] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. *Proceedings of Machine Learning and Systems*, 3: 711–732, 2021.

[44] Zhe Jia and Peter Van Sandt. Dissecting the Ampere GPU architecture via microbenchmarking. GPU Technology Conference, 2021.

[45] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. Dissecting the nvidia Volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826*, 2018.

[46] Zhe Jia, Blake Tillman, Marco Maggioni, and Daniele Paolo Scarpazza. Dissecting the graphcore IPU architecture via microbenchmarking. *arXiv preprint arXiv:1912.03413*, 2019.

[47] Alistair EW Johnson, Tom J Pollard, Lu Shen, Li-wei H Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G Mark. Mimic-iii, a freely accessible critical care database. *Scientific data*, 3(1):1–9, 2016.

[48] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*, pages 1–12, 2017.

[49] Thomas Kailath, Sun-Yuan Kung, and Martin Morf. Displacement ranks of matrices and linear equations. *Journal of Mathematical Analysis and Applications*, 68(2):395–407, 1979.

[50] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.

[51] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.

[52] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite BEDRT for self-supervised learning of language representations. In *The International Conference on Learning Representations (ICLR)*, 2020.

[53] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. The deep learning compiler: A comprehensive survey. *IEEE Transactions on Parallel and Distributed Systems*, 32(3):708–727, 2020.

[54] Valerii Likhosherstov, Krzysztof Choromanski, Jared Davis, Xingyou Song, and Adrian Weller. Sub-linear memory: How to make performers slim. *arXiv preprint arXiv:2012.11346*, 2020.

[55] Ji Lin, Yongming Rao, Jiwen Lu, and Jie Zhou. Runtime neural pruning. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[56] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[57] Xuezhe Ma, Xiang Kong, Sinong Wang, Chunting Zhou, Jonathan May, Hao Ma, and Luke Zettlemoyer. Luna: Linear unified nested attention. *Advances in Neural Information Processing Systems*, 34, 2021.

[58] Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, et al. Mlperf training benchmark. *Proceedings of Machine Learning and Systems*, 2:336–349, 2020.

[59] Frank McSherry, Michael Isard, and Derek G Murray. Scalability! but at what {COST}? In *15th Workshop on Hot Topics in Operating Systems (HotOS XV)*, 2015.

[60] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

[61] NVIDIA. Nvidia Tesla V100 GPU architecture, 2017.

[62] NVIDIA. Nvidia A100 tensor core GPU architecture, 2020.

[63] NVIDIA. Nvidia H100 tensor core GPU architecture, 2022.

[64] D Stott Parker. Random butterfly transformations with applications in computational linear algebra. 1995.

[65] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[66] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

[67] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[68] Jack Rae and Ali Razavi. Do transformers need deep long-range memory? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Online, July 2020. Association for Computational Linguistics. URL `https://www.aclweb.org/anthology/2020.acl-main.672`.

[69] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. In *The International Conference on Learning Representations (ICLR)*, 2020.

[51] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.

[52] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite BEDRT for self-supervised learning of language representations. In *The International Conference on Learning Representations (ICLR)*, 2020.

[53] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. The deep learning compiler: A comprehensive survey. *IEEE Transactions on Parallel and Distributed Systems*, 32(3):708–727, 2020.

[54] Valerii Likhosherstov, Krzysztof Choromanski, Jared Davis, Xingyou Song, and Adrian Weller. Sub-linear memory: How to make performers slim. *arXiv preprint arXiv:2012.11346*, 2020.

[55] Ji Lin, Yongming Rao, Jiwen Lu, and Jie Zhou. Runtime neural pruning. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[56] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[57] Xuezhe Ma, Xiang Kong, Sinong Wang, Chunting Zhou, Jonathan May, Hao Ma, and Luke Zettlemoyer. Luna: Linear unified nested attention. *Advances in Neural Information Processing Systems*, 34, 2021.

[58] Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, et al. Mlperf training benchmark. *Proceedings of Machine Learning and Systems*, 2:336–349, 2020.

[59] Frank McSherry, Michael Isard, and Derek G Murray. Scalability! but at what {COST}? In *15th Workshop on Hot Topics in Operating Systems (HotOS XV)*, 2015.

[60] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

[61] NVIDIA. Nvidia Tesla V100 GPU architecture, 2017.

[62] NVIDIA. Nvidia A100 tensor core GPU architecture, 2020.

[63] NVIDIA. Nvidia H100 tensor core GPU architecture, 2022.

[64] D Stott Parker. Random butterfly transformations with applications in computational linear algebra. 1995.

[65] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[66] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

[67] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[68] Jack Rae and Ali Razavi. Do transformers need deep long-range memory? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Online, July 2020. Association for Computational Linguistics. URL `https://www.aclweb.org/anthology/2020.acl-main.672`.

[69] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. In *The International Conference on Learning Representations (ICLR)*, 2020.

[70] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6):519–530, 2013.

[71] Raghu Ramakrishnan, Johannes Gehrke, and Johannes Gehrke. *Database management systems*, volume 3. McGraw-Hill New York, 2003.

[72] Benjamin Recht and Christopher Ré. Parallel stochastic gradient algorithms for large-scale matrix completion. *Mathematical Programming Computation*, 5(2):201–226, 2013.

[73] Hongyu Ren, Hanjun Dai, Zihang Dai, Mengjiao Yang, Jure Leskovec, Dale Schuurmans, and Bo Dai. Combiner: Full attention transformer with sparse computation cost. *Advances in Neural Information Processing Systems*, 34, 2021.

[74] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9: 53–68, 2021.

[75] Amit Sabne. XLA: Compiling machine learning for peak performance. 2020.

[76] Victor Sanh, Thomas Wolf, and Alexander M Rush. Movement pruning: Adaptive sparsity by fine-tuning. *arXiv preprint arXiv:2005.07683*, 2020.

[77] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[78] Vikas Sindhwani, Tara Sainath, and Sanjiv Kumar. Structured transforms for small-footprint deep learning. In *Advances in Neural Information Processing Systems*, pages 3088–3096, 2015.

[79] Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. Adaptive attention span in transformers. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2019.

[80] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena: A benchmark for efficient transformers. In *International Conference on Learning Representations*, 2020.

[81] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.

[82] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[83] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, and Furu Wei. Deepnet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022.

[84] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[85] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[86] Michael E Wolf and Monica S Lam. A data locality optimizing algorithm. In *Proceedings of the ACM SIGPLAN 1991 conference on Programming language design and implementation*, pages 30–44, 1991.

[70] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6):519–530, 2013.

[71] Raghu Ramakrishnan, Johannes Gehrke, and Johannes Gehrke. *Database management systems*, volume 3. McGraw-Hill New York, 2003.

[72] Benjamin Recht and Christopher Ré. Parallel stochastic gradient algorithms for large-scale matrix completion. *Mathematical Programming Computation*, 5(2):201–226, 2013.

[73] Hongyu Ren, Hanjun Dai, Zihang Dai, Mengjiao Yang, Jure Leskovec, Dale Schuurmans, and Bo Dai. Combiner: Full attention transformer with sparse computation cost. *Advances in Neural Information Processing Systems*, 34, 2021.

[74] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9: 53–68, 2021.

[75] Amit Sabne. XLA: Compiling machine learning for peak performance. 2020.

[76] Victor Sanh, Thomas Wolf, and Alexander M Rush. Movement pruning: Adaptive sparsity by fine-tuning. *arXiv preprint arXiv:2005.07683*, 2020.

[77] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[78] Vikas Sindhwani, Tara Sainath, and Sanjiv Kumar. Structured transforms for small-footprint deep learning. In *Advances in Neural Information Processing Systems*, pages 3088–3096, 2015.

[79] Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. Adaptive attention span in transformers. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2019.

[80] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena: A benchmark for efficient transformers. In *International Conference on Learning Representations*, 2020.

[81] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.

[82] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[83] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, and Furu Wei. Deepnet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022.

[84] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[85] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[86] Michael E Wolf and Monica S Lam. A data locality optimizing algorithm. In *Proceedings of the ACM SIGPLAN 1991 conference on Programming language design and implementation*, pages 30–44, 1991.

[87] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics. URL `https://www.aclweb.org/anthology/2020.emnlp-demos.6`.

[88] David P Woodruff. Optimal space lower bounds for all frequency moments. In *SODA*, volume 4, pages 167–175. Citeseer, 2004.

[89] Felix Wu, Angela Fan, Alexei Baevski, Yann N Dauphin, and Michael Auli. Pay less attention with lightweight and dynamic convolutions. In *The International Conference on Learning Representations (ICLR)*, 2019.

[90] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence. AAAI Conference on Artificial Intelligence*, volume 35, page 14138, 2021.

[91] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Zi-Hang Jiang, Francis EH Tay, Jiashi Feng, and Shuicheng Yan. Tokens-to-token vit: Training vision transformers from scratch on imagenet. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 558–567, 2021.

[92] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

[93] Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, and Josh Susskind. An attention free transformer. *arXiv preprint arXiv:2105.14103*, 2021.

[94] Chen Zhu, Wei Ping, Chaowei Xiao, Mohammad Shoeybi, Tom Goldstein, Anima Anandkumar, and Bryan Catanzaro. Long-short transformer: Efficient transformers for language and vision. *Advances in Neural Information Processing Systems*, 34, 2021.

[87] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics. URL `https://www.aclweb.org/anthology/2020.emnlp-demos.6`.

[88] David P Woodruff. Optimal space lower bounds for all frequency moments. In *SODA*, volume 4, pages 167–175. Citeseer, 2004.

[89] Felix Wu, Angela Fan, Alexei Baevski, Yann N Dauphin, and Michael Auli. Pay less attention with lightweight and dynamic convolutions. In *The International Conference on Learning Representations (ICLR)*, 2019.

[90] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence. AAAI Conference on Artificial Intelligence*, volume 35, page 14138, 2021.

[91] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Zi-Hang Jiang, Francis EH Tay, Jiashi Feng, and Shuicheng Yan. Tokens-to-token vit: Training vision transformers from scratch on imagenet. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 558–567, 2021.

[92] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

[93] Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, and Josh Susskind. An attention free transformer. *arXiv preprint arXiv:2105.14103*, 2021.

[94] Chen Zhu, Wei Ping, Chaowei Xiao, Mohammad Shoeybi, Tom Goldstein, Anima Anandkumar, and Bryan Catanzaro. Long-short transformer: Efficient transformers for language and vision. *Advances in Neural Information Processing Systems*, 34, 2021.

A Related Work

IO-Aware Runtime Optimization. The broad concept of optimizing for reading and writing to fast/slow memory has a long history in computer science and has been known by many names. We draw the most direct connection to the literature of analyzing I/O complexity in this work [1], but concepts of memory hierarchies are fundamental and has appeared in many forms, from the working set model [21], to data locality [86], to the Roofline model of arithmetic intensity [85], to analyses of scalability [59], to standard textbook treatments of computer architecture [40]. We hope that this work encourages the community to adopt these ideas in more parts of the deep learning stack.

Efficient ML Models with Structured Matrices. Matrix multiply is the core computational bottleneck of most machine learning models. To reduce the computational complexity, there have been numerous approaches to learn over a more efficient set of matrices. These matrices are called *structured matrices*, which have subquadratic ($o(n^2)$ for dimension $n \times n$) number of parameters and runtime. Most common examples of structured matrices are sparse and low-rank matrices, along with fast transforms commonly encountered in signal processing (Fourier, Chebyshev, sine/cosine, orthogonal polynomials). There have been several more general classes of structured matrices proposed in machine learning: Toeplitz-like [78], low-displacement rank [49], quasi-separable [25]). The butterfly pattern we use for our block-sparse attention is motivated by the fact that butterfly matrices [15, 64] and their products have been shown to be able to express any structured matrices with almost optimal runtime and number of parameters [16, 20]. However, even though structured matrices are efficient in theory, they have not seen wide adoption since it is hard to translate their efficiency to wall-clock speedup since dense unconstrained matrix multiply has very optimize implementation, a phenomenon known as the hardware lottery [41]. Extensions of butterfly matrices [17, 18] aimed to make butterfly matrices more hardware-friendly.

Sparse Training. Our block-sparse FlashAttention can be seen as a step towards making sparse model training more efficient. Sparse models have seen success in compressing models for inference (pruning) by sparsifying the weight matrices [23, 38, 39, 55, 76]. For model training, the lottery tickets hypothesis [28, 29, 30] suggests that there are a set of small sub-networks derived from a larger dense network that performs as well as the original dense network. Out block-sparse FlashAttention can also be seen as a fixed lottery ticket in the context of attention: we fix the sparsity pattern to be the butterfly pattern through training, and observe that it performs almost as well as the (dense) FlashAttention on the Long-range Arena tasks.

Efficient Transformer. Transformer-based models have become the most widely-used architecture in natural language processing [22] and computer vision [24, 91]. However, one of their computational bottlenecks is that their time and memory scales quadratic in the sequence length. There are numerous approaches to overcome this bottleneck, including approximation with hashing (i.e., sparse) such as Reformer [51] and Smyrf [19] and with low-rank approximation such as Performer [12, 54]. One can even combine sparse and low-rank approximation for better accuracy (e.g., Longformer [3], BigBird [92], Scatterbrain [9], Long-short transformer [94], Combiner [73]). Other approaches include compressing along the sequence dimension to attend to multiple tokens at once [52, 57, 79, 89]. One can also attend over the states from previous sequences to help lengthen the context (e.g., Transformer-XL [14] and Compressive Transformer [69]). We recommend the survey [81] for more details.

There are several lines of work on developing other modules instead of attention to model longer context. HiPPO [35] and its extensions, most notably S4 [31, 36, 37] projects the history on a polynomial basis, allowing accurate reconstruction of the history through state-space models. They combine the strengths of CNNs (efficient training), RNNs (efficient inference), and continuous models (robust to change in sampling rates). LambdaNetworks [2], AFT [93] and FLASH [42] are other attempts at replacing attention in the context of image classification and language modeling.

B Algorithm Details

We first derive the forward and backward passes of attention and show that they can be computed in a memory-efficient manner (requiring extra memory linear instead of quadratic in the sequence length). Though they reduce the amount of extra memory required, naively they still incur quadratic HBM accesses, resulting in slower execution speed. We describe the FlashAttention algorithm to implement both the forward

相关研究

具备IO感知能力的运行时间优化。针对快速/慢速内存的读写操作进行优化的理念在计算机科学领域由来已久，且有着多种不同的称谓。在本工作中，我们与分析I/O复杂度的相关研究 [1], 有着最直接的关联；不过内存层次结构的概念本身十分基础，它以多种形式出现过，从工作集模型 [21], 到数据局部性 [86], ，再到体现算术强度特性的Roofline模型 [85], 、对系统可扩展性的分析 [59], ，以及计算机体系结构领域的标准教材论述 [40]，皆是其体现。我们希望这项工作能够激励业界在深度学习框架的更多环节中应用这些理念。

Efficient ML Models with Structured Matrices. Matrix multiply is the core computational bottleneck of most machine learning models. To reduce the computational complexity, there have been numerous approaches to learn over a more efficient set of matrices. These matrices are called *structured matrices*, which have subquadratic ($o(n^2)$ for dimension $n \times n$) number of parameters and runtime. Most common examples of structured matrices are sparse and low-rank matrices, along with fast transforms commonly encountered in signal processing (Fourier, Chebyshev, sine/cosine, orthogonal polynomials). There have been several more general classes of structured matrices proposed in machine learning: Toeplitz-like [78], low-displacement rank [49], quasi-separable [25]). The butterfly pattern we use for our block-sparse attention is motivated by the fact that butterfly matrices [15, 64] and their products have been shown to be able to express any structured matrices with almost optimal runtime and number of parameters [16, 20]. However, even though structured matrices are efficient in theory, they have not seen wide adoption since it is hard to translate their efficiency to wall-clock speedup since dense unconstrained matrix multiply has very optimize implementation, a phenomenon known as the hardware lottery [41]. Extensions of butterfly matrices [17, 18] aimed to make butterfly matrices more hardware-friendly.

稀疏训练。我们所提出的分块稀疏FlashAttention算法可被视为提升稀疏模型训练效率的重要一步。通过使权重矩阵 [23, 38, 39, 55, 76]呈现稀疏结构，稀疏模型已在模型压缩以用于推理（剪枝）方面取得了良好效果。在模型训练领域，彩票票假说 [28, 29, 30]指出，从一个更大的密集网络中可以衍生出一组小型子网络，其性能与原始的密集网络相当。我们的分块稀疏FlashAttention算法在注意力机制的背景下也可被视作一种固定的“彩票票”：我们通过训练将稀疏模式固定为蝴蝶型结构，并发现它在长距离任务集上的表现几乎可与（密集型的）FlashAttention算法相媲美。

Efficient Transformer. Transformer-based models have become the most widely-used architecture in natural language processing [22] and computer vision [24, 91]. However, one of their computational bottlenecks is that their time and memory scales quadratic in the sequence length. There are numerous approaches to overcome this bottleneck, including approximation with hashing (i.e., sparse) such as Reformer [51] and Smyrf [19] and with low-rank approximation such as Performer [12, 54]. One can even combine sparse and low-rank approximation for better accuracy (e.g., Longformer [3], BigBird [92], Scatterbrain [9], Long-short transformer [94], Combiner [73]). Other approaches include compressing along the sequence dimension to attend to multiple tokens at once [52, 57, 79, 89]. One can also attend over the states from previous sequences to help lengthen the context (e.g., Transformer-XL [14] and Compressive Transformer [69]). We recommend the survey [81] for more details.

为了解决建模更长上下文的问题，除了基于注意力机制之外，还有多种研究方向致力于开发其他模块。HiPPO [35] 及其衍生版本，尤其是S4 [31, 36, 37]，会将历史信息投影到多项式基础上，从而借助状态空间模型实现对历史信息的精准重建。这些方法结合了卷积神经网络（训练效率高）、循环神经网络（推理效率高）以及连续模型（对采样率的变化具有较强鲁棒性）的优势。在图像分类和语言建模领域，LambdaNetworks [2], 、AFT [93]和FLASH [42]也是试图替代注意力机制的其他尝试。

B尺寸算法细节

我们首先推导出注意力机制的向前传递与向后传递过程，并证明这两种过程可以通过高效利用内存的方式来计算（所需额外内存与序列长度成线性关系，而非二次关系）。尽管这种方式减少了所需的额外内存量，但若直接采用传统方法，仍会带来二次级的HBM访问次数，从而导致执行速度变慢。为此，我们介绍了FlashAttention算法，该算法能够在GPU上实现这两种传递过程，进而降低HBM访问次数，最终实现更快的运行时间以及更小的内存占用。

and the backward passes on GPUs that reduces HBM accesses, leading to both faster runtime and smaller memory footprint.

B.1 Memory-efficient forward pass

The main challenge in making attention memory-efficient is the softmax that couples the columns of $\mathbf{K}$ (and columns of $\mathbf{V}$). Our approach is to compute the softmax normalization constant separately to decouple the columns. This technique [60] has been used in the literature [51, 66] to show that attention computation does not need quadratic *extra* memory (though the number of HBM accesses is still quadratic, resulting in slow run-time).

For simplicity, we omit here the max-shifting step during softmax. The full algorithm in Appendix B.3 contains all the steps.

Recall that given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}.$$

We have that $S_{ij} = q_i^T k_j$ where q_i and k_j are the i-th and j-th columns of $\mathbf{Q}$ and $\mathbf{K}$ respectively. Define the normalization constants of softmax:

$$L_i = \sum_j e^{q_i^T k_j}. \tag{1}$$

Let v_j be the j-th column of $\mathbf{V}$, then the i-th columns of the output is

$$o_i = P_{i:}\mathbf{V} = \sum_j P_{ij} v_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j. \tag{2}$$

We see that once L_i is computed, we can compute o_i without extra memory by repeatedly summing $\frac{e^{q_i^T k_j}}{L_i} v_j$. Therefore the forward pass can be computed with $O(n)$ extra memory:

1. Compute L_i for all i according to Eq. (1), which takes $O(n)$ extra memory.
2. Compute o_i for all i according to Eq. (2), which takes $O(d)$ extra memory.

B.2 Memory-efficient backward pass

We derive the backward pass of attention and show that it can also be computed with linear memory. Rabe and Staats [66] suggests that the backward pass can be done without quadratic extra memory by applying gradient checkpointing to the memory-efficient forward pass. We instead derive the backward pass explicitly and show how it can be computed in a memory-efficient manner.

Suppose that there is a scalar loss function ϕ, and let the output gradient be $\mathbf{dO} \in \mathbb{R}^{n \times d}$ (where $\mathbf{dO}$ denotes $\frac{\partial \phi}{\partial \mathbf{O}}$). We want to compute the input gradients $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{n \times d}$ (where $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ denote $\frac{\partial \phi}{\partial \mathbf{Q}}, \frac{\partial \phi}{\partial \mathbf{K}}, \frac{\partial \phi}{\partial \mathbf{V}}$ respectively).

The gradient $\mathbf{dV}$ is easy to see. Applying reverse-mode autodiff by hand (aka the chain rule), we obtain (in matrix notation) $\mathbf{dV} = \mathbf{P}^T \mathbf{dO}$. Thus:

$$dv_j = \sum_i P_{ij} do_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} do_i. \tag{3}$$

Since we already computed L_i, dv_j can be computed without extra memory by repeated summing.

The gradients $\mathbf{dQ}$ and $\mathbf{dK}$ are a little more complicated. We go through the gradients $\mathbf{dP}$ and $\mathbf{dS}$ first. From Eq. (2), we have that $\mathbf{dP} = \mathbf{dO}\mathbf{V}^T$, and so:

$$dP_{ij} = do_i^T v_j.$$

Recall that $P_{i:} = \mathrm{softmax}(S_{i:})$. Using the fact that the Jacobian of $y = \mathrm{softmax}(x)$ is $\mathrm{diag}(y) - yy^T$, we have that

$$dS_{i:} = (\mathrm{diag}(P_{i:}) - P_{i:}P_{i:}^T) dP_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^T dP_{i:}) P_{i:},$$

通过这种算法，GPU上的注意力机制计算既能提升运行速度，又能减少内存消耗。

B.1 高内存效率的前向传播

在让注意力机制具备内存高效性的过程中，最大的挑战在于用于连接$\mathbf{K}$ 矩阵各列（以及$\mathbf{V}$矩阵各列）的Softmax函数。我们的解决方案是单独计算Softmax的归一化常数，以此将这些列解耦开来。这一技术 [60] 已在相关文献 [51, 66] 中被采用，用以证明注意力机制的计算并不需要二次方的额外内存占用（尽管HBM访问次数仍为二次方级，从而导致运行速度较慢）。

为简化说明，此处省略了Softmax运算过程中的最大位移步骤。附录B.3中的完整算法包含了所有步骤。

请记住，给定 输入序列为 $\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V} \in \mathbb{R}^{N \times d}$，我们需要计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ ：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}.$$

我们有 $S_{ij} = q_i^T k_j$，其中 q_i和 k_j分别是$\mathbf{Q}$与$\mathbf{K}$ 的第 i列和第 j列。定义Softmax函数的归一化常数：

$$L_i = \sum_j e^{q_i^T k_j}. \tag{1}$$

令v_j 为$\mathbf{V}$的第 j列，则输出的第 i列即为

$$o_i = P_{i:}\mathbf{V} = \sum_j P_{ij} v_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j. \tag{2}$$

我们可以看到，一旦计算出 L_i ，就可以通过反复对$\frac{e^{q_i^T k_j}}{L_i} v_j$求和来计算 o_i ，而无需额外内存。因此，前向传播只需 $O(n)$的额外内存即可完成计算。

根据公式(1)为所有的 i 计算 L_i ，这一过程会额外占用 $O(n)$的内存。

2. 根据公式(2)为所有 i 计算 o_i ，该操作会额外占用 $O(d)$的内存。

B.2 高内存效率的反向传播

我们推导出了注意力机制的后向传播公式，并证明该过程同样可以通过线性内存来完成计算。Rabe和Staats [66] 提出，通过对内存效率较高的前向传播应用梯度检查点技术，无需额外的二次方级内存即可实现后向传播。而我们则直接明确推导出后向传播的算法，并展示了如何以高效节省内存的方式对其进行计算。

Suppose that there is a scalar loss function ϕ, and let the output gradient be $\mathbf{dO} \in \mathbb{R}^{n \times d}$ (where $\mathbf{dO}$ denotes $\frac{\partial \phi}{\partial \mathbf{O}}$). We want to compute the input gradients $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{n \times d}$ (where $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ denote $\frac{\partial \phi}{\partial \mathbf{Q}}, \frac{\partial \phi}{\partial \mathbf{K}}, \frac{\partial \phi}{\partial \mathbf{V}}$ respectively).

梯度 $\mathbf{dV}$ 很容易理解。通过手动应用反向模式自动微分（即链式法则），我们用矩阵表示法得到 $\mathbf{dV} = \mathbf{P}^T\mathbf{dO}$。因此：

$$dv_j = \sum_i P_{ij} do_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} do_i. \tag{3}$$

由于我们已经计算出了 L_i，因此可以通过反复求和的方式在无需额外内存的情况下计算出 dv_j 。

梯度$\mathbf{dQ}$ 和$\mathbf{dK}$ 的计算稍显复杂一些。我们先来分析梯度$\mathbf{dP}$ 和$\mathbf{dS}$ 。根据方程(2)，可知$\mathbf{dP}$ 等于 $= \mathbf{dO}\mathbf{V}^T$，因此：

$$dP_{ij} = do_i^T v_j.$$

请记住， $P_{i:} = \mathrm{Softmax}(S_{i:})$的含义。由于 $y = \mathrm{Softmax}(x)$的雅可比矩阵为$\mathrm{diag}(y) - yy^T$，因此我们可以得出如下结论。

$$dS_{i:} = (\mathrm{diag}(P_{i:}) - P_{i:}P_{i:}^T) dP_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^T dP_{i:}) P_{i:},$$

where $\circ$ denotes pointwise multiplication.

Define

$$D_i = P_{i:}^T dP_{i:} = \sum_j \frac{e^{q_i^T k_j}}{L_i} do_i^T v_j = do_i^T \sum_j \frac{e^{q_i^\top k_j}}{L_i} v_j = do_i^T o_i, \tag{4}$$

then

$$dS_{i:} = P_{i:} \circ dP_{i:} - D_i P_{i:}.$$

Hence

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij}(dP_{ij} - D_i).$$

Now we can get the gradients $\mathbf{dQ}$ and $\mathbf{dK}$. Recall that $S_{ij} = q_i^T k_j$, so

$$dq_i = \sum_j dS_{ij} k_j = \sum_j P_{ij}(dP_{ij} - D_i) k_j = \sum_j \frac{e^{q_i^T k_j}}{L_i}(do_i^T v_j - D_i) k_j. \tag{5}$$

Similarly,

$$dk_j = \sum_i dS_{ij} q_i = \sum_i P_{ij}(dP_{ij} - D_i) q_i = \sum_i \frac{e^{q_i^T k_j}}{L_i}(do_i^T v_j - D_i) q_i. \tag{6}$$

Therefore the backward pass can also be computed with $O(n)$ extra memory:

1. Compute dv_j for all j according to Eq. (3), which takes $O(d)$ extra memory.
2. Compute D_i for all i according to Eq. (4), which takes $O(n)$ extra memory.
3. Compute dq_i for all i according to Eq. (5), which takes $O(d)$ extra memory.
4. Compute dk_j for all j according to Eq. (6), which takes $O(d)$ extra memory.

B.3 FlashAttention: Forward Pass

We describe the full details of FlashAttention forward pass. Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \tau \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{S}^{\text{masked}} = \text{MASK}(S) \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}^{\text{masked}}) \in \mathbb{R}^{N \times N},$$
$$\mathbf{P}^{\text{dropped}} = \text{dropout}(\mathbf{P}, p_{\text{drop}}), \quad \mathbf{O} = \mathbf{P}^{\text{dropped}} \mathbf{V} \in \mathbb{R}^{N \times d},$$

where $\tau \in \mathbb{R}$ is some softmax scaling (typically $\frac{1}{\sqrt{d}}$), MASK is some masking function that sets some entries of the input to $-\infty$ and keep other entries the same (e.g., key padding mask when sequences in the batch don't have the same lengths and are padded), and $\text{dropout}(x, p)$ applies dropout to x elementwise (i.e., output $\frac{x}{1-p}$ with probability $1 - p$ and output 0 with probability p for each element x).

The full algorithm is in Algorithm 2. We save the output $\mathbf{O}$, the softmax statistics ℓ and m, and the pseudo-random number generator state $\mathcal{R}$ for the backward pass.

其中 $\circ$ 表示逐元素相乘。

定义

$$D_i = P_{i:}^T dP_{i:} = \sum_j \frac{e^{q_i^T k_j}}{L_i} do_i^T v_j = do_i^T \sum_j \frac{e^{q_i^\top k_j}}{L_i} v_j = do_i^T o_i, \tag{4}$$

then

$$dS_{i:} = P_{i:} \circ dP_{i:} - D_i P_{i:}.$$

因此，

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij}(dP_{ij} - D_i).$$

现在我们可以得到梯度 $\mathbf{dQ}$ 和 $\mathbf{dK}$。需要记住的是 $S_{ij} = q_i^T k_j$，所以……

$$dq_i = \sum_j dS_{ij} k_j = \sum_j P_{ij}(dP_{ij} - D_i) k_j = \sum_j \frac{e^{q_i^T k_j}}{L_i}(do_i^T v_j - D_i) k_j. \tag{5}$$

同理，

$$dk_j = \sum_i dS_{ij} q_i = \sum_i P_{ij}(dP_{ij} - D_i) q_i = \sum_i \frac{e^{q_i^T k_j}}{L_i}(do_i^T v_j - D_i) q_i. \tag{6}$$

因此，通过 $O(n)$ 的额外内存，也可以计算出后向传播所需的数据。

1. 根据公式（3）为所有的 j 计算 dv_j，这一过程会占用 $O(d)$ 的额外内存。
2. 根据公式(4)为所有的 i 计算 D_i，这一过程会额外占用 $O(n)$的内存。
3. 根据公式(5)为所有的 i 计算 dq_i，这一过程会额外占用 $O(d)$的内存。
4. 根据公式(6)为所有的 j计算 dk_j，该操作会额外占用 $O(d)$的内存。

B.3 FlashAttention算法：ForwardPass传递

我们详细介绍了FlashAttention算法的前向传播过程。给定输入序列$\mathbf{Q}$、$\mathbf{K}$以及$\mathbf{V} \in \mathbb{R}^{N \times d}$，我们需要计算注意力输出$\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \tau \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{S}^{\text{masked}} = \text{MASK}(S) \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}^{\text{masked}}) \in \mathbb{R}^{N \times N},$$
$$\mathbf{P}^{\text{dropped}} = \text{dropout}(\mathbf{P}, p_{\text{drop}}), \quad \mathbf{O} = \mathbf{P}^{\text{dropped}} \mathbf{V} \in \mathbb{R}^{N \times d},$$

其中 $\tau \in \mathbb{R}$ 为某种softmax归一化值（通常为 $\frac{1}{\sqrt{d}}$），mask为某种掩码函数，它会将输入的某些元素设置为 $-\infty$，而保持其他元素不变（例如当批次中的序列长度不一致且需要填充时所使用的键值填充掩码），而$\text{dropout}(x, p)$则会对 x 进行逐元素运算式的丢弃机制处理（即对于每个元素 x，以 $1 - p$ 的概率输出 $\frac{x}{1-p}$，以 p 的概率输出0）。

完整的算法内容见算法2。我们会为后向传播保存输出$\mathbf{O}$、Softmax函数的统计信息l 与 m，以及伪随机数生成器状态R。

Algorithm 2 FlashAttention Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}.

Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil, B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
    On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
    On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
    On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
    Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
    Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
  end for
end for
Return $\mathbf{O}, \ell, m, \mathcal{R}$.

B.4 FlashAttention: Backward Pass

We describe the full details of FlashAttention backward pass. Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, the output $\mathbf{O} \in \mathbb{R}^{N \times d}$, and the output gradient $\mathbf{dO}$, we want to compute the input gradients $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$.

We first describe the standard attention backward pass in Algorithm 3 for completeness.

Algorithm 3 Standard Attention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ in HBM.

Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$, write $\mathbf{dS}$ to HBM.
Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dSK}$, write $\mathbf{dQ}$ to HBM.
Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

We now make two observations about FlashAttention backward pass:

1. We do not need to store the dropout mask of size $O(N^2)$ from the forward pass. Instead, we can save the pseudo-random number generator states from the forward pass and re-generate the dropout mask in the backward pass. This allows us to only use $O(N)$ extra memory.

2. When computing the softmax gradient, we use Eq. (4) to compute $D_i = P_{i:}^\top dP_{i:}$ without reducing over $P_{i:}$ and $dP_{i:}$ of size N (they might not fit into SRAM). Instead we can rewrite $D_i = do_i^\top o_i$ and compute the dot product between vectors of size d.

Algorithm 2 FlashAttention Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}.

Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil, B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
    On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
    On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
    On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
    Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
    Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
  end for
end for
Return $\mathbf{O}, \ell, m, \mathcal{R}$.

B.4 FlashAttention算法：反向传播过程

我们详细阐述了FlashAttention算法中的标准注意力反向传播过程。给定输入序列$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V} \in \mathbb{R}^{N \times d}$，输出$\mathbf{O} \in \mathbb{R}^{N \times d}$以及输出梯度$\mathbf{dO}$，我们需要计算相应的输入梯度$\mathbf{dQ}$、$\mathbf{dK}$、$\mathbf{dV} \in \mathbb{R}^{N \times d}$。为便于理解，我们首先在算法3中介绍标准注意力反向传播过程。

算法3 标准注意力机制反向传播过程**需要：**存储在HBM中的矩阵 $\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$以及$\mathbf{dO} \in \mathbb{R}^{N \times d}$，还有 $\mathbf{P} \in \mathbb{R}^{N \times N}$。

Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$, write $\mathbf{dS}$ to HBM.
Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dSK}$, write $\mathbf{dQ}$ to HBM.
Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

现在，我们针对FlashAttention算法的后向传播给出两个观察结果：

1. 我们无需存储前向传播过程中生成的、大小为 $O(N^2)$的丢弃掩码。相反，我们可以保存前向传播时的伪随机数生成器状态，再在后向传播时重新生成该丢弃掩码。这样一来，我们仅需额外使用 $O(N)$的内存即可。

2. 在计算Softmax函数的梯度时，我们利用公式(4)来计算 $D_i = P_{i:}^\top dP_{i:}$，而无需对大小为 N 的$P_{i:}$ 和 $dP_{i:}$ 进行降维处理（因为这些数据可能无法完全存储在SRAM中）。相反，我们可以重新表述 $D_i = do_i^\top o_i$，进而计算大小为 d的向量之间的点积。

The full FLASHATTENTION backward pass algorithm is in Algorithm 4. Conceptually it is just a block version of the derivation in Appendix B.2.

Algorithm 4 FLASHATTENTION Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, pseudo-random number generator state $\mathcal{R}$ from the forward pass.

Set the pseudo-random number generator state to $\mathcal{R}$.
Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil$, $B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}, \mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  Initialize $\mathbf{d\tilde{K}}_j = (0)_{B_c \times d}, \mathbf{d\tilde{V}}_j = (0)_{B_c \times d}$ on SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
    On chip, compute $\mathbf{P}_{ij} = \text{diag}(l_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability p_{drop}.
    On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
    On chip, compute $\mathbf{d\tilde{V}}_j \leftarrow \mathbf{d\tilde{V}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
    On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
    On chip, compute $D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
    On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.
    On chip, compute $\mathbf{d\tilde{K}}_j \leftarrow \mathbf{d\tilde{K}}_j + \tau \mathbf{dS}_{ij}^\top \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
  end for
  Write $\mathbf{dK}_j \leftarrow \mathbf{d\tilde{K}}_j, \mathbf{dV}_j \leftarrow \mathbf{d\tilde{V}}_j$ to HBM.
end for
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

We see that similar to the forward pass, the backward pass performs $O(N^2)$ FLOPs and only requires $O(N)$ extra memory beyond inputs, output, output gradient, and input gradients.

We analyze the IO-complexity of the backward pass, similar to the forward pass (Theorem 2).

Theorem 5. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. Standard attention (Algorithm 0) backward pass requires $\Theta(Nd + N^2)$ HBM accesses, while* FLASHATTENTION *backward pass (Algorithm 4) requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses.*

The proof is in Appendix C.

完整的FlashAttention算法的后向传播过程记载于算法4中。从概念上来讲，它不过是附录B.2中所给出推导过程的块化版本而已。

算法4 FlashAttention算法的后向传播

所需条件： 位于高带宽内存中的$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$、$\mathbf{O}$、$\mathbf{dO} \in \mathbb{R}^{N \times d}$大小的矩阵，位于高带宽内存中的$l, m \in \mathbb{R}^N$大小的向量，以及容量为$M$的片上静态随机存取存储器。

Softmax归一化常量 $\tau \in \mathbb{R}$、掩码函数mask、丢弃概率 p_{drop}、伪随机数来自前向传播的数值生成器状态R。
将伪随机数生成器的状态设置为R。
设置块大小为 $B_c = \left\lceil \frac{M}{4d} \right\rceil$、$B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$。
将$\mathbf{Q}$ 分成 $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ 个大小为 $B_r \times d$ 的$\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ 的块，同时将$\mathbf{K}$、$\mathbf{V}$ 也分成 $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ 个块。$\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ 以及$\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$，其大小为 $B_c \times d$ 个/个。
将$\mathbf{O}$ 拆分为 T_r 个B尺寸的块 $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$，其大小为 $B_r \times d$ 个，再将$\mathbf{dO}$ 拆分为 T_r 这些B尺寸的输出梯度。$i, \dots, \mathbf{dO}_{T_r}$ 其大小为 $B_r \times d$ 每个元素都需要被l分割成 T_r 个B尺寸的l_i，即……，l_{T_r} 大小为 B_r 。each，将 m 拆分为 T 个部分 i 块数为 $m_1, \dots, m_{T_r}$ 其大小为 B_r 每一个。
首先在高带宽内存中初始化 $\mathbf{dQ} = (0)_{N \times d}$，然后将其划分为 T_r 个块，即 $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ 每个块的大小为 $B_r \times d$ 。接下来进行初始化操作。
$\mathbf{dK} = (0)_{N \times d}$，$\mathbf{dV} = (0)_{N \times d}$在高带宽内存中，然后将$\mathbf{dK}$、$\mathbf{dV}$划分为 T_c个$\mathbf{dK}_1$,……、$\mathbf{dK}_{T_c}$以及$\mathbf{dV}_1$,……、$\mathbf{dV}_{T_c}$。大小为 $B_c \times d$ 中的每一个。
for $1 \le j \le T_c$ **do**
  将$\mathbf{K}_j$、$\mathbf{V}_j$从高带宽内存加载到片上静态随机存取存储器中。
  初始化 $\mathbf{d\tilde{K}}_j = (0)_{B_c \times d}$, $\mathbf{d\tilde{V}}_j = (0)_{B_c \times d}$ 在静态随机存取存储器上。
  对于 $1 \le i \le T_r$，**执行循环**
    将$\mathbf{Q}_i$、$\mathbf{O}_i$、$\mathbf{dO}_i$、$\mathbf{dQ}_i$以及l_i等 m_i从高带宽内存加载到片上静态随机存取存储器中。
    在芯片上直接计算$\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$。
    在芯片上，计算 $\mathbf{S}_{ij}^{\text{masked}} = \text{mask}(\mathbf{S}_{ij})$。
    在芯片上计算 $\mathbf{P}_{ij} = \text{diag}(l_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$。
    在芯片上，计算丢弃掩码 $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$，其中每个元素的值为 $\frac{1}{1-p_{\text{drop}}}$ ——— 以一定的概率以 p_{drop}的概率输出$1 - p_{\text{drop}}$ 以及值0。
    在芯片上，计算$\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$（逐点相乘）。
    在芯片上，计算$\mathbf{d\tilde{V}}_j \leftarrow \mathbf{d\tilde{V}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$。
    在芯片上计算$\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$。
    在芯片上，对$\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}}$与$\mathbf{Z}_{ij}$进行逐点相乘运算。
    在芯片上，计算 $D_i =$ 行和运算$(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$。
    在芯片上，计算 $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$。
    将$\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$写入高带宽内存中。
    在芯片上直接进行计算 $\mathbf{d\tilde{K}}_j \leftarrow \mathbf{d\tilde{K}}_j + \tau \mathbf{dS}_{ij}^\top \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$。
  结束循环
  将$\mathbf{dK}$写入高带宽内存。$_j \leftarrow \mathbf{d\tilde{K}}_j, \mathbf{dV}_j \leftarrow \mathbf{d\tilde{V}}_j$ 至高带宽内存。
结束循环
返回 $\mathbf{dQ}$、$\mathbf{dK}$、$\mathbf{dV}$。

我们发现，与前向传播类似，后向传播也会执行 $O(N^2)$次浮点运算次数，且除了输入、输出、输出梯度以及输入梯度之外，仅需要$O(N)$的额外内存。与前向传播相同，我们也对后向传播的I/O复杂度进行了分析（见定理2）。

定理5。设 N 为序列长度， d 为头维度， M 为具有 $d \le M \le Nd$参数的*SRAM*容量。标准注意力机制（算法*0*）的后向传播需要 $\Theta(Nd + N^2)$ 次*HBM*访问，而FlashAttention算法的后向传播（算法*4*）则需要 $\Theta(N^2 d^2 M^{-1})$ 次*HBM*访问。

相关证明见附录C。

B.5 Comparison with Rabe and Staats [66]

We describe here some similarities and differences between our FlashAttention algorithm and the algorithm of Rabe and Staats [66].

Conceptually, both FlashAttention and Rabe and Staats [66] operate on blocks of the attention matrix using the well-established technique of tiling (or softmax scaling) [51, 60]. To reduce the memory footprint, both methods avoid storing the large attention matrix in the forward pass and recompute it in the backward pass.

The first major difference is that Rabe and Staats [66] focuses on the reducing the total memory footprint (maximum amount of GPU memory required) while FlashAttention focuses on reducing memory accesses (the number of memory reads/writes). As mentioned in Section 2, the amount of memory access is the primary determining factor of runtime. Reducing memory accesses also necessarily reduces the total amount of memory required (e.g., if an operation incurs A memory accesses, then its total memory requirement is at most A). As a result, FlashAttention is faster than standard attention (2-4×) while Rabe and Staats [66] is around the same speed or slightly slower than standard attention. In terms of total memory required, both methods offer substantial memory saving.

The second difference between the two methods is the way information is summarized from each block to pass to the next block. Rabe and Staats [66] summarizes each block with its temporary output along with the softmax normalization statistics. At the end of the forward pass, the temporary outputs of all the blocks are combined using the statistics to produce the final output. FlashAttention instead incrementally updates the output (Algorithm 1 line 12) after processing each block, so only one copy of the output is needed (instead of K copies for K blocks). This means that FlashAttention has smaller total memory requirement compared to Rabe and Staats [66].

The final major difference is the way the backward pass is computed. Rabe and Staats [66] uses gradient checkpointing to recompute the attention matrix and the temporary output of each block. FlashAttention instead simplifies the backward pass analytically (Appendices B.2 and B.4). It only recomputes the attention matrix and does not recompute the temporary output of each block. This reduces the memory requirement for the backward pass and yields speedup.

C Proofs

Proof of Theorem 1. We first count the number of FLOPs and extra memory required.

The dominating FLOPs are from matrix multiplication. In the inner loop, (Algorithm 1 line 9), we compute $\mathbf{Q}_i\mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$ for $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}_j \in \mathbb{R}^{B_c \times d}$, which takes $O(B_r B_c d)$ FLOPs. We also compute (Algorithm 1 line 12) $\tilde{\mathbf{P}}_{ij}\mathbf{V}_j \in \mathbb{R}^{B_r \times d}$ for $\tilde{\mathbf{P}}_{ij} \in \mathbb{R}^{B_r \times B_c}$ and $\mathbf{V}_j \in \mathbb{R}^{B_c \times d}$, which takes $O(B_r B_c d)$ FLOPs. We execute the inner loops $T_c T_r = \left\lceil \frac{N}{B_c} \right\rceil \left\lceil \frac{N}{B_r} \right\rceil$ times. Therefore the total number of FLOPs is

$$O\left(\frac{N^2}{B_c B_r} B_r B_c d\right) = O(N^2 d).$$

In terms of extra memory required, we see that we need $O(N)$ memory to store the statistics (ℓ, m).

We now prove the algorithm's correctness by induction on j for $0 \le j \le T_c$. Let $\mathbf{K}_{:j} \in \mathbb{R}^{jB_c \times d}$ be the first jB_c rows of $\mathbf{K}$, and similarly $\mathbf{V}_{:j} \in \mathbb{R}^{jB_c \times d}$ the the first jB_c rows of $\mathbf{V}$. Let $\mathbf{S}_{:,:j} = \mathbf{Q}\mathbf{K}_{:j}^\top \in \mathbb{R}^{N \times jB_c}$, and $\mathbf{P}_{:,:j} = \mathrm{softmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^{N \times jB_c}$ (softmax applied row-wise). Let $m^j, \ell^{(j)}, \mathbf{O}^{(j)}$ be the values of $m, \ell, \mathbf{O}$ in HBM after the j-th iteration of the outer loop (Algorithm 1 line 5). (Note that these values of $m, \ell, \mathbf{O}$ are updated after each iteration of the outer loop.) We want to show that after the j-th iteration of the outer loop, we have computed in HBM:

$$m^{(j)} = \mathrm{rowmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^N, \quad \ell^{(j)} = \mathrm{rowsum}(\exp(\mathbf{S}_{:,:j} - m^{(j)})) \in \mathbb{R}^N, \quad \mathbf{O}^{(j)} = \mathbf{P}_{:,:j}\mathbf{V}_{:j} \in \mathbb{R}^{N \times d}.$$

Based on our initialization (Algorithm 1 line 2), this claim is true for $j = 0$ (i.e., before the any iteration of the outer loop is executed). Suppose that the claim holds for some $j = 0, \ldots, T_c - 1$. We want to show that the claim also holds for $j + 1$. Indeed, when we update the statistics in the inner loop (Algorithm 1 line 10)

B.5 与Rabe和Staats的对比 [66]

此处我们将介绍我们的FlashAttention算法与Rabe和Staats的算法 [66]之间存在的一些相似之处与差异。

从概念上讲，FlashAttention算法与Rabe和Staats的算法 [66] 都采用成熟的平铺技术（或softmax归一化） [51, 60]，对注意力矩阵的各个块进行处理。为降低内存占用，这两种方法均避免在前向传播阶段存储庞大的注意力矩阵，而是在后向传播时重新计算它。

第一个主要区别在于，Rabe和Staats [66] 侧重于降低总内存占用（即所需的GPU内存最大量），而FlashAttention则着眼于减少内存访问次数（即内存读写操作的频次）。如第2节所述，内存访问次数是决定运行时间的关键因素。减少内存访问次数必然也会降低所需的总内存量（例如，如果某项操作会产生 A 次内存访问，那么它所需的总内存量最多为 A）。正因如此，FlashAttention的运行速度比标准注意力机制快2-4×，而Rabe和Staats [66]的运行速度与标准注意力机制大致相同或略慢一些。在所需总内存方面，这两种方法都能实现显著的内存节省。

这两种方法之间的第二个差异在于信息从每个块汇总并传递到下一个块的方式。Rabe和Staats [66] 会将该块的临时输出以及Softmax归一化统计量一同汇总起来。在前向传播结束时，利用这些统计量将所有块的临时输出合并，从而生成最终输出。而FlashAttention则在处理完每个块后逐步更新输出（算法1的第12行），因此仅需一份输出副本（而非 K 个块所需的 K 份副本）。这意味着与Rabe和Staats相比，FlashAttention的总内存需求更小 [66]。

最后一个主要差异在于后向传播的计算方式。Rabe和Staats [66] 采用梯度检查点技术来重新计算注意力矩阵以及每个块的临时输出。而FlashAttention则通过解析方法简化了后向传播过程（见附录B.2和B.4），它仅重新计算注意力矩阵，而不需要重新计算每个块的临时输出。这样一来既降低了后向传播所需的内存占用，又提升了运算速度。

C语言证明

定理*1*的证明。我们首先统计所需的浮点运算次数以及额外内存占用量 .

The dominating FLOPs are from matrix multiplication. In the inner loop, (Algorithm 1 line 9), we compute $\mathbf{Q}_i\mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$ for $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}_j \in \mathbb{R}^{B_c \times d}$, which takes $O(B_r B_c d)$ FLOPs. We also compute (Algorithm 1 line 12) $\tilde{\mathbf{P}}_{ij}\mathbf{V}_j \in \mathbb{R}^{B_r \times d}$ for $\tilde{\mathbf{P}}_{ij} \in \mathbb{R}^{B_r \times B_c}$ and $\mathbf{V}_j \in \mathbb{R}^{B_c \times d}$, which takes $O(B_r B_c d)$ FLOPs. We execute the inner loops $T_c T_r = \left\lceil \frac{N}{B_c} \right\rceil \left\lceil \frac{N}{B_r} \right\rceil$ times. Therefore the total number of FLOPs is

$$O\left(\frac{N^2}{B_c B_r} B_r B_c d\right) = O(N^2 d).$$

就外存使用量而言就所需的存储内存而言，我们可以看出需要 $O(N)$容量的SRAM来存储这些统计信息$(l, \ m)$。

We now prove the algorithm's correctness by induction on j for $0 \le j \le T_c$. Let $\mathbf{K}_{:j} \in \mathbb{R}^{jB_c \times d}$ be the first jB_c rows of $\mathbf{K}$, and similarly $\mathbf{V}_{:j} \in \mathbb{R}^{jB_c \times d}$ the the first jB_c rows of $\mathbf{V}$. Let $\mathbf{S}_{:,:j} = \mathbf{Q}\mathbf{K}_{:j}^\top \in \mathbb{R}^{N \times jB_c}$, and $\mathbf{P}_{:,:j} = \mathrm{softmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^{N \times jB_c}$ (softmax applied row-wise). Let $m^j, \ell^{(j)}, \mathbf{O}^{(j)}$ be the values of $m, \ell, \mathbf{O}$ in HBM after the j-th iteration of the outer loop (Algorithm 1 line 5). (Note that these values of $m, \ell, \mathbf{O}$ are updated after each iteration of the outer loop.) We want to show that after the j-th iteration of the outer loop, we have computed in HBM:

$$m^{(j)} = \mathrm{rowmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^N, \quad \ell^{(j)} = \mathrm{rowsum}(\exp(\mathbf{S}_{:,:j} - m^{(j)})) \in \mathbb{R}^N, \quad \mathbf{O}^{(j)} = \mathbf{P}_{:,:j}\mathbf{V}_{:j} \in \mathbb{R}^{N \times d}.$$

根据我们的初始化方式（算法1的第2行），该结论在 $j = 0$ 时是成立的（即在外层循环的任何一次迭代执行之前）。假设该结论对某些 $j = 0$、.....、 $T_c - 1$也成立。我们想要证明该结论对 $j + 1$同样成立。事实上，当我们在内层循环中更新统计信息时（算法1的第10行），这一结论依然成立。

on the $(j+1)$-th iteration of the outer loop, we update $m^{(j+1)} = \max(m^{(j)}, \tilde{m})$ where $\tilde{m} \in \mathbb{R}^N$ is the row-max of $\mathbf{S}_{:,j:j+1}$, the slice of $\mathbf{S}$ from column jB_c to column $(j+1)B_c - 1$. This implies that

$$m^{(j+1)} = \text{rowmax}(\mathbf{S}_{:,:j+1}) \in \mathbb{R}^N.$$

Similarly, we update

$$\ell^{(j+1)} = e^{m^{(j)} - m^{(j+1)}} \ell^{(j)} + e^{\tilde{m} - m^{(j+1)}} \tilde{\ell},$$

where $\tilde{\ell} = \text{rowsum}(\exp(\mathbf{S}_{:,j:j+1} - \tilde{m})) \in \mathbb{R}^N$. By the same algebraic manipulation in Section 3.1, we obtain:

$$\ell^{(j+1)} = \text{rowsum}(\exp(\mathbf{S}_{:,:j+1} - m^{(j+1)})) \in \mathbb{R}^N.$$

Let $\mathbf{V}_{j:j+1}$ be the slice of $\mathbf{V}$ from column jB_c to column $(j+1)B_c - 1$, we also update:

$$\begin{aligned}
\mathbf{O}^{(j+1)} &= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{O}^{(j)} + e^{\tilde{m} - m^{(j+1)}} \exp(\mathbf{S}_{j:j+1} - \tilde{m})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{P}_{:,:j}\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\text{diag}(\ell^{(j)})\exp(\mathbf{S}_{:,:j} - m^{(j)})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(e^{-m^{(j+1)}} \exp(\mathbf{S}_{:,:j})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\exp(\mathbf{S}_{:,:j} - m^{(j+1)})\mathbf{V}_{:j} + \exp(\mathbf{S}_{j:j+1} - m^{(j+1)})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}\left(\exp\left(\begin{bmatrix}\mathbf{S}_{:,:j} & \mathbf{S}_{j:j+1}\end{bmatrix} - m^{(j+1)}\right)\right)\begin{bmatrix}\mathbf{V}_{:j} \\ \mathbf{V}_{j:j+1}\end{bmatrix} \\
&= \text{softmax}(\mathbf{S}_{:j+1})\mathbf{V}_{:j+1}.
\end{aligned}$$

We then see that the claim is also true for $j+1$. By induction, the claim is true for all $j = 0, \ldots, T_c$.

When $j = T_c$, we conclude that the final value of $\mathbf{O}$ in HBM is $\text{softmax}(\mathbf{S})\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$.

□

Proof of Theorem 2. We first analyze the IO complexity of standard attention implementation. The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ reside in HBM, and the at the end of the algorithm the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ is written to HBM.

In the first step of computing the matrix multiply $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, the inputs $\mathbf{Q}, \mathbf{K}$ are read from HBM and the output $\mathbf{S} \in \mathbb{R}^{N \times N}$ is written to HBM (Algorithm 0 line 1). This incurs $\Theta(Nd + N^2)$ HBM accesses.

In the second step of computing $\mathbf{P} = \text{softmax}(\mathbf{S})$, the input $\mathbf{S}$ is read from HBM and the output $\mathbf{P}$ is written to HBM (Algorithm 0 line 2). This incurs $\Theta(N^2)$ HBM accesses.

In the last step of computing $\mathbf{O} = \mathbf{P}\mathbf{V}$, the inputs $\mathbf{P}, \mathbf{V}$ are read from global memory and the output $\mathbf{O}$ is written to HBM (Algorithm 0 line 3). This incurs $\Theta(Nd + N^2)$ HBM accesses.

Overall, standard attention implementation requires $\Theta(Nd + N^2)$ global memory accesses.

We now analyze the IO complexity of streaming attention.

Following Algorithm 1, we see that each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded from HBM once (Algorithm 1 line 6). We make T_c passes over $\mathbf{Q}$ and $\mathbf{O}$, each pass loading all of $\mathbf{Q}$ and all of $\mathbf{O}$ to HBM (Algorithm 1 line 8). Therefore the number of HBM accesses is $\Theta(Nd + NdT_c) = \Theta(NdT_c)$.

We derive the conditions on the block sizes B_c and B_r. We need the blocks $\mathbf{K}_j$ and $\mathbf{V}_j$ of size $B_c \times d$ to fit into on-chip memory, which translates to:

$$B_c d = O(M) \Leftrightarrow B_c = O\left(\frac{M}{d}\right).$$

Similarly, we need the blocks $\mathbf{Q}_i, \mathbf{O}_i$ of size $B_r \times d$ to fit into on-chip memory, which translates to:

$$B_r d = O(M) \Leftrightarrow B_r = O\left(\frac{M}{d}\right).$$

Finally, we need the block $\mathbf{S}_{ij}$ of size $B_r \times B_c$ to fit into on-chip memory, which translates to:

$$B_r B_c = O(M).$$

在外层循环的（$j+1$）次迭代中，我们会更新 $m^{(j+1)}$=$\max(m^{(j)}, \tilde{m})$，其中 $\tilde{m} \in \mathbb{R}^N$指的是$\mathbf{S}_{:,j:j+1}$的行最大值，即$\mathbf{S}$从第 jB_c列到第（$j+1$）$B_c - 1$列的子矩阵。这意味着

$$m^{(j+1)} = \text{rowmax}(\mathbf{S}_{:,:j+1}) \in \mathbb{R}^N.$$

同样地，我们也会进行更新。

$$\ell^{(j+1)} = e^{m^{(j)} - m^{(j+1)}} \ell^{(j)} + e^{\tilde{m} - m^{(j+1)}} \tilde{\ell},$$

其中 l =行和运算 $(\exp(\mathbf{S}_{:,j:j+1} - \tilde{m})) \in \mathbb{R}^N$. 通过3.1节中相同的代数运算，我们可以得到 in:

$$\ell^{(j+1)} = \text{rowsum}(\exp(\mathbf{S}_{:,:j+1} - m^{(j+1)})) \in \mathbb{R}^N.$$

令$\mathbf{V}_{j:j+1}$表示从第 jB_c列到第$(j+1)B_c - 1$列的$\mathbf{V}$切片，同时我们也会进行更新：

$$\begin{aligned}
\mathbf{O}^{(j+1)} &= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{O}^{(j)} + e^{\tilde{m} - m^{(j+1)}} \exp(\mathbf{S}_{j:j+1} - \tilde{m})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{P}_{:,:j}\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\text{diag}(\ell^{(j)})\exp(\mathbf{S}_{:,:j} - m^{(j)})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(e^{-m^{(j+1)}} \exp(\mathbf{S}_{:,:j})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\exp(\mathbf{S}_{:,:j} - m^{(j+1)})\mathbf{V}_{:j} + \exp(\mathbf{S}_{j:j+1} - m^{(j+1)})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}\left(\exp\left(\begin{bmatrix}\mathbf{S}_{:,:j} & \mathbf{S}_{j:j+1}\end{bmatrix} - m^{(j+1)}\right)\right)\begin{bmatrix}\mathbf{V}_{:j} \\ \mathbf{V}_{j:j+1}\end{bmatrix} \\
&= \text{softmax}(\mathbf{S}_{:j+1})\mathbf{V}_{:j+1}.
\end{aligned}$$

我们进一步发现该结论对 $j+1$也同样成立。通过归纳法可证，该结论对所有的 j = 0、.....、T_c均成立。

当 $j = T_c$时，我们可以得出结论：HBM中$\mathbf{O}$ 的最终值为$\text{Softmax}(\mathbf{S})\mathbf{V} = \text{Softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$。

□

Proof of Theorem 2. We first analyze the IO complexity of standard attention implementation. The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ reside in HBM, and the at the end of the algorithm the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ is written to HBM.

In the first step of computing the matrix multiply $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, the inputs $\mathbf{Q}, \mathbf{K}$ are read from HBM and the output $\mathbf{S} \in \mathbb{R}^{N \times N}$ is written to HBM (Algorithm 0 line 1). This incurs $\Theta(Nd + N^2)$ HBM accesses.

在计算$\mathbf{P}$ =Softmax($\mathbf{S}$)的第二步中，输入$\mathbf{S}$ 会从高带宽内存中读取，而输出$\mathbf{P}$ 则会被写入高带宽内存中（算法0的第2行）。这一过程会产生 $\Theta(N^2)$次HBM访问。

在计算$\mathbf{O}$ = $\mathbf{P}\mathbf{V}$的最后一步中，输入$\mathbf{P}$、$\mathbf{V}$ 会从全局内存中读取，而输出$\mathbf{O}$ 则会被写入高带宽内存中（算法0的第3行）。这一过程会产生 $\Theta(Nd + N^2)$次HBM访问。

Ov总体而言，标准注意力机制的实现需要 $\Theta(Nd + N^2)$次全局内存访问。 .

接下来我们分析流式注意力机制的输入输出复杂度。

根据算法1的描述，$\mathbf{K}$ 与$\mathbf{V}$ 中的每个元素都会从高带宽内存中加载一次（算法1第6行）。我们会对$\mathbf{Q}$ 和$\mathbf{O}$进行 T_c 次传递，每次传递都会将所有的$\mathbf{Q}$ 数据以及所有的$\mathbf{O}$ 数据加载到高带宽内存中（算法1第8行）。因此，HBM的访问次数为 $\Theta($ $Nd + NdT_c) = \Theta(NdT_c)$。

我们推导出了关于块大小 B_c和 B_r的约束条件。为使大小为 $B_c \times d$的$\mathbf{K}_j$以及$\mathbf{V}_j$这两个块能够容纳在片上内存中，需要满足如下要求：

$$B_c d = O(M) \Leftrightarrow B_c = O\left(\frac{M}{d}\right).$$

同样地，w 为使大小为 $B_r \times d$ 的$\mathbf{Q}_i$、$\mathbf{O}_i$ 这两个块能够容纳在片上内存中，我们也有相应的需求。

$$B_r d = O(M) \Leftrightarrow B_r = O\left(\frac{M}{d}\right).$$

最终 为此，我们需要将大小为 $B_r \times B_c$ 的$\mathbf{S}_{ij}$ 块纳入片上内存中，换算下来即 :

$$B_r B_c = O(M).$$

We therefore set:

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, \frac{M}{B_c}\right)\right) = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

We then have:

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

As a result, the number of HBM accesses is:

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

Proof of Proposition 3. For contradiction, suppose that there exists an algorithm that computes exact attention where the number for HBM access for all $M \in [d, Nd]$ is

$$o\left(\frac{N^2d^2}{M}\right).$$

In the regime of $M = \Theta(Nd)$, this results in the number of HBM accesses:

$$o\left(\frac{N^2d^2}{Nd}\right) = o(Nd).$$

However, the input to attention (matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}$) and the output $\mathbf{O}$ have size Nd and they start out being in HBM, so if the algorithm computes exact attention it must incur at least $\Omega(Nd)$ HBM accesses. This is a contradiction. □

Proof of Theorem 5. The IO complexity of the attention backward is very similar to the IO complexity of the attention forward (Theorem 2). Here we provide a sketch of the proof.

We first analyze the IO complexity of standard attention backward pass. The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ reside in HBM, and the at the end of the algorithm the outputs $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ are written to HBM.

At each step of the standard attention backward pass, one needs to load inputs of size Nd or N^2 from HBM, and needs to write the outputs of size N^2 or Nd to HBM. This incurs $\Theta(Nd + N^2)$ HBM accesses.

We now analyze the IO complexity of FlashAttention backward pass.

Similar to Theorem 2, we see that each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded from HBM once. Each element of $\mathbf{dK}$ and $\mathbf{dV}$ is only written to HBM once. We make T_c passes over $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$, each pass loading all of $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$ to HBM. We also make T_c passes over $\mathbf{dQ}$, each pass reading/writing all of $\mathbf{dQ}$ from/to HBM. Therefore the number of HBM accesses is $\Theta\left(Nd + NdT_c\right) = \Theta(NdT_c)$.

As in the proof of Theorem 2, the constraints on the block sizes are that:

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

We then have:

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

As a result, the number of HBM accesses is:

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

所以我们设定：

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, \frac{M}{B_c}\right)\right) = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

接下来我们有：

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

因此，HBM访问次数为：

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

命题3的证明。为进行反证，假设存在这样一种算法：它能够计算出精确的注意力机制，且所有 $M \in [d, Nd]$ 对应的HBM访问次数均为

$$o\left(\frac{N^2d^2}{M}\right).$$

在 $M = \Theta(Nd)$的条件下，这将导致HBM访问次数的值为：

$$o\left(\frac{N^2d^2}{Nd}\right) = o(Nd).$$

然而，注意力机制的输入（$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$这些矩阵）与输出$\mathbf{O}$的规模均为 Nd，且它们最初都存储在高带宽内存中；因此，如果该算法要计算精确的注意力机制，就必然至少需要进行 $\Omega(Nd)$次HBM访问。这显然存在矛盾。□

定理5的证明。 注意力机制的向后传播过程所具有的输入输出复杂度与向前传播过程的输入输出复杂度极为相似（见定理2）。此处我们给出该证明的概要。

我们首先分析标准注意力机制标准反向传播过程的输入输出复杂度。算法的输入$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$以及$\mathbf{dO} \in \mathbb{R}^{N \times d}$均存储在高带宽内存中，而在算法结束时，输出$\mathbf{dQ}$、$\mathbf{dK}$、$\mathbf{dV} \in \mathbb{R}^{N \times d}$也会被写入高带宽内存中。

在标准注意力反向传播过程的每一步中，都需要从高带宽内存中读取大小为 Nd 或 N^2 的输入数据，同时要将大小为 N^2 或 Nd 的输出数据写入高带宽内存。这一过程会引发 $\Theta(Nd + N^2)$次HBM访问。

接下来我们将分析FlashAttention算法在反向传播过程中的输入输出复杂度。

Similar to Theorem 2, we see that each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded from HBM once. Each element of $\mathbf{dK}$ and $\mathbf{dV}$ is only written to HBM once. We make T_c passes over $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$, each pass loading all of $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$ to HBM. We also make T_c passes over $\mathbf{dQ}$, each pass reading/writing all of $\mathbf{dQ}$ from/to HBM. Therefore the number of HBM accesses is $\Theta\left(Nd + NdT_c\right) = \Theta(NdT_c)$.

与定理2的证明中所提到的相同，关于块大小的约束条件为：

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

接下来我们有：

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

因此，HBM访问次数为：

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

Algorithm 5 Block-Sparse FlashAttention Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$, block sparsity mask $M \in \{0,1\}^{N/B_r \times N/B_c}$..

Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  for $1 \le i \le T_r$ **do**
    if $M_{ij} \neq 0$ **then**
      Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
      On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
      On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
      On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
      On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
      On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
      Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
      Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
    end if
  end for
end for
Return $\mathbf{O}, \ell, m, \mathcal{R}$.

D Extension Details

D.1 Block-sparse FlashAttention

We describe the full block-sparse FlashAttention algorithm in Algorithm 5. The algorithm is identical to Algorithm 2, except that we skip zero blocks.

We prove the IO-complexity of block-sparse FlashAttention.

Proof of Proposition 4. The proof is very similar to the proof of Theorem 2. For the block-sparse case, notice that we only need to load blocks corresponding to nonzero blocks. As a result, the number of HBM accesses are scaled by s, the fraction of nonzero blocks in the block-sparsity mask. However, for small values of s, we would still need to write the result $\mathbf{O} \in \mathbb{R}^{N \times d}$. Therefore the number of HBM accesses is

$$\Theta\left(Nd + \frac{N^2 d^2}{M} s\right).$$

□

D.2 Potential Extensions

We discuss here a few potential extensions of the IO-aware approach to speed up deep learning training.

Multi-GPU Attention. Large language models are trained on hundreds or thousands of GPUs, and one typically splits the attention computation between 4-8 GPUs on the same node [77]. This introduces another level of memory hierarchy: beside GPU SRAM and GPU HBM, we also have the HBM of other

Algorithm 5 Block-Sparse FlashAttention Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$, block sparsity mask $M \in \{0,1\}^{N/B_r \times N/B_c}$..

Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  for $1 \le i \le T_r$ **do**
    if $M_{ij} \neq 0$ **then**
      Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
      On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
      On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
      On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
      On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
      On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
      Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
      Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
    end if
  end for
end for
Return $\mathbf{O}, \ell, m, \mathcal{R}$.

D 扩展详情

D.1 分块稀疏FlashAttention算法

我们在算法5中详细描述了完整的分块稀疏FlashAttention算法。该算法与算法2基本相同，不同之处在于它跳过了那些值为零的块。

我们证明了分块稀疏FlashAttention算法的I/O复杂度。

命题*4*的证明。该证明与定理2的证明极为相似。在块稀疏情况下，只需加载对应于非零块的那些块即可。因此，HBM访问次数会按照 s这一块稀疏掩码中非零块所占的比例来变化。不过，当 s的数值较小时，我们仍然需要将结果写入$\mathbf{O} \in \mathbb{R}^{N \times d}$的形式。所以HBM访问次数的具体数值为

$$\Theta\left(Nd + \frac{N^2 d^2}{M} s\right).$$

□

D.2 潜在的扩展方向

我们在此探讨的是 即针对具备IO感知能力的算法思路，探索若干能够加速深度学习训练的扩展方案。 .

多GPU注意力机制。大型语言模型是在数百甚至数千颗GPU上训练的，通常会将注意力计算任务分配到同一节点上的4到8颗GPU之间来完成 [77]。这样一来就形成了另一层内存层次结构：除了GPU SRAM和GPU高带宽内存之外，还有其他GPU所配备的高带宽内存。

GPUs. For very long sequences, the different GPUs on the same node can cooperate to compute attention by taking into account the asymmetry of different levels of memory hierarchy.

Sparse MLP layers. Typical dense MLP layers are compute-bound and not memory-bound. To improve their efficiency, MLP layers with sparse weight matrices can be used [17]. However, many sparse MLP layers are instead memory-bound, and their speedup is often not proportional to the sparsity. We believe that an IO-aware implementation can alleviate this issue and realize the benefits of sparsity. We are excited about future work in this direction, to reduce the computational requirement of large models and improve their wall-block runtime.

Kernel machine learning. Our approach in FlashAttention relies on the fact that the $N \times N$ attention matrix is a function of a low-rank matrix $\mathbf{Q}\mathbf{K}^\top$ (of rank $d \ll N$). As a result, we can repeatedly load the inputs $\mathbf{Q}, \mathbf{K}$ and recompute the block of the attention matrix that we need, significantly reducing HBM access. As similar scenario happens in kernel machine learning: each element K_{ij} of the $N \times N$ kernel matrix $\mathbf{K}$ is a function of two vectors of size $d \ll N$, as it measures the similarity between two datapoints x_i and x_j. The KeOps library [8, 26] is a successful example of how reducing memory reads/writes can speed up kernel operations. We hope that this will motivate kernel methods that focus more on reducing IOs instead of just FLOPs.

E Full Experimental Results

E.1 BERT

We train BERT-large following the training procedure and hyperparameters of the reference MLPerf 1.1 implementation. In particular, we use the LAMB optimizer with learning rate 3.75e-3, with batch size 448, trained for at most 7100 steps. The training is stopped once the validation accuracy (for masked language modeling) reaches the target 72.0%, and the wall-clock run-time is measured. We train with FP16 precision using Apex AMP (with O2 optimization level).

We compare our results with the reported training speed from Nvidia that was submitted to MLPerf 1.1 (Table 1).

We use the same train / validation data split provided by MLPerf 1.1 reference implementation. In particular, we evaluate on the same 10000 validation examples as the baseline from Nvidia.

We train the model on 8×A100-80GB GPUs. Each training run takes between 16 and 19 minutes, and we average the results of 10 runs.

E.2 GPT-2

We use the standard implementations of GPT-2 [67] from Huggingface `transformers` library and from Nvidia's Megatron-LM repo. We follow the training recipe of the Megatron-LM repo.

We use an effective batch size of 512, and use gradient accumulation to fit into available GPU memory. We use the AdamW optimizer, with learning rate 6e-4 for GPT-2 small and 1.5e-4 for GPT-2 medium, and weight decay of 0.1. All models are trained with the same hyperparameters for 400K steps. We run all implementations with mixed-precision training (PyTorch AMP).

We use the Openwebtext dataset, with the GPT-2 BPE tokenizer. We randomly select 0.5% of the dataset as the validation set, with the rest being used as training set. This random selection of validation set is done once, and all models are evaluated on the same validation set.

We train the model on 8×A100-40GB GPUs, and we measure the wall-clock training time. Training GPT-2 small takes between 2.7-9.5 days, and training GPT-2 medium takes between 6.9-21.0 days (Table 2).

In Fig. 4, we plot of the validation perplexity throughout training of GPT-2 small/medium, using either HuggingFace implementation or our FlashAttention implementation. We see that FlashAttention behaves the same as the baseline implementation and the validation perplexity curves of the two implementations almost lie on top of each other.

Long Document Classification. For MIMIC-III and ECtHR, we follow the hyperparameters of Dai et al. [13].

对于序列长度极长的情况，同一节点上的不同GPU可以通过充分考虑各层内存层次结构之间的差异性，协同完成注意力计算。

稀疏MLP层。典型的密集型MLP层属于计算受限型而非内存受限型。为提升其效率，可采用具有稀疏权重矩阵的MLP层 [17]。然而，许多稀疏MLP层实际上属于内存受限型，且其加速效果往往与稀疏度不成正比。我们认为，引入具备I/O感知能力的实现方式能够缓解这一问题，从而充分发挥稀疏性的优势。我们对未来在这一方向上的研究充满期待，期望借此降低大型模型的计算需求，并提升其运行时间。

Kernel machine learning. Our approach in FlashAttention relies on the fact that the $N \times N$ attention matrix is a function of a low-rank matrix $\mathbf{Q}\mathbf{K}^\top$ (of rank $d \ll N$). As a result, we can repeatedly load the inputs $\mathbf{Q}, \mathbf{K}$ and recompute the block of the attention matrix that we need, significantly reducing HBM access. As similar scenario happens in kernel machine learning: each element K_{ij} of the $N \times N$ kernel matrix $\mathbf{K}$ is a function of two vectors of size $d \ll N$, as it measures the similarity between two datapoints x_i and x_j. The KeOps library [8, 26] is a successful example of how reducing memory reads/writes can speed up kernel operations. We hope that this will motivate kernel methods that focus more on reducing IOs instead of just FLOPs.

E 完整实验结果

E.1 BERT

我们依据参考的MLPerf 1.1实现方案所规定的训练流程与超参数来训练BERT-large模型。具体而言，我们采用学习率为3.75e-3、批量大小为448的LAMB优化器，训练步数最多不超过7100步。一旦掩码语言建模任务的验证准确率达到72.0%的目标值，训练就会停止，并同时记录实际运行的墙钟时间。在整个训练过程中，我们均使用FP16精度并通过Apex AMP（配合O2优化级别）来进行运算。

我们将自己的实验结果与英伟达提交给MLPerf 1.1测试标准的训练速度数据进行了对比（见表1）。

我们采用了MLPerf 1.1参考实现所提供的相同训练/验证数据划分方式。具体而言，我们的评估所使用的10000个验证样本与英伟达提供的基准测试版本完全一致。

我们在 8×台A100-80GB GPU上训练该模型。每次训练耗时在16到19分钟之间，最终我们会取10次训练结果的平均值。

E.2 GPT-2模型

我们采用了来自HuggingFace transformers库以及英伟达Megatron-LM仓库的GPT-2 [67] 标准实现版本，并遵循了Megatron-LM仓库中的训练方案。

我们采用的有效批量大小为512，并通过梯度累积的方式来确保训练能在现有的GPU内存范围内进行。优化器方面选用了AdamW，其中GPT-2小型模型的学习率为6e-4，GPT-2中型模型的学习率为1.5e-4，权重衰减系数则为0.1。所有模型均使用相同的超参数进行训练，训练步数为400K步。此外，所有的实现方案都采用了混合精度训练技术（PyTorch AMP）。

我们使用OpenWebText数据集，并搭配GPT-2模型的BPE分词器。我们从该数据集中随机抽取0.5%作为验证集，其余部分则用作训练集。验证集的随机选取仅进行一次，所有模型都会在同一个验证集上接受评估。

我们在 8×台配备A100-40GB GPU的硬件上对模型进行训练，并记录实际训练时间。训练GPT-2小型模型的所需时间在2.7天到9.5天之间，而训练GPT-2中型模型则需要6.9天到21.0天不等（见表2）。

在图4中，我们展示了使用HuggingFace实现或我们自研的FlashAttention算法对GPT-2小型/中型模型进行训练过程中的验证困惑度变化情况。可以看出，FlashAttention算法的验证困惑度曲线与基准实现的结果基本一致，两者几乎完全重合。

长文档分类。针对MIMIC-III和欧洲人权法院的数据集，我们采用了Dai等人设定的超参数[13]。

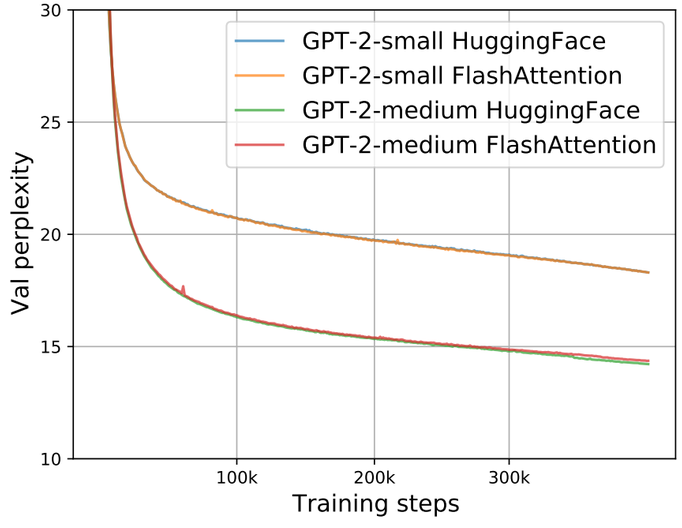


Figure 4: Validation perplexity of GPT-2 small/medium using two implementations. We confirm that FlashAttention yields the same validation curves as the baseline implementation from HuggingFace.

E.3 LRA details

We follow the hyperparameters from the Long-range arena paper [80], the Long-range arena repo (`https://github.com/google-research/long-range-arena`), and the Nyströmformer reproduction [90]. To be generous to the baseline methods, if we are unable to reproduce the performance of any baseline for any of the five tasks, we report the better performance from Tay et al. [80] or Xiong et al. [90] for that baseline on that task.

After hyperparameter tuning, almost all of the attention methods achieve similar accuracy on all of the five LRA tasks.

We run all methods with mixed-precision training, except for Performer (not stable with mixed precision) and Local Attention (implementation does not support FP16).

To calculate the overall wallclock-time speedup, we take the geometric mean of the wallclock-time speedup of each of the five tasks.

Path-X For Path-X and Path-256, we follow the hyperparameters from the PathFinder-32 experiments from the long-range arena paper[80]. For both, we first pretrain a model on Path-64. We take the checkpoint after 200 epochs, upsample its positional embedding (we duplicate the positional embeddings gridwise in space), and fine-tune it on the downstream task for 200 epochs with one epoch of linear warmup, and cosine decay of the learning rate. For Path-X, we take the best performing checkpoint (according to val accuracy), and additionally fine-tune it for 200 epochs with the same warmup and learning rate (this adds roughly 4 points of accuracy to FlashAttention for Path-X, but the model starts overfitting afterwards).

E.4 Comparison with Apex FMHA

We compare our method/implementation with Apex FMHA (`https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha`).

When we started this project, Apex FMHA was the fastest implementation of attention (that we knew of), tailored for short sequences of length at most 512. In fact, almost all MLPerf submissions for BERT training benchmark running on Nvidia GPUs use FMHA for their model code, as of MLPerf 1.1 [58]. Since

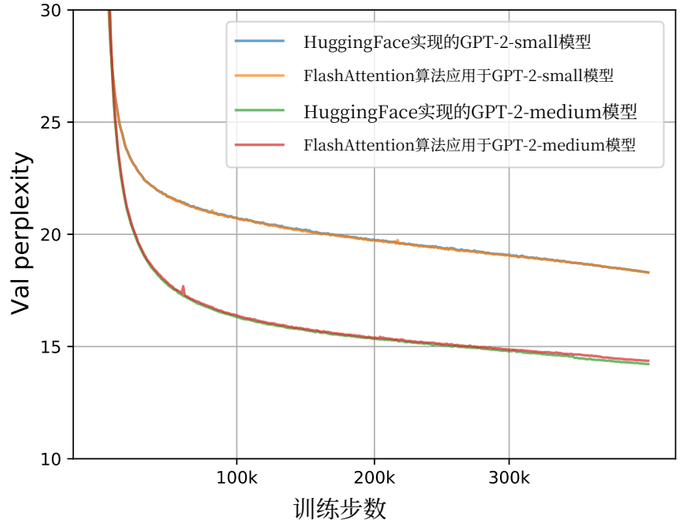


图4：使用两种不同实现得到的GPT-2 small/medium模型的验证困惑度曲线。实验结果表明，FlashAttention算法生成的验证曲线与HuggingFace提供的基准实现结果完全一致。

E.3 长距离注意力相关细节

我们的超参数参考了《长距离任务集》论文 [80], 、Long-range arena代码仓库（https://github.com/google-research/long-range-arena）以及Nyströmformer算法的复现结果 [90]。为对基线方法保持公平，若我们无法在五项任务中的任何一项上复现某基线方法的性能，便会采用Tay等人 [80] 或Xiong等人 [90] 在该任务上为该基线方法取得的更好性能作为替代结果。

在完成超参数调优后，几乎所有的注意力机制在五种长距离注意力任务上的准确率都相当接近。

除Performer模型（使用混合精度训练时稳定性较差）和局部注意力机制（其实现不支持FP16格式）外，其余所有方法均采用混合精度训练进行测试。

为了计算小于符号的值为了得到整体的壁钟时间加速比，我们会取这五种任务各自壁钟时间加速比的几何平均值。这五种任务中每一种的…… sks.

Path-X 在Path-X和Path-256的实验中，我们采用了《长距离任务集》论文[80]中PathFinder-32实验所设定的超参数。对于这两种任务，我们首先在Path-64数据集上对模型进行预训练，选取训练200轮后的检查点，对其位置嵌入进行上采样处理（即在空间上按网格方式复制位置嵌入），随后在下游任务上再进行200轮的微调，过程中包含1轮的线性热身阶段以及学习率的余弦衰减策略。对于Path-X，我们会选择在验证集准确率方面表现最佳的检查点，然后使用相同的预热方式和学习率再对其进行200轮的微调（这一操作能为Path-X版本的FlashAttention提升大约4点的准确率，但模型随后会开始出现过拟合现象）。

E.4 与Apex FMHA的对比

我们将自己的方法/实现与Apex FMHA（https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha）进行了对比。

在启动这个项目时，Apex FMHA是我们所知最快的注意力机制实现方案，专为长度最多为512的短序列设计。事实上，截至MLPerf 1.1 [58]版本，几乎所有在英伟达GPU上运行BERT训练基准测试的MLPerf参赛作品，其模型代码都会采用FMHA。

Table 7: Runtime (ms) of FlashAttention compared to FMHA by sequence length, with masking and dropout, measured on an A100-SXM4-40GB GPU. Batch size 64, 16 heads, head dimension 64 (i.e., BERT-large size).

Attention Method	128	256	512
Apex FMHA forward	0.10	0.29	1.14
FlashAttention forward	**0.08**	**0.22**	**0.81**
Apex FMHA backward	**0.17**	**0.52**	**1.81**
FlashAttention backward	0.20	0.53	2.00
Apex FMHA forward + backward	**0.27**	0.81	2.95
FlashAttention forward + backward	0.28	**0.75**	**2.81**

FMHA targets BERT models, it only supports head dimension 64, and only runs on A100 GPUs. FMHA fuses the attention computation dropout(softmax(mask($\mathbf{QK}^\top$)))$\mathbf{V}$ into one CUDA kernel. In the forward pass, it stores the attention matrix softmax(mask($\mathbf{QK}^T$)) to HBM to be used in gradient computation. As a result, it does not offer substantial memory saving (though for shorter sequences memory footprint is often not a primary concern).

We use FMHA code as a starting point, and apply two well-established techniques (tiling and recomputation) to deal with long sequences and to save memory as mentioned in Section 3. As a result, we can support much longer sequences (e.g., up to length 64K). We also support more head dimensions (16, 32, 64, 128) and broader GPU types (all Turing and Ampere GPUs at the time of writing).

In Table 7, we compare the performance of FlashAttention and Apex FMHA for short sequences (as FMHA only supports sequence length at most 512). Generally FlashAttention is slightly faster than FMHA in the forward pass and slightly slower than FMHA in the backward pass. This is because we do not store the attention matrix in the forward pass and recompute it in the backward pass. Compared to FMHA, the overall runtime of FlashAttention is about 4% slower for sequence length 128, 8% faster for sequence length 256, and 5% faster for sequence length 512.

E.5 Speedup On Different Hardware and Configurations

Speedup varies between different types of GPU types and generations depending on HBM bandwidth and SRAM size. In this section, we profile FlashAttention speedup on different GPUs and configurations.

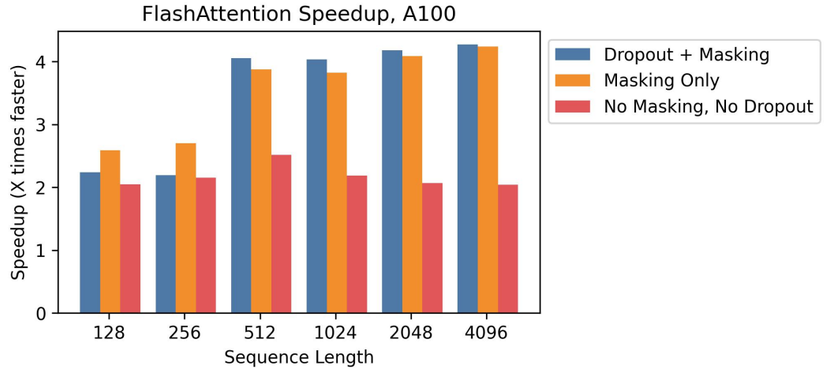


Figure 5: Speedup over standard PyTorch attention at different sequence lengths, on A100.

A100 Figure 5 shows speedup on an A100 GPU with batch size 8, head dimension 64, and 12 attention heads, across different sequence lengths. We generally see 2-4× speedup, and we see more speedup when using dropout and masking due to kernel fusion.

表7：在A100-SXM4-40GB GPU上，针对不同序列长度，对比了带掩码处理与丢弃机制的FlashAttention算法与FMHA算法的运行时间（单位：毫秒）。此时批量大小为64，模型包含16个头部，每个头部的维度为64，即对应BERT-large模型的参数规模。

注意力机制	128	256	512
Apex FMHA前向计算	0.10	0.29	1.14
FlashAttention前向计算	**0.08**	**0.22**	**0.81**
Apex FMHA反向计算	**0.17**	**0.52**	**1.81**
FlashAttention反向计算	0.20	0.53	2.00
Apex FMHA前向 +后向	**0.27**	0.81	2.95
FlashAttention前向传播 +反向传播	0.28	**0.75**	**2.81**

FMHA针对BERT模型设计，仅支持头维度为64的情况，并且只能在A100 GPU上运行。该算法将注意力机制的计算过程——即dropout(softmax(mask($\mathbf{QK}^\top$)))$\mathbf{V}$ ——整合到一个CUDA内核中。在前向传播过程中，它会将注意力矩阵softmax(mask($\mathbf{QK}^T$))存储到高带宽内存中，以便在梯度计算时使用。因此，它并不能实现显著的内存节省（尽管对于较短的序列而言，内存占用通常并非首要考虑因素）。

我们以FMHA代码作为起点，并如第3节所述，运用两种成熟的技术——平铺技术与重新计算技术——来处理长序列并节省内存。借此，我们能够支持更长的序列（例如长度可达64K）。此外，我们还支持更多的头维度（16、32、64、128），以及更多类型的GPU（在撰写本文时涵盖所有Turing架构和Ampere架构的GPU）。

在表格7中，我们对比了FlashAttention与Apex FMHA在短序列上的性能表现（因为FMHA仅支持最多512的序列长度）。总体而言，FlashAttention在前向传播时的速度略快于FMHA，而在后向传播时的速度则略慢于FMHA。这是因为我们在前向传播时不存储注意力矩阵，而是在后向传播时重新计算它。与FMHA相比，当序列长度为128时，FlashAttention的运行时间大约慢4%；当序列长度为256时，其速度快8%；而当序列长度为512时，则快5%。

E.5 不同硬件与配置条件下的加速效果

由于高带宽内存的带宽大小及静态随机存取存储器的容量不同，不同类型及不同代次的GPU所带来的加速程度也会存在差异。在本节中，我们将详细介绍FlashAttention算法在各类GPU及不同配置下的加速表现。

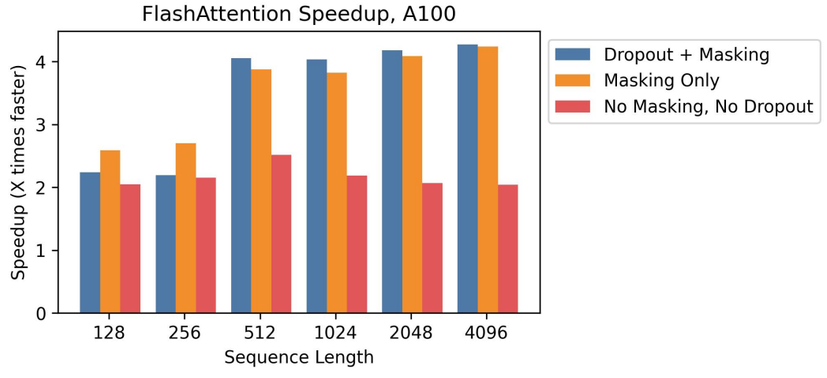


图5：在A100显卡上，针对不同序列长度，相较于标准PyTorch注意力机制的加速比。

A100 图5展示了在A100显卡上，当批量大小为8、头部维度为64且拥有12个注意力头时，不同序列长度下的加速效果。通常情况下我们能够获得2-4× 倍的加速，而通过结合丢弃机制与掩码处理以及内核融合技术，加速效果还会进一步提升。

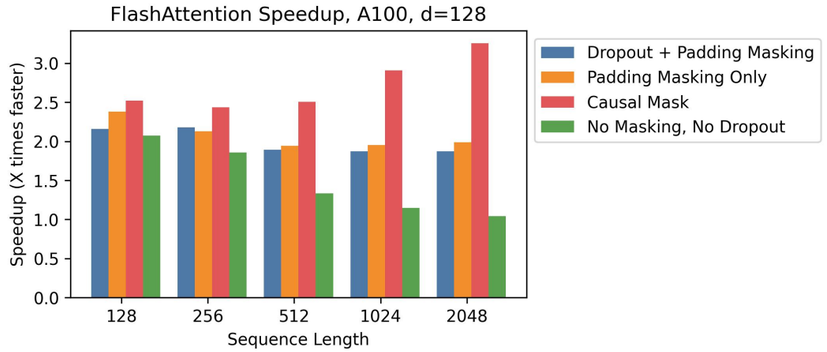


Figure 6: Speedup over standard PyTorch attention at different sequence lengths, on A100, with head dimension 128.

A100, Head Dimension 128 Speedup also changes when we increase the head dimension. Each block requires more memory, so we need to use smaller block sizes to fit into SRAM. Figure 6 shows speedup with head dimension 128 on an A100 (batch size 16, 12 heads). We see less speedup overall—but we can still see significant speedup (up to 3×) with a causal mask, where half the blocks are masked out.

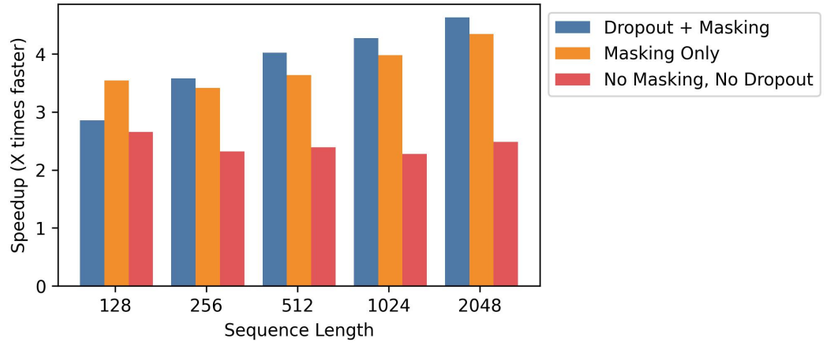


Figure 7: Speedup over standard PyTorch attention at different sequence lengths, on RTX 3090.

RTX 3090 Figure 7 shows speedup on an RTX 3090 GPU. Here, we use batch size 12 with 12 attention heads. We observe slightly higher speedups on the RTX 3090 (between 2.5-4.5×), since the memory bandwidth on an RTX 3090 is lower than on an A100 (roughly 900 GB/s vs. 1.5 TB/s).

T4 Figure 8 shows speedup on a T4 GPU. T4 SRAM is smaller than A100, so we need to make the block sizes smaller in FlashAttention. As a result, we observe less speedup on T4, which matches the IO complexity analysis in Section 3.2. T4 GPUs are commonly used for inference, so we also report speedup on the forward pass only.

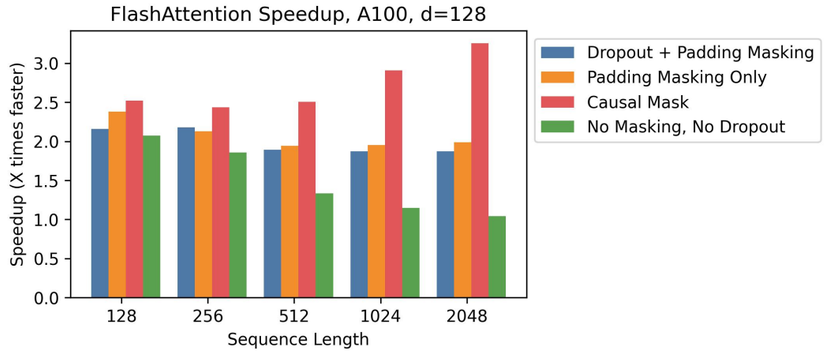


图6：在A100显卡上，头部维度为128时，不同序列长度下相较于标准PyTorch注意力机制的加速比。

A100, 头部维度128 当头部维度增大时，加速程度也会随之变化。由于每个块需要消耗更多的内存，因此我们必须采用更小的块大小才能将其存储在静态随机存取存储器中。图6显示了在A100显卡上（批量大小为16，拥有12个注意力头），头部维度为128时的加速情况。虽然整体加速幅度有所下降，但若使用因果掩码将一半的块屏蔽掉，依然能够实现相当可观的加速效果（最高可达 3×倍）。

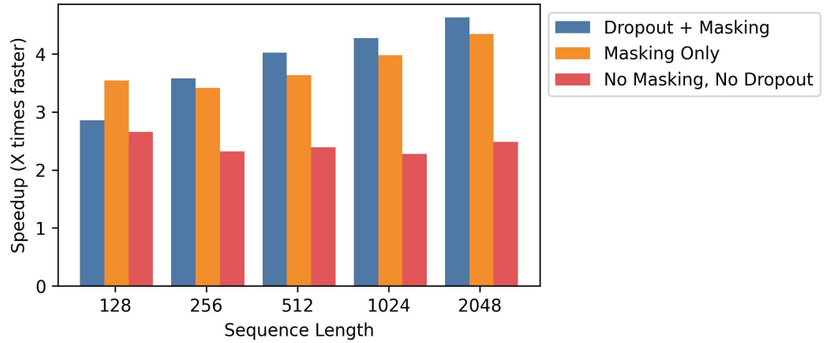


图7：在RTX 3090芯片上，不同序列长度下相较于标准PyTorch注意力机制的加速比。

RTX 3090 图7展示了在RTX 3090 GPU上的加速效果。实验中我们采用批量大小为12且拥有12个注意力头的配置。由于RTX 3090的内存带宽低于A100（约为900 GB/s，而A100为1.5 TB/s），因此在RTX 3090上获得的加速比略高，介于2.5到4.5×之间。

T4 图8展示了在T4 GPU上的加速效果。由于T4的SRAM容量小于A100，因此在FlashAttention算法中不得不缩小块大小。正因如此，我们在T4上观察到的加速幅度较低，这一结果与3.2节中的输入输出复杂度分析相符。鉴于T4 GPU通常用于推理任务，此处我们也仅报告了前向传播阶段的加速情况。

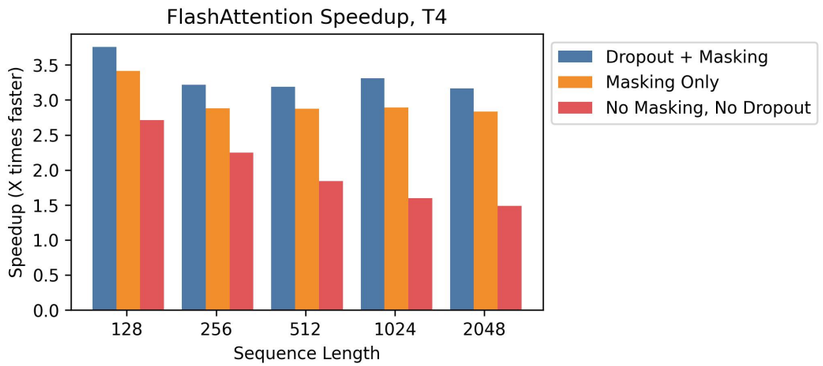




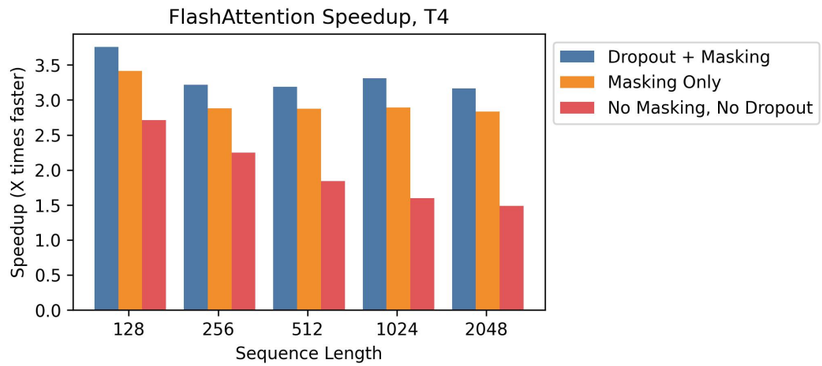
Figure 8: Speedup over standard PyTorch attention at different sequence lengths, on T4. **Top:** Combined forward pass + backward pass. **Bottom:** Forward pass only.

E.6 Full Benchmarking Results

We report the full benchmarking results and experimental details on A100.

Baselines We compare against reference implementations for exact attention from PyTorch/HuggingFace and Megatron, approximate attention, and sparse attention. For approximate attention, we compare against reference implementations of Reformer [51], Local Attention [68], Linformer Attention [84], Smyrf [19], and LongShortFormer (LSFormer) [94]. For sparse attention, we compare against reference implementations of Block-Sparse Attention form OpenAI [11], Longformer[3], and BigBird Attention [92]. For the approximate and sparse attention, we use a compression ratio of 1/8, or a compressed sequence length of 256, whichever is smaller.

Setup We measure runtime and memory usage of the attention computation with 8 heads of dimension 64, and batch size 16 on a machine with one A100 GPU with 40 GB of GPU HBM. We vary sequence length in our experiments. We compute attention on random vectors for **Q**, **K**, and **V** (we do not measure the projection from the hidden layer). For dropout, we use dropout 0.1; for masking, we use a padding mask with uniformly-random mask lengths between the total sequence length and the total sequence length minus 20. To measure runtime, we take the average of 100 measurements of the attention call. We only measure memory footprint once, since it does not vary between runs.



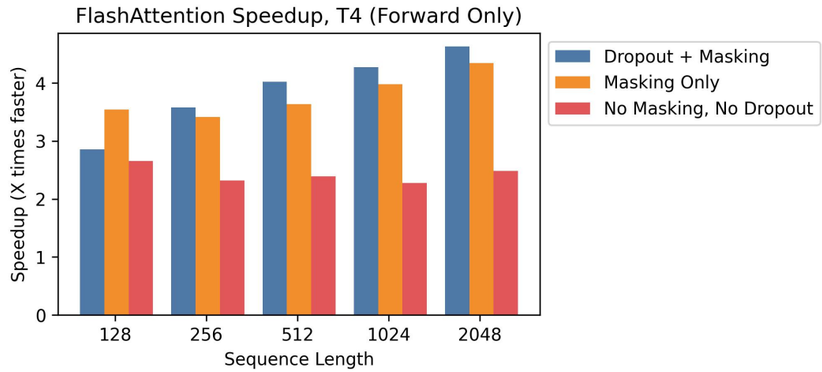


图8：在T4芯片上，不同序列长度下相较于标准PyTorch注意力机制的加速比。**上方部分：** 包含 +前向传播与后向传播的完整过程。**下方部分：** 仅进行前向传播的情况。

E.6 完整基准测试结果

关于A100平台的完整基准测试结果及实验细节，我们将在文中详细阐述。

基准模型 我们对比了来自PyTorch/HuggingFace和Megatron的精确注意力机制参考实现、近似注意力机制以及稀疏注意力机制的性能。在近似注意力机制方面，我们将其与Reformer [51], 、局部注意力机制 [68], 、Linformer注意力机制 [84], 、SMYRF [19], 以及LongShortFormer（LSFormer） [94]的参考实现进行了比较。对于稀疏注意力机制，我们则对比了OpenAI提出的Block-Sparse Attention [11], 、Longformer[3], 以及BigBird注意力机制 [92]的参考实现。在处理近似注意力机制和稀疏注意力机制时，我们采用的压缩比为1/8，对应的压缩后序列长度为256，以两者中较小的值为准。

Setup We measure runtime and memory usage of the attention computation with 8 heads of dimension 64, and batch size 16 on a machine with one A100 GPU with 40 GB of GPU HBM. We vary sequence length in our experiments. We compute attention on random vectors for **Q**, **K**, and **V** (we do not measure the projection from the hidden layer). For dropout, we use dropout 0.1; for masking, we use a padding mask with uniformly-random mask lengths between the total sequence length and the total sequence length minus 20. To measure runtime, we take the average of 100 measurements of the attention call. We only measure memory footprint once, since it does not vary between runs.

Table 8: Pointers to results tables.

Dropout	Masking	Pass	Table
Yes	Yes	Forward	Table 9
Yes	Yes	Backward	Table 10
Yes	Yes	Combined	Table 11
No	Yes	Forward	Table 12
No	Yes	Backward	Table 13
No	Yes	Combined	Table 14
Yes	No	Forward	Table 15
Yes	No	Backward	Table 16
Yes	No	Combined	Table 17
No	No	Forward	Table 18
No	No	Backward	Table 19
No	No	Combined	Table 20
No	No	Memory Usage (Combined)	Table 21

Table 9: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout and masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.36	0.34	0.78	2.54	9.33	36.33	-	-	-	-
Megatron	0.40	0.40	1.10	3.65	16.19	-	-	-	-	-
Reformer	2.03	3.15	5.67	11.02	22.59	46.14	97.38	212.13	-	-
Local Attention	0.83	0.86	1.01	2.20	7.13	14.32	28.60	57.79	117.67	-
Linformer	0.67	0.52	0.69	0.71	1.65	3.18	6.15	12.16	24.17	52.39
Smyrf	2.27	2.34	3.91	7.44	14.71	29.22	58.27	116.41	-	-
LSformer	1.18	1.27	1.34	3.38	11.40	22.55	44.95	89.76	179.66	-
Block Sparse	1.12	1.11	2.13	2.77	6.95	20.91	-	-	-	-
Longformer	1.22	1.14	1.08	1.95	5.72	12.98	-	-	-	-
BigBird	1.13	1.12	1.12	1.77	6.03	13.68	-	-	-	-
FlashAttention	**0.04**	0.06	0.21	0.82	2.85	10.41	41.74	167.19	670.76	2682.35
Block-Sparse FlashAttention	0.06	**0.06**	**0.06**	**0.12**	**0.44**	**0.86**	**1.70**	**3.29**	**6.55**	**13.34**

We report timing results on the forward pass, backward pass, and combined forward + backward pass. We measure each method with and without dropout, masking, or both—except for Block Sparse, Longformer, and BigBird. These methods did not successfully run the backward pass with masking due to a bug in external libraries, so we measured them without masking to be generous. We use FP16 for all measurements, except for Local Attention, whose implementation only supports FP32.

For each baseline, we increase sequence length until it runs out of memory on the GPU, except for the following exceptions: The Megatron implementation does not support sequence lengths longer than 2048. Block-Sparse (OpenAI) does not support sequence lengths longer than 4096. Longformer and BigBird do not support sequence lengths longer than 8092.

We measure memory usage on the combined forward + backward pass, without dropout or masking.

Results Table 8 summarizes all the experimental configurations and contains pointers to the results tables.

表8：各结果表格的链接。

丢弃机制	掩码处理	Pass	表格
Yes	Yes	前向传播	表9
Yes	Yes	反向传播	表10
Yes	Yes	组合模式	表11
No	Yes	前向传播	表12
No	Yes	反向传播	表13
No	Yes	组合方式	表14
Yes	No	前向传播	表15
Yes	No	反向传播	表16
Yes	No	组合方式	表17
No	No	前向传播	表18
No	No	反向传播	表19
No	No	综合值	表20
No	No	总内存占用	表21

表9：按序列长度划分的各类精确/近似/稀疏注意力机制的前向传播运行时间（毫秒），**已考虑丢弃机制与掩码处理**。表现最佳的选项以**粗体**标出，次优的选项则用下划线标示。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.36	0.34	0.78	2.54	9.33	36.33	-	-	-	-
Megatron架构	0.40	0.40	1.10	3.65	16.19	-	-	-	-	-
Reformer架构	2.03	3.15	5.67	11.02	22.59	46.14	97.38	212.13	-	-
局部注意力机制	0.83	0.86	1.01	2.20	7.13	14.32	28.60	57.79	117.67	-
Linformer模型	0.67	0.52	0.69	0.71	1.65	3.18	6.15	12.16	24.17	52.39
SMYRF模型	2.27	2.34	3.91	7.44	14.71	29.22	58.27	116.41	-	-
LSformer模型	1.18	1.27	1.34	3.38	11.40	22.55	44.95	89.76	179.66	-
块稀疏结构	1.12	1.11	2.13	2.77	6.95	20.91	-	-	-	-
Longformer模型	1.22	1.14	1.08	1.95	5.72	12.98	-	-	-	-
BigBird模型	1.13	1.12	1.12	1.77	6.03	13.68	-	-	-	-
FlashAttention算法	**0.04**	0.06	0.21	0.82	2.85	10.41	41.74	167.19	670.76	2682.35
分块稀疏FlashAttention算法	0.06	**0.06**	**0.06**	**0.12**	**0.44**	**0.86**	**1.70**	**3.29**	**6.55**	**13.34**

我们报告了关于前向传播、后向传播以及结合前向 + 后向传播的计时测试结果。对于每种方法，我们都分别测试了启用和禁用丢弃机制、掩码处理，或是同时启用这两种措施的情况——不过Block Sparse、Longformer和BigBird模型除外。由于外部库中的缺陷，这些模型在启用掩码处理的情况下无法成功完成后向传播，因此出于宽容考虑，我们仅对它们在未启用掩码处理时的性能进行了测试。所有测试均采用FP16格式进行，局部注意力机制除外，因其实现仅支持FP32格式。

对于每一种基准模型，我们都会逐步增加序列长度，直到GPU内存耗尽为止；不过也存在一些例外情况：Megatron架构不支持长度超过2048的序列，Block-Sparse（OpenAI版）不支持长度超过4096的序列，而Longformer模型和BigBird模型则不支持长度超过8092的序列。

我们会测量m的值即在未应用丢弃机制及掩码的条件下，综合计算前向 +后向传播过程中的GPU内存使用量。 g.

结果表格8汇总了所有的实验配置，并提供了对应结果表格的引用链接。

Table 10: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout and masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.37	0.49	1.66	5.81	22.32	87.67	-	-	-	-
Megatron	0.35	0.32	0.77	2.42	8.43	-	-	-	-	-
Reformer	2.37	4.59	8.91	17.68	35.13	70.05	140.01	-	-	-
Local Attention	0.55	0.62	1.49	4.03	13.78	27.61	55.20	110.27	221.40	-
Linformer	0.89	0.80	0.81	0.93	2.48	4.75	9.29	18.27	36.53	-
Smyrf	1.41	2.83	5.43	10.72	21.25	42.31	84.48	168.95	-	-
LSformer	1.75	1.76	3.01	7.50	20.07	39.08	76.39	150.82	-	-
Block Sparse	1.29	1.28	2.18	3.04	7.27	21.16	-	-	-	-
Longformer	1.27	1.31	1.29	2.04	5.24	10.74	25.95	-	-	-
BigBird	1.33	1.28	1.32	1.81	5.55	11.44	27.45	-	-	-
FlashAttention	**0.30**	**0.26**	0.68	2.02	6.84	26.89	105.70	418.96	1666.89	6660.44
Block-Sparse FlashAttention	**0.30**	0.27	**0.29**	**0.59**	**1.50**	**2.94**	**5.82**	**11.85**	**23.98**	**47.61**

Table 11: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout and masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.84	0.86	2.35	8.29	31.75	124.19	-	-	-	-
Megatron	0.87	0.89	1.33	4.21	16.50	-	-	-	-	-
Reformer	4.30	7.76	14.60	28.74	57.79	116.34	237.57	-	-	-
Local Attention	1.40	1.60	2.06	6.06	20.94	42.01	84.08	168.48	339.45	-
Linformer	1.57	1.49	1.55	1.60	4.19	8.04	15.71	30.92	61.47	-
Smyrf	3.41	5.08	9.35	18.18	36.03	71.68	143.04	285.87	-	-
LSformer	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
Block Sparse	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FlashAttention	**0.43**	**0.41**	0.95	2.55	9.56	37.49	147.75	586.61	2339.11	9341.30
Block-Sparse FlashAttention	0.44	0.44	**0.45**	**0.89**	**1.95**	**4.12**	**7.64**	**16.60**	**32.73**	**64.11**

Table 12: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.30	0.30	0.63	1.93	7.08	27.45	112.90	-	-	-
Megatron	0.45	0.41	0.43	1.52	5.80	-	-	-	-	-
Reformer	1.87	3.00	5.37	10.43	21.40	43.83	92.80	203.24	-	-
Local Attention	0.70	0.81	1.02	2.09	6.64	13.34	26.77	54.02	110.11	-
Linformer	0.63	0.50	0.67	0.65	1.36	2.60	5.04	9.92	19.69	43.47
Smyrf	2.38	2.32	3.76	7.16	14.14	28.09	55.98	111.73	-	-
LSformer	1.22	1.29	1.44	3.28	10.99	21.72	43.29	86.32	172.76	-
Block Sparse	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FlashAttention	**0.03**	**0.04**	0.17	0.68	2.28	8.40	33.55	134.14	537.50	2150.88
Block-Sparse FlashAttention	0.05	**0.04**	**0.05**	**0.11**	**0.35**	**0.68**	**1.33**	**2.54**	**5.34**	**10.73**

Table 13: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.44	0.46	1.53	5.33	20.34	79.87	-	-	-	-
Megatron	0.29	0.31	0.65	1.95	6.49	-	-	-	-	-
Reformer	2.31	4.47	8.68	17.20	34.14	68.09	136.02	-	-	-
Local Attention	0.51	0.62	1.30	3.81	13.33	26.72	53.41	106.82	214.15	-
Linformer	0.76	0.81	0.94	0.87	2.24	4.25	8.35	16.38	32.67	72.11
Smyrf	1.34	2.77	5.30	10.46	20.73	41.27	82.41	164.86	-	-
LSformer	1.66	1.61	3.09	7.42	19.68	38.35	74.92	147.86	-	-
Block Sparse	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FlashAttention	**0.21**	**0.22**	0.62	1.84	5.77	22.25	86.21	338.91	1343.91	5361.09
Block-Sparse FlashAttention	0.22	0.22	**0.26**	**0.57**	**1.55**	**3.13**	**5.98**	**12.21**	**23.49**	**47.85**

表10：按序列长度划分的各种精确/近似/稀疏注意力机制的后向传播运行时间（毫秒），**已考虑丢弃机制与掩码处理**。最佳结果以**粗体**标出，次优结果则用下划线标示。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.37	0.49	1.66	5.81	22.32	87.67	-	-	-	-
Megatron架构	0.35	0.32	0.77	2.42	8.43	-	-	-	-	-
Reformer架构	2.37	4.59	8.91	17.68	35.13	70.05	140.01	-	-	-
局部注意力机制	0.55	0.62	1.49	4.03	13.78	27.61	55.20	110.27	221.40	-
Linformer模型	0.89	0.80	0.81	0.93	2.48	4.75	9.29	18.27	36.53	-
SMYRF模型	1.41	2.83	5.43	10.72	21.25	42.31	84.48	168.95	-	-
LSformer模型	1.75	1.76	3.01	7.50	20.07	39.08	76.39	150.82	-	-
块稀疏结构	1.29	1.28	2.18	3.04	7.27	21.16	-	-	-	-
Longformer模型	1.27	1.31	1.29	2.04	5.24	10.74	25.95	-	-	-
BigBird模型	1.33	1.28	1.32	1.81	5.55	11.44	27.45	-	-	-
FlashAttention算法	**0.30**	**0.26**	0.68	2.02	6.84	26.89	105.70	418.96	1666.89	6660.44
分块稀疏FlashAttention算法	**0.30**	0.27	**0.29**	**0.59**	**1.50**	**2.94**	**5.82**	**11.85**	**23.98**	**47.61**

表11：按序列长度划分的各类精确/近似/稀疏注意力机制的前向传播 + 后向传播运行时间（毫秒），**已考虑丢弃机制与掩码处理**。表现最佳的选项以**粗体**标出，次优的选项则用下划线标注。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.84	0.86	2.35	8.29	31.75	124.19	-	-	-	-
Megatron架构	0.87	0.89	1.33	4.21	16.50	-	-	-	-	-
Reformer架构	4.30	7.76	14.60	28.74	57.79	116.34	237.57	-	-	-
局部注意力机制	1.40	1.60	2.06	6.06	20.94	42.01	84.08	168.48	339.45	-
Linformer模型	1.57	1.49	1.55	1.60	4.19	8.04	15.71	30.92	61.47	-
SMYRF模型	3.41	5.08	9.35	18.18	36.03	71.68	143.04	285.87	-	-
LSformer模型	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
块稀疏结构	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer模型	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird模型	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FlashAttention算法	**0.43**	**0.41**	0.95	2.55	9.56	37.49	147.75	586.61	2339.11	9341.30
分块稀疏FlashAttention算法	0.44	0.44	**0.45**	**0.89**	**1.95**	**4.12**	**7.64**	**16.60**	**32.73**	**64.11**

表12：按序列长度划分的各类精确/近似/稀疏注意力机制的前向传播运行时间（毫秒），**已进行掩码处理**。表现最佳的用**粗体**标出，次优的则用下划线标出。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.30	0.30	0.63	1.93	7.08	27.45	112.90	-	-	-
Megatron架构	0.45	0.41	0.43	1.52	5.80	-	-	-	-	-
Reformer架构	1.87	3.00	5.37	10.43	21.40	43.83	92.80	203.24	-	-
局部注意力机制	0.70	0.81	1.02	2.09	6.64	13.34	26.77	54.02	110.11	-
Linformer模型	0.63	0.50	0.67	0.65	1.36	2.60	5.04	9.92	19.69	43.47
SMYRF模型	2.38	2.32	3.76	7.16	14.14	28.09	55.98	111.73	-	-
LSformer模型	1.22	1.29	1.44	3.28	10.99	21.72	43.29	86.32	172.76	-
块稀疏结构	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer模型	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird模型	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FlashAttention算法	**0.03**	**0.04**	0.17	0.68	2.28	8.40	33.55	134.14	537.50	2150.88
分块稀疏FlashAttention算法	0.05	**0.04**	**0.05**	**0.11**	**0.35**	**0.68**	**1.33**	**2.54**	**5.34**	**10.73**

表13：按序列长度划分的各类精确/近似/稀疏注意力机制的后向传播运行时间（毫秒），**已进行掩码处理**。表现最佳的选项以**粗体**标出，次优的选项则用下划线标示。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.44	0.46	1.53	5.33	20.34	79.87	-	-	-	-
Megatron架构	0.29	0.31	0.65	1.95	6.49	-	-	-	-	-
Reformer架构	2.31	4.47	8.68	17.20	34.14	68.09	136.02	-	-	-
局部注意力机制	0.51	0.62	1.30	3.81	13.33	26.72	53.41	106.82	214.15	-
Linformer模型	0.76	0.81	0.94	0.87	2.24	4.25	8.35	16.38	32.67	72.11
SMYRF模型	1.34	2.77	5.30	10.46	20.73	41.27	82.41	164.86	-	-
LSformer模型	1.66	1.61	3.09	7.42	19.68	38.35	74.92	147.86	-	-
块稀疏结构	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer模型	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird模型	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FlashAttention算法	**0.21**	**0.22**	0.62	1.84	5.77	22.25	86.21	338.91	1343.91	5361.09
分块稀疏FlashAttention算法	0.22	0.22	**0.26**	**0.57**	**1.55**	**3.13**	**5.98**	**12.21**	**23.49**	**47.85**

Table 14: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.80	0.81	2.08	7.23	27.51	107.58	-	-	-	-
Megatron	0.81	0.83	1.09	3.36	12.39	-	-	-	-	-
Reformer	4.16	7.46	14.06	27.68	55.66	112.15	229.37	-	-	-
Local Attention	1.39	1.68	2.08	5.83	20.04	40.16	80.44	161.35	325.11	-
Linformer	1.51	1.42	1.56	1.67	3.67	6.99	13.63	26.77	53.36	117.56
Smyrf	3.38	4.93	9.07	17.66	34.94	69.55	138.72	277.41	-	-
LSformer	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
Block Sparse	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FlashAttention	**0.32**	**0.30**	0.83	2.37	7.95	30.77	119.98	473.65	1883.43	7513.01
Block-Sparse FlashAttention	0.34	0.34	**0.36**	**0.69**	**1.85**	**3.89**	**7.16**	**14.85**	**30.46**	**60.03**

Table 15: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.26	0.24	0.57	1.80	6.56	25.34	-	-	-	-
Megatron	0.27	0.27	0.56	1.88	6.56	-	-	-	-	-
Reformer	1.83	2.96	5.31	10.33	21.19	43.42	91.96	201.34	-	-
Local Attention	0.51	0.60	0.78	2.01	6.23	12.52	25.07	50.50	102.18	-
Linformer	0.47	0.37	0.49	**0.52**	1.37	2.65	5.12	10.13	20.25	44.16
Smyrf	2.12	2.01	3.15	5.97	11.83	23.36	46.48	92.72	-	-
LSformer	1.28	1.33	1.51	3.39	11.40	22.54	44.96	89.85	179.73	-
Block Sparse	1.03	1.00	1.72	2.39	5.96	17.88	-	-	-	-
Longformer	1.02	1.03	1.03	1.73	5.10	11.63	34.22	-	-	-
BigBird	0.99	1.03	1.01	1.58	5.36	12.27	35.56	-	-	-
FlashAttention	**0.10**	**0.10**	**0.22**	0.83	2.81	10.38	41.63	167.01	668.74	2678.11
Block-Sparse FlashAttention	0.54	0.51	0.68	0.61	**0.67**	**1.10**	**1.89**	**3.71**	**7.18**	**14.41**

Table 16: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.44	0.35	0.90	2.94	10.77	41.67	-	-	-	-
Megatron	0.28	0.33	0.92	2.94	10.80	-	-	-	-	-
Reformer	2.24	4.34	8.39	16.62	33.02	65.77	131.52	-	-	-
Local Attention	0.51	0.58	1.41	3.71	12.96	25.98	51.94	103.72	207.78	-
Linformer	0.84	0.74	0.79	0.85	2.28	4.37	8.66	17.02	33.78	-
Smyrf	1.27	2.56	4.90	9.66	19.16	38.13	76.17	152.39	-	-
LSformer	1.67	1.77	3.03	7.52	20.10	39.13	76.35	150.83	-	-
Block Sparse	1.27	1.36	2.15	3.04	7.27	21.18	-	-	-	-
Longformer	1.28	1.34	1.38	1.98	5.24	10.74	25.95	-	-	-
BigBird	1.48	1.47	1.50	1.81	5.57	11.38	27.43	-	-	-
FlashAttention	**0.15**	0.18	0.58	1.86	6.50	26.21	104.27	416.10	1661.92	6643.01
Block-Sparse FlashAttention	0.17	**0.17**	**0.17**	**0.40**	**1.10**	**2.04**	**4.43**	**9.33**	**18.28**	**37.31**

Table 17: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.66	0.67	1.43	4.82	17.47	67.29	-	-	-	-
Megatron	0.88	0.90	1.49	4.73	17.41	-	-	-	-	-
Reformer	4.06	7.28	13.68	26.98	54.27	109.39	223.80	-	-	-
Local Attention	1.09	1.40	1.99	5.61	19.23	38.62	77.30	154.63	311.12	-
Linformer	1.31	1.21	1.30	1.39	3.73	7.15	14.05	27.69	55.00	-
Smyrf	3.00	4.37	8.05	15.66	31.04	61.64	123.04	245.65	-	-
LSformer	3.07	3.17	4.31	10.89	31.54	61.78	121.56	240.94	-	-
Block Sparse	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FlashAttention	**0.35**	**0.36**	**0.80**	2.52	9.16	36.70	146.13	583.45	2332.01	9323.63
Block-Sparse FlashAttention	0.91	0.83	0.94	**0.92**	**1.83**	**3.50**	**7.02**	**13.56**	**26.71**	**53.92**

表14：按序列长度划分的各类精确/近似/稀疏注意力机制的前向传播 + 后向传播运行时间（毫秒），**已进行掩码处理**。表现最佳的选项以**粗体**标出，次优的选项则用下划线标注。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.80	0.81	2.08	7.23	27.51	107.58	-	-	-	-
Megatron架构	0.81	0.83	1.09	3.36	12.39	-	-	-	-	-
Reformer架构	4.16	7.46	14.06	27.68	55.66	112.15	229.37	-	-	-
局部注意力机制	1.39	1.68	2.08	5.83	20.04	40.16	80.44	161.35	325.11	-
Linformer模型	1.51	1.42	1.56	1.67	3.67	6.99	13.63	26.77	53.36	117.56
SMYRF模型	3.38	4.93	9.07	17.66	34.94	69.55	138.72	277.41	-	-
LSformer模型	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
块稀疏结构	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer模型	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird模型	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FlashAttention算法	**0.32**	**0.30**	0.83	2.37	7.95	30.77	119.98	473.65	1883.43	7513.01
分块稀疏FlashAttention算法	0.34	0.34	**0.36**	**0.69**	**1.85**	**3.89**	**7.16**	**14.85**	**30.46**	**60.03**

表15：按序列长度划分的各类精确/近似/稀疏注意力机制的前向传播运行时间（毫秒），**已包含丢弃机制**。表现最佳的以**粗体**标出，次优的则用下划线标示。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.26	0.24	0.57	1.80	6.56	25.34	-	-	-	-
Megatron架构	0.27	0.27	0.56	1.88	6.56	-	-	-	-	-
Reformer架构	1.83	2.96	5.31	10.33	21.19	43.42	91.96	201.34	-	-
局部注意力机制	0.51	0.60	0.78	2.01	6.23	12.52	25.07	50.50	102.18	-
Linformer模型	0.47	0.37	0.49	**0.52**	1.37	2.65	5.12	10.13	20.25	44.16
SMYRF模型	2.12	2.01	3.15	5.97	11.83	23.36	46.48	92.72	-	-
LSformer模型	1.28	1.33	1.51	3.39	11.40	22.54	44.96	89.85	179.73	-
块稀疏结构	1.03	1.00	1.72	2.39	5.96	17.88	-	-	-	-
Longformer模型	1.02	1.03	1.03	1.73	5.10	11.63	34.22	-	-	-
BigBird模型	0.99	1.03	1.01	1.58	5.36	12.27	35.56	-	-	-
FlashAttention算法	**0.10**	**0.10**	**0.22**	0.83	2.81	10.38	41.63	167.01	668.74	2678.11
分块稀疏FlashAttention算法	0.54	0.51	0.68	0.61	**0.67**	**1.10**	**1.89**	**3.71**	**7.18**	**14.41**

表16：按序列长度划分的各类精确/近似/稀疏注意力机制的后向传播运行时间（毫秒），**已包含丢弃机制**。表现最佳的用**粗体**标出，次优的则用下划线标出。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.44	0.35	0.90	2.94	10.77	41.67	-	-	-	-
Megatron架构	0.28	0.33	0.92	2.94	10.80	-	-	-	-	-
Reformer架构	2.24	4.34	8.39	16.62	33.02	65.77	131.52	-	-	-
局部注意力机制	0.51	0.58	1.41	3.71	12.96	25.98	51.94	103.72	207.78	-
Linformer模型	0.84	0.74	0.79	0.85	2.28	4.37	8.66	17.02	33.78	-
SMYRF模型	1.27	2.56	4.90	9.66	19.16	38.13	76.17	152.39	-	-
LSformer模型	1.67	1.77	3.03	7.52	20.10	39.13	76.35	150.83	-	-
块稀疏结构	1.27	1.36	2.15	3.04	7.27	21.18	-	-	-	-
Longformer模型	1.28	1.34	1.38	1.98	5.24	10.74	25.95	-	-	-
BigBird模型	1.48	1.47	1.50	1.81	5.57	11.38	27.43	-	-	-
FlashAttention算法	**0.15**	0.18	0.58	1.86	6.50	26.21	104.27	416.10	1661.92	6643.01
分块稀疏FlashAttention算法	0.17	**0.17**	**0.17**	**0.40**	**1.10**	**2.04**	**4.43**	**9.33**	**18.28**	**37.31**

表17：按序列长度划分的各类精确/近似/稀疏注意力机制的前向传播 + 后向传播运行时间（毫秒），**并已应用丢弃机制**。表现最佳的选项以**粗体**标出，次优的选项则用下划线标注。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.66	0.67	1.43	4.82	17.47	67.29	-	-	-	-
Megatron架构	0.88	0.90	1.49	4.73	17.41	-	-	-	-	-
Reformer架构	4.06	7.28	13.68	26.98	54.27	109.39	223.80	-	-	-
局部注意力机制	1.09	1.40	1.99	5.61	19.23	38.62	77.30	154.63	311.12	-
Linformer模型	1.31	1.21	1.30	1.39	3.73	7.15	14.05	27.69	55.00	-
SMYRF模型	3.00	4.37	8.05	15.66	31.04	61.64	123.04	245.65	-	-
LSformer模型	3.07	3.17	4.31	10.89	31.54	61.78	121.56	240.94	-	-
块稀疏结构	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer模型	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird模型	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FlashAttention算法	**0.35**	**0.36**	**0.80**	2.52	9.16	36.70	146.13	583.45	2332.01	9323.63
分块稀疏FlashAttention算法	0.91	0.83	0.94	**0.92**	**1.83**	**3.50**	**7.02**	**13.56**	**26.71**	**53.92**

Table 18: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.21	0.22	0.43	1.27	4.32	16.47	67.77	-	-	-
Megatron	0.24	0.26	0.42	1.33	4.28	-	-	-	-	-
Reformer	1.77	2.82	5.01	9.74	20.03	41.11	87.39	192.40	-	-
Local Attention	0.48	0.57	0.80	1.90	5.76	11.56	23.13	46.65	94.74	-
Linformer	0.46	0.36	0.45	**0.50**	1.09	2.09	4.01	7.90	15.70	35.40
Smyrf	1.94	1.96	3.01	5.69	11.26	22.23	44.21	88.22	-	-
LSformer	1.21	1.34	1.34	3.31	11.01	21.71	43.27	86.32	172.85	-
Block Sparse	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FlashAttention	**0.08**	**0.09**	**0.18**	0.68	2.40	8.42	33.54	134.03	535.95	2147.05
Block-Sparse FlashAttention	0.56	0.52	0.63	0.65	**0.61**	**0.96**	**1.69**	**3.02**	**5.69**	**11.77**

Table 19: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.26	0.29	0.78	2.44	8.82	33.87	-	-	-	-
Megatron	0.29	0.30	0.80	2.59	8.86	-	-	-	-	-
Reformer	2.18	4.21	8.14	16.12	32.02	63.84	127.60	-	-	-
Local Attention	0.51	0.64	1.28	3.60	12.52	25.08	50.22	100.23	200.66	-
Linformer	0.69	0.76	0.69	0.80	2.04	3.88	7.67	15.04	30.11	63.15
Smyrf	1.24	2.49	4.77	9.42	18.65	37.12	74.15	148.35	-	-
LSformer	1.68	1.61	3.02	7.40	19.72	38.27	74.89	147.99	-	-
Block Sparse	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FlashAttention	**0.11**	0.16	0.52	1.62	5.45	21.57	84.75	336.00	1338.56	5343.19
Block-Sparse FlashAttention	0.11	**0.12**	**0.16**	**0.38**	**1.20**	**2.34**	**4.69**	**9.10**	**18.74**	**37.04**

Table 20: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.67	0.70	1.18	3.67	13.22	50.44	-	-	-	-
Megatron	0.74	0.65	1.23	3.80	13.21	-	-	-	-	-
Reformer	3.93	7.01	13.15	25.89	52.09	105.00	215.13	-	-	-
Local Attention	1.09	1.27	1.99	5.38	18.32	36.77	73.67	147.29	296.35	-
Linformer	1.31	1.25	1.30	1.29	3.20	6.10	11.93	23.39	46.72	100.52
Smyrf	2.98	4.23	7.78	15.12	29.96	59.45	118.60	237.02	-	-
LSformer	3.03	3.05	4.26	10.70	30.77	60.15	118.33	234.94	-	-
Block Sparse	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FlashAttention	**0.31**	**0.31**	**0.73**	2.29	7.64	30.09	118.50	470.51	1876.08	7492.85
Block-Sparse FlashAttention	0.74	0.77	0.82	**0.88**	**1.71**	**3.21**	**6.56**	**12.60**	**24.93**	**50.39**

Table 21: Memory usage (MB) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	36	104	336	1184	4416	17024	-	-	-	-
Megatron	36	104	336	1184	4416	-	-	-	-	-
Reformer	377	754	1508	3016	6033	12067	24134	-	-	-
Local Attention	53	110	232	592	1696	3392	6784	13568	27136	-
Linformer	25	52	114	287	832	1652	3292	6572	13132	26252
Smyrf	217	434	868	1737	3474	6947	13894	27788	-	-
LSformer	72	152	333	796	2540	5068	10125	20240	-	-
Block Sparse	33	82	228	408	910	2401	-	-	-	-
Longformer	30	61	124	277	681	1370	2748	-	-	-
BigBird	33	66	131	294	708	1431	2872	-	-	-
FlashAttention	**22**	**44**	**104**	**209**	**418**	**836**	**1672**	**3344**	**6688**	**13376**
Block-Sparse FlashAttention	22	44	104	209	418	836	1672	3344	6690	13384

表18：按序列长度划分的各种精确/近似/稀疏注意力机制的前向传播运行时间（毫秒）。表现最佳的选项以**粗体**显示，次优的选项则用下划线标出。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.21	0.22	0.43	1.27	4.32	16.47	67.77	-	-	-
Megatron架构	0.24	0.26	0.42	1.33	4.28	-	-	-	-	-
Reformer架构	1.77	2.82	5.01	9.74	20.03	41.11	87.39	192.40	-	-
局部注意力机制	0.48	0.57	0.80	1.90	5.76	11.56	23.13	46.65	94.74	-
Linformer模型	0.46	0.36	0.45	**0.50**	1.09	2.09	4.01	7.90	15.70	35.40
SMYRF模型	1.94	1.96	3.01	5.69	11.26	22.23	44.21	88.22	-	-
LSformer模型	1.21	1.34	1.34	3.31	11.01	21.71	43.27	86.32	172.85	-
块稀疏结构	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer模型	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird模型	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FlashAttention算法	**0.08**	**0.09**	**0.18**	0.68	2.40	8.42	33.54	134.03	535.95	2147.05
分块稀疏FlashAttention算法	0.56	0.52	0.63	0.65	**0.61**	**0.96**	**1.69**	**3.02**	**5.69**	**11.77**

表19：不同序列长度下各类精确/近似/稀疏注意力机制的后向传播运行时间（毫秒）。表现最佳的选项以**粗体**标出，次优的选项则用下划线标示。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.26	0.29	0.78	2.44	8.82	33.87	-	-	-	-
Megatron架构	0.29	0.30	0.80	2.59	8.86	-	-	-	-	-
Reformer架构	2.18	4.21	8.14	16.12	32.02	63.84	127.60	-	-	-
局部注意力机制	0.51	0.64	1.28	3.60	12.52	25.08	50.22	100.23	200.66	-
Linformer模型	0.69	0.76	0.69	0.80	2.04	3.88	7.67	15.04	30.11	63.15
SMYRF模型	1.24	2.49	4.77	9.42	18.65	37.12	74.15	148.35	-	-
LSformer模型	1.68	1.61	3.02	7.40	19.72	38.27	74.89	147.99	-	-
块稀疏结构	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer模型	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird模型	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FlashAttention算法	**0.11**	0.16	0.52	1.62	5.45	21.57	84.75	336.00	1338.56	5343.19
分块稀疏FlashAttention算法	0.11	**0.12**	**0.16**	**0.38**	**1.20**	**2.34**	**4.69**	**9.10**	**18.74**	**37.04**

表20：按序列长度划分的各类精确/近似/稀疏注意力机制的前向传播 + 后向传播运行时间（毫秒）；表现最佳的选项以**粗体**标出，次优的选项则用下划线标注。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	0.67	0.70	1.18	3.67	13.22	50.44	-	-	-	-
Megatron架构	0.74	0.65	1.23	3.80	13.21	-	-	-	-	-
Reformer架构	3.93	7.01	13.15	25.89	52.09	105.00	215.13	-	-	-
局部注意力机制	1.09	1.27	1.99	5.38	18.32	36.77	73.67	147.29	296.35	-
Linformer模型	1.31	1.25	1.30	1.29	3.20	6.10	11.93	23.39	46.72	100.52
SMYRF模型	2.98	4.23	7.78	15.12	29.96	59.45	118.60	237.02	-	-
LSformer模型	3.03	3.05	4.26	10.70	30.77	60.15	118.33	234.94	-	-
块稀疏结构	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer模型	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird模型	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FlashAttention算法	**0.31**	**0.31**	**0.73**	2.29	7.64	30.09	118.50	470.51	1876.08	7492.85
分块稀疏FlashAttention算法	0.74	0.77	0.82	**0.88**	**1.71**	**3.21**	**6.56**	**12.60**	**24.93**	**50.39**

表21：不同序列长度下各类精确/近似/稀疏注意力机制的内存占用情况（单位：MB）。表现最佳的选项以**粗体**标出，次优的选项则用下划线标示。

注意力机制	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力机制	36	104	336	1184	4416	17024	-	-	-	-
Megatron架构	36	104	336	1184	4416	-	-	-	-	-
Reformer架构	377	754	1508	3016	6033	12067	24134	-	-	-
局部注意力机制	53	110	232	592	1696	3392	6784	13568	27136	-
Linformer模型	25	52	114	287	832	1652	3292	6572	13132	26252
SMYRF模型	217	434	868	1737	3474	6947	13894	27788	-	-
LSformer模型	72	152	333	796	2540	5068	10125	20240	-	-
块稀疏结构	33	82	228	408	910	2401	-	-	-	-
Longformer模型	30	61	124	277	681	1370	2748	-	-	-
BigBird模型	33	66	131	294	708	1431	2872	-	-	-
FlashAttention算法	**22**	**44**	**104**	**209**	**418**	**836**	**1672**	**3344**	**6688**	**13376**
分块稀疏FlashAttention算法	22	44	104	209	418	836	1672	3344	6690	13384